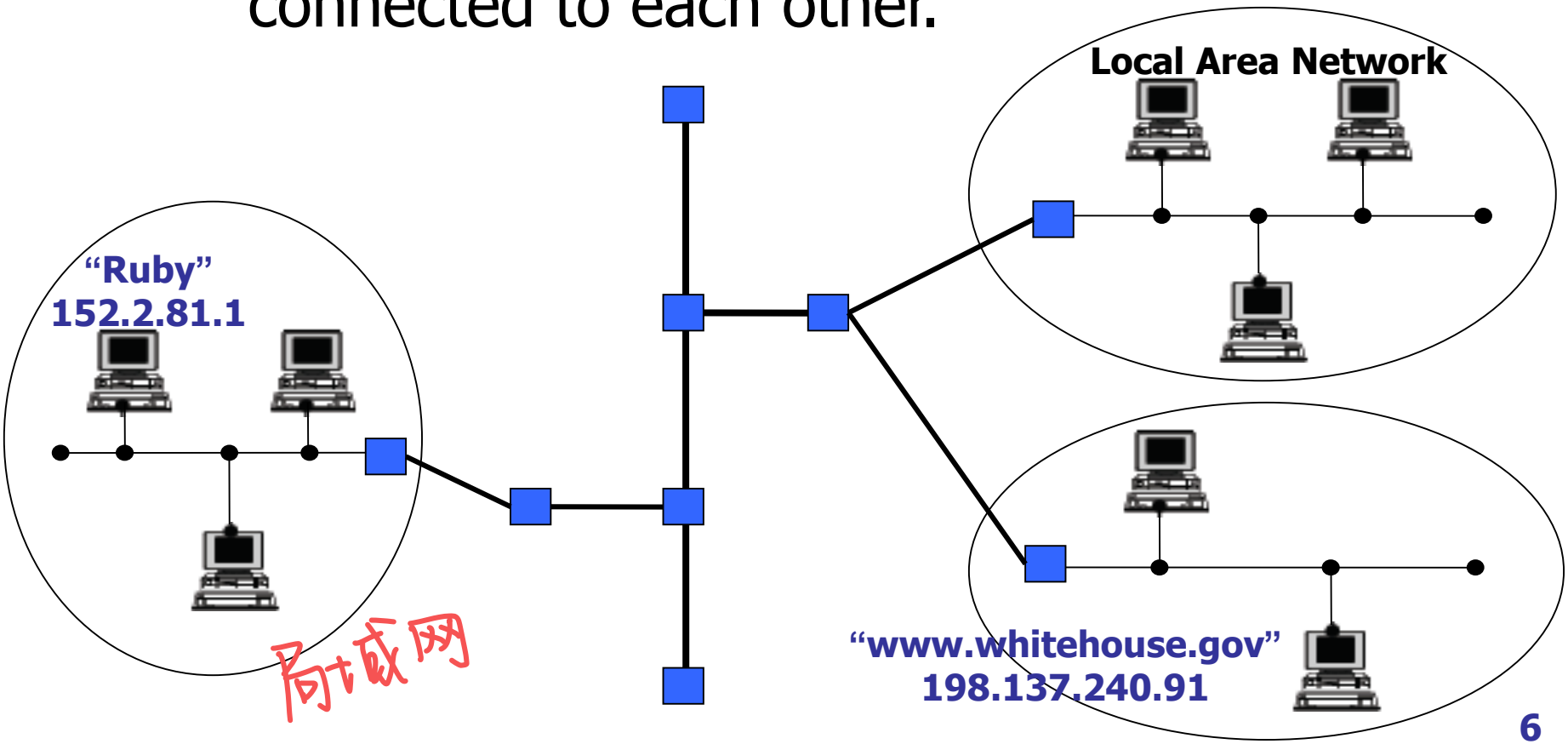
Protocol	Transport layer protocol	Well-known port
DHCP	UDP	Server:67; Client:68
DNS	Mostly UDP, sometimes TCP	53
Telnet	TCP	23
TFTP	UDP	69
FTP	TCP	Control connection: 21
		Data connection in active mode: 20
SMTP	TCP	25
POP3	TCP	110
HTTP	TCP	80 server
SNMP	UDP	Get/set: 161 Polling Trap: 162

internet vs. Internet

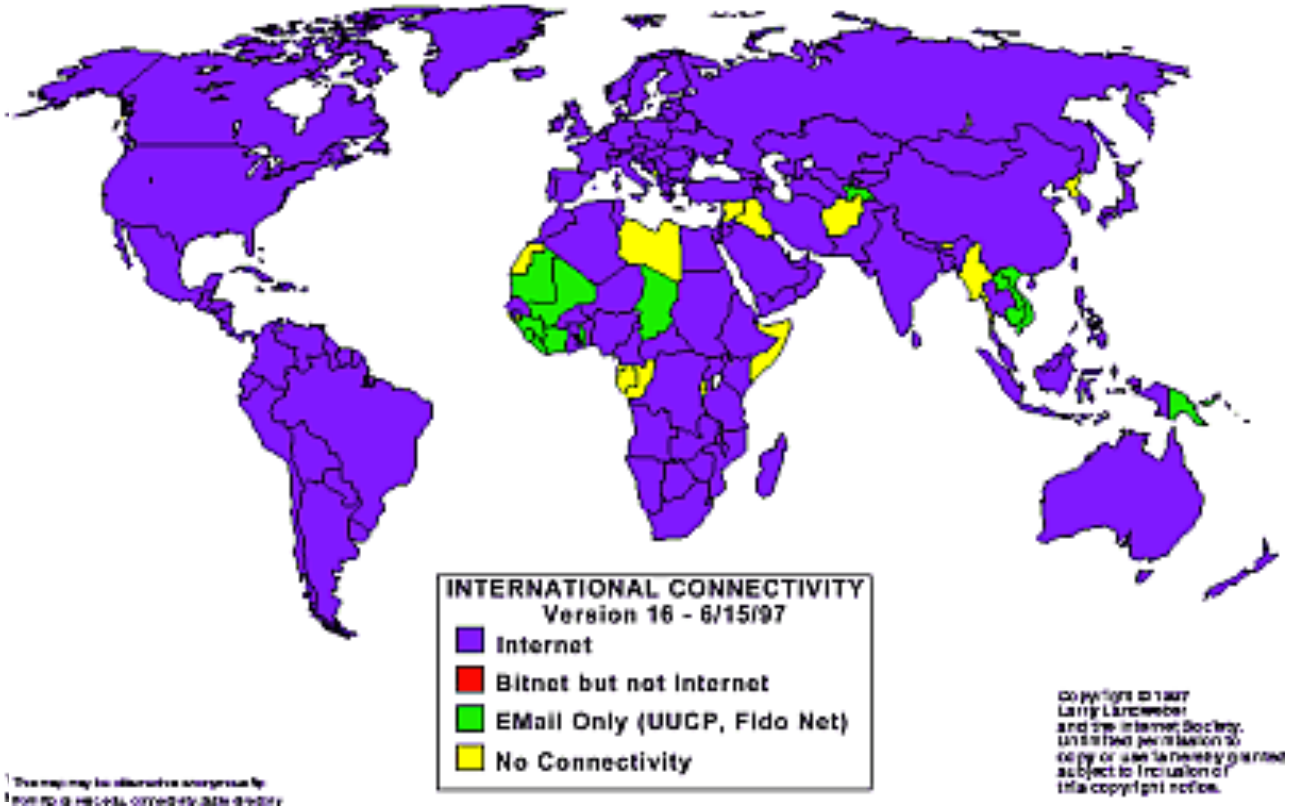
- internet: a set of computer networks that are connected to each other.



internet vs. Internet

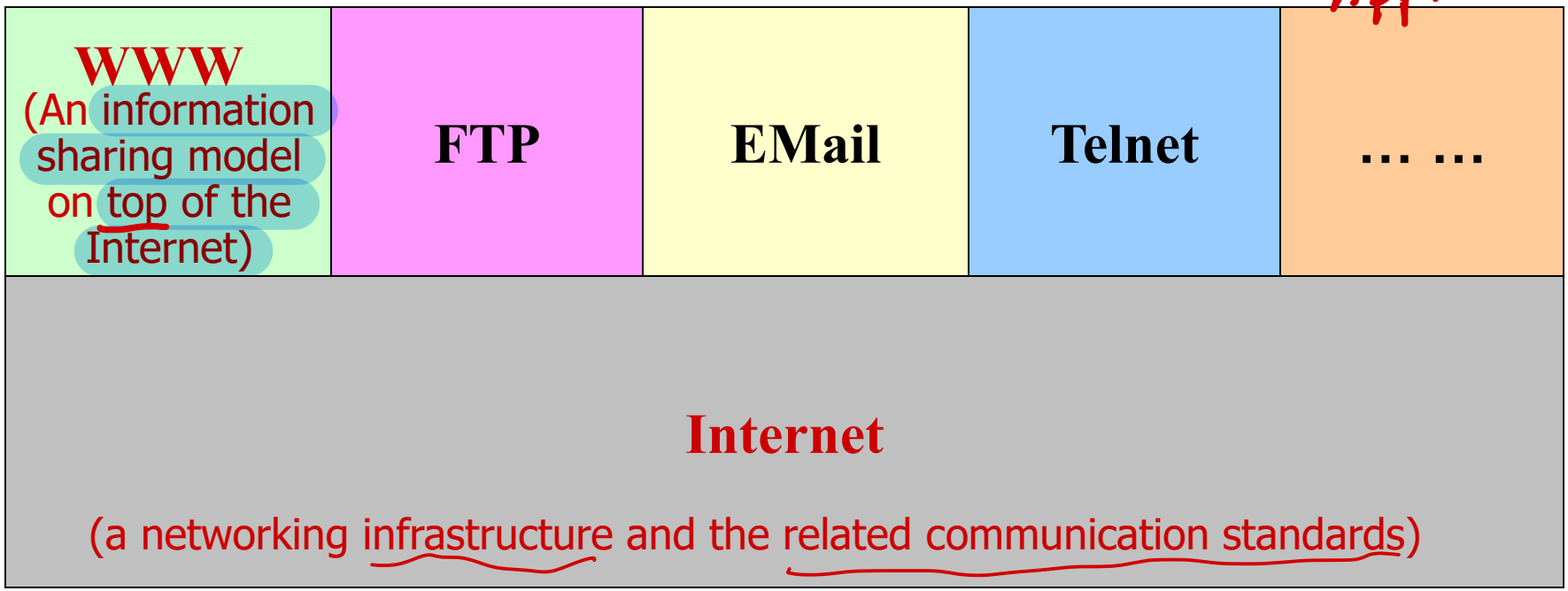
全球系统且使用TCP/IP

- **Internet:** a worldwide sets of networks that interoperate using TCP/IP protocols.



Internet vs. WWW

Application





Q2: How does the Internet work?
—— the components of the Internet

- Internet applications
  - telnet, ftp, Email, Web browser etc.
- Internet protocols
  - TCP/IP, FTP, SMTP,HTTP etc.
- Internet addresses
  - IPv4, IPv6, Domain Name System
- Physical infrastructure
  - hosts, routers, wiring

Q2: How does the Internet work?
—— the components of the Internet

- Internet protocols
- Protocol = A set of rules for communicating

Network layer

- Internet Protocol (IP)
  - Basic data transport: the glue of the Internet
  - Unreliable delivery
  - Versions
    - IPv4 with 32 bit/4 Byte address
    - IPv6 with 128 bit/16 Byte address

unreliable

All Internet Applications use at least IP, most use TCP/UDP and IP

Transport layer

- Transmission Control Protocol (TCP)
  - Reliable data transmission
  - Connection-oriented
- User Datagram Protocol (UDP)
  - Unreliable data transmission
  - Connectionless-oriented

+IP → 可靠传输

四次挥手. 三次握手

速度快. 实时性强.

Q2: How does the Internet work?
—— the components of the Internet

- Internet Protocols
- *Application protocols*
  - File Transfer Protocol (FTP)
    - Used by file exchange applications
  - Simple Mail Transfer Protocol (SMTP)
    - Used by email applications
  - HyperText Transfer Protocol (HTTP)
    - Used by WWW applications
  - … …

Q2: How does the Internet work?
important concepts

- Layered networking model
- Client-server paradigm

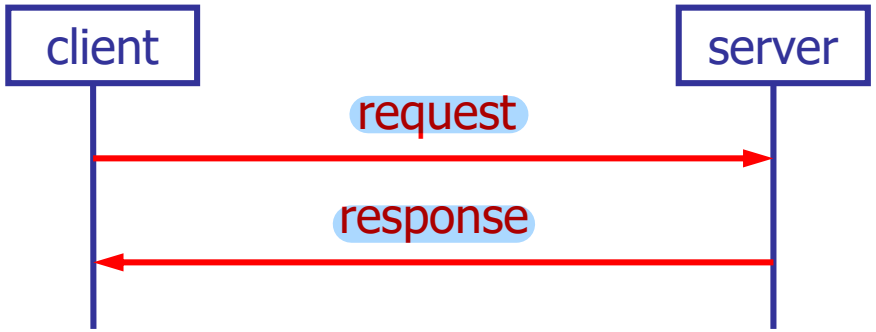
Q2: How does the Internet work?
—— important concepts

- Layered networking model
  - ISO's 7 layer model



Q2: How does the Internet work?
—— important concepts

- Client-server paradigm
  - Client – the user of the service
    - Initiates interaction through requests
  - Server – the provider of the service
    - Must be listening
    - Waits and responds to the incoming requests



  - Clients and servers need a protocol that defines the interaction between them

Basic Network Definitions

—— Terms for network devices

Node

- a device that is connected as part of a network with a network address
  - E.g. Computer, PDA, Cell Phone, router, switch, bridge etc.

Host Node

- the computer attached directly to the Internet (eg: ISPs and NSPs) - end point of a network

Link

- the inter-connection between network devices

Network Component

- the equipment that is part of the network infrastructure
  - E.g. Gateway, router, bridge/switch, hub/repeater

Basic Network Definitions

—— Terms for network performance parameters

Bandwidth

- Indicates how much stuff you can send through a connection
- bps (bit per second)
- Bps (Byte per second)

Delay (Latency)

- is an expression of how much time it takes for a packet of data to get from one designated point to another
- Contributors
  - Propagation
  - Transmission
  - Processing
  - Storage(Queuing)

Jitter

- The variation in delay

Error Rate

- The probability of the data units which are transmitted in error, are lost or are retransmitted
- BER (Bit Error Rate)
- FER (Frame Error Rate)
- PER (Packet Error Rate)

Throughput: the average rate of successful message delivery over a communication channel.

Sources of Delay

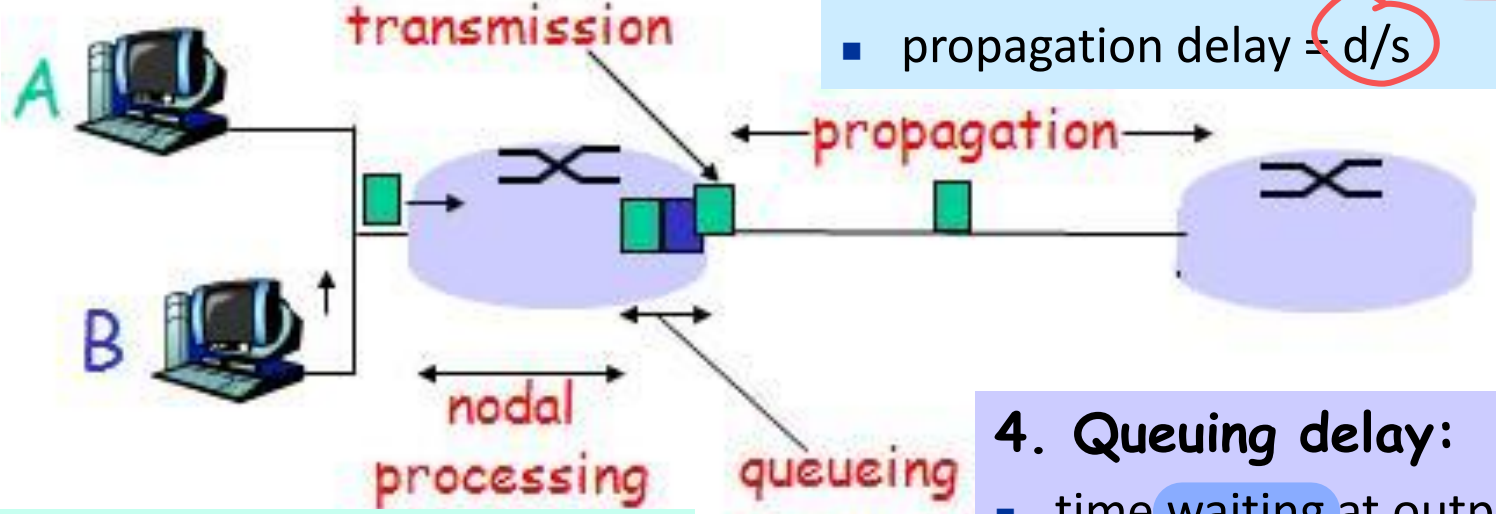
1. Transmission delay:

- R=link bandwidth (bps)
- L=packet length (bits)
- time to send bits into link = L/R

2. Propagation delay:

- d = length of physical link
- s = propagation speed in medium
- propagation delay = d/s

光纤/铜线...



3. Nodal processing delay:

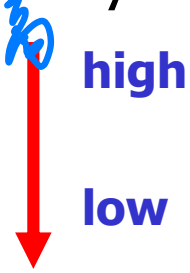
- check bit errors
- determine output link

4. Queuing delay:

- time waiting at output link for transmission
- depends on congestion level of router

Basic Network Definitions
—— Terms for network performance parameters

- Other similar parameters used for QoS (Quality of Service)
  - Throughput : the average rate of successful message delivery over a communication channel (*wikipedia*)
    - PLR (Packet Loss Rate)
- Different applications have different QoS requirements
  - E.g., four application classes defined by 3GPP according to their sensitivity to delay
    - Session Class
    - Interactive Class
    - Streaming Class
    - Background Class



Basic Network Definitions

—— Terms for network performance parameters

Class	Requirements	Examples
Session Class	Low delay, low jitter, without high requirements on BER	VoIP Video conference
Interactive Class	Low BER, low response delay, without high requirements on jitter	Web browsing
Streaming Class	Low jitter, without high requirements on delay and BER	Video/audio streaming
Background Class	Low BER, without high requirements on delay and jitter	Email

Basic Network Definitions
—— Terms for network types

- According to the switching function in the network
  - Circuit switching network
  - Message switching network
  - Packet switching network
  - Hybrid switching network

用以上方法的2种 or 3种

Basic Network Definitions
—— Terms for network types

- Different channel access technologies
  - Multi-access means **shared medium**
    - Many end-systems share the same physical communication resources (wire, frequency, etc.)
    - There must be some arbitration mechanism
    - Complex channel access control, efficient resource usage
    - Example: LANs
  - point-to-point ← 长距离通信（可检测冲突）
    - Between two points in the network, there must exists a physical channel
    - No contention or collision 无争用和冲突
    - Simple access control, bandwidth waste
    - Example: WANs

Basic Network Definitions
—— Terms for network types

- According to the range of the network
  - WAN (Wide Area Network) - network that spans a large geographic area
  - MAN (Metropolitan Area Network) - network that spans a medium area such as a campus to a city
  - LAN (Local Area Network) - network that spans a limited area such as a lab, or a building
  - PAN (Personal Area Network) – network that spans a small space such a room, less than 10m

Basic Network Definitions

—— Terms for network types

LAN

LAN (Local Area Network)

Features

- Connects computers that are physically close together
- Range: < 1 km
- high speed
- multi-access

内部不用路由器

e.g. network in a lab

Technologies

- Ethernet: IEEE 802.3, 10 M, 100M, 1000M/1G, 10G, 100Gbps
- Wireless LAN: IEEE802.11b/a/g/n/ac
- Token Ring 16 Mbps
- FDDI 100 Mbps

Basic Network Definitions

—— Terms for network types

MAN (Metropolitan Area Network)

Features

- Larger than a LAN and smaller than a WAN
- Range: < 10 km
- Example: city network / campus network
- multi-access

Technologies

- coaxial cable
- Microwave, IEEE802.16/WiMAX

Basic Network Definitions

—— Terms for network types

WAN (Wide Area Network)

Features

- Connects computers that are physically far apart. "long-haul network"
- Traditionally slower and less reliable than a LAN
- Range: 10 - 1000 km
- Point-to-point ring or partial mesh ← channel access

e.g. in country

Technologies

波分复用

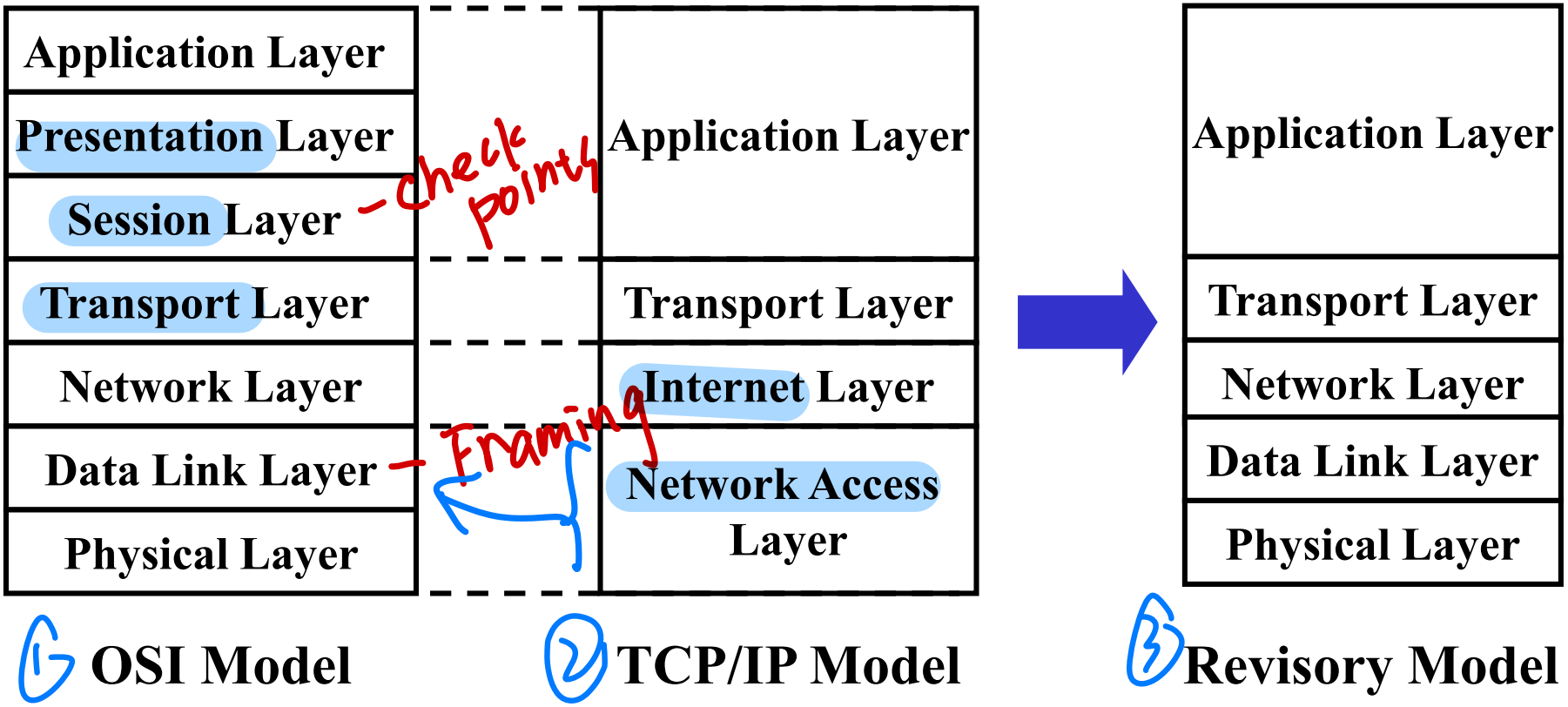
- D-WDM, SDH + ATM, Frame Relay
- PSTN Telephone lines
- Satellite communications
- Cellular mobile communications

Basic Network Definitions
—— Terms for network types

- According to the user of the network
  - Public network 公网
    - The large scale network built by the telecommunication companies
    - All the users can use the network as long as they pay the money
  - Private network 私网
    - The network built by a certain agency for its special requirements
    - Only providing services to the user inside this agency
    - E.g., the military network, the railway network

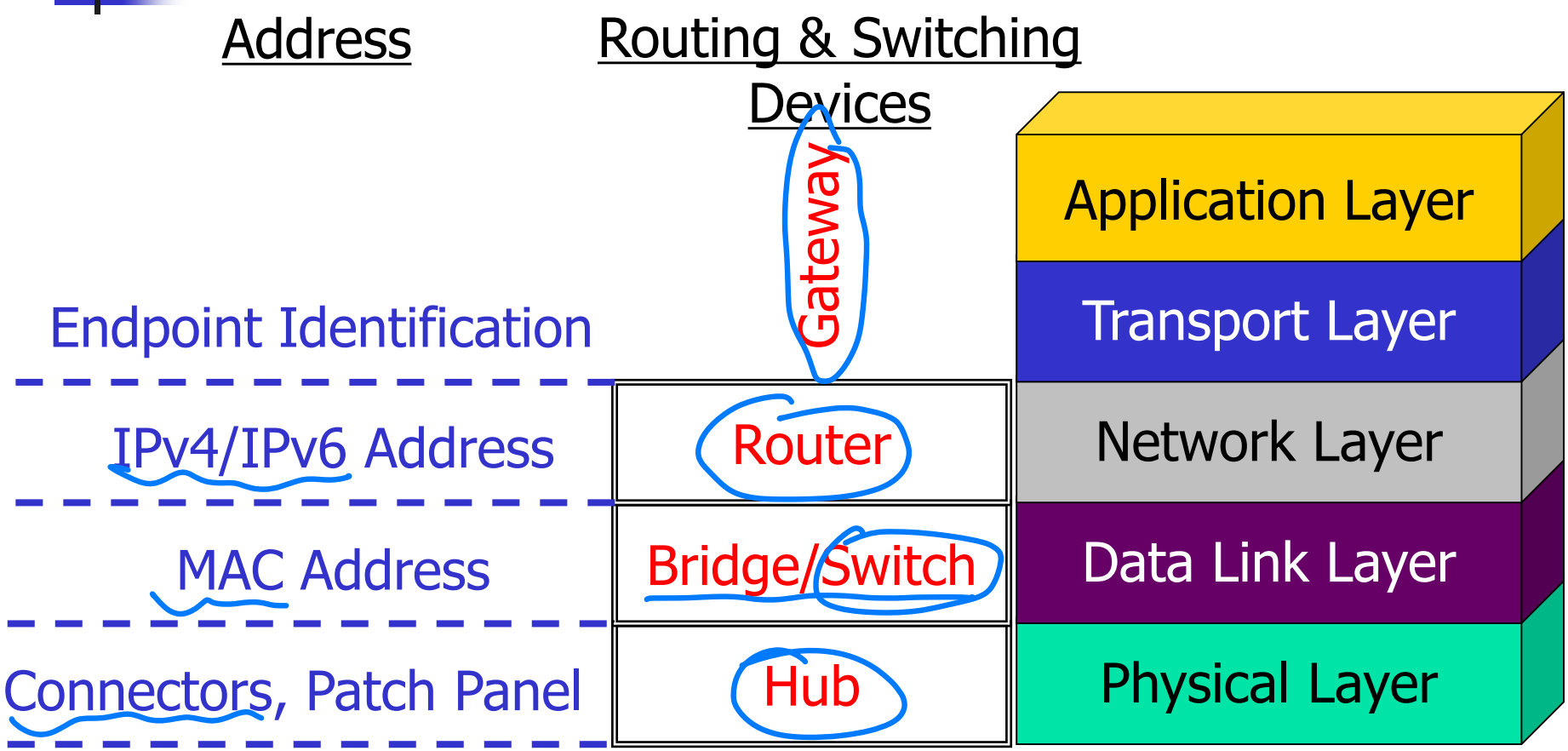
Layered Architecture

——Revisory Model 5云模型



Layered Architecture

—— devices and addresses at different layers



Basic Concepts - *definitions*

Process

- A process is an instance of a program that is being executed by the operating system.

program的动态版

动态的. 执行结束释放内存

System Call

- Linux/Unix kernel provides a limited number (typically between 60 and 200) of direct entry points through which an active process can obtain services from the Kernel.

接口

File Descriptor

- A file descriptor is a small integer used to identify a file that has been opened for I/O operation.

文件标识等
(非负整数)

Signal

- A signal is a notification to a process that an event has occurred.

传递消息.
不能传数据

e.g. 中止信号

例: control+C
中止

Basic concepts - *process*

- One of the most basic abstractions in Unix (the other one is File)
- process ≠ program
  - Program：a file containing instructions to be executed, static
  - Process: an instance of a program in execution, live entity
  - One program can have multiple processes
  - One process can invoke multiple programs
- Alias: task, job

多进程（并发）

可调用其他程序

Basic concepts – *process*

- PID (Process ID): Every process has a unique PID. The PID is an integer, typically in the range 0 through 32,767.
- PPID (Parent PID): Every process has a parent process ID.
- Special process
  - PID = 1: init process
  - PID = 0: special kernel process (e.g., idle/swapper process)
  - PID = 2: special kernel process (e.g., page daemon process)

Basic concepts – *process*

- Related system calls
  - fork(): to create a child process
  - getpid(): to obtain the PID of a process
  - getppid(): to obtain the PPID(Parent Process ID) of a process
  - exec(): often used after fork() to load another process
    - execl(), execv(), execle(), execve(), execlp(), execvp()
  - exit(): to terminate a process and release all the resources

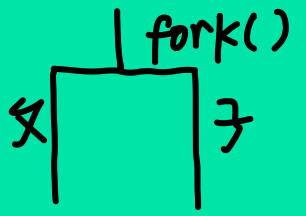
Basic concepts – *process*

- Simple sample program of fork() – fork1.c

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
        pid_t t;
        t=fork();
        printf("fork returned %d\n",t);
        exit(0);
}
```

```
$ gcc fork1.c -o fork1
$ ./fork1
fork returned 0
fork returned 22770
```



Basic concepts – *process*

- Complete sample program of fork() – fork2.c

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main (void) {

        pid_t t;

        printf("Original program, pid=%d\n", getpid());
        t = fork();
        if (t == 0) {
                printf("In child process, pid=%d, ppid=%d\n",
                        getpid(), getppid());
        } else {
                printf("In parent, pid=%d, fork returned=%d\n",
                        getpid(), t);
        }
}
```

```
Original program, pid=987
In child process, pid=988, ppid=987
In parent, pid=987, fork returned=988
```

Basic concepts – *process*

- Sample program of exec() – exec1.c

```
#include <unistd.h>
#include <stdio.h>

int main (void) {

        char *arg[] = { "/bin/ls", 0 };

        /* fork, and exec within child process */
        if (fork() == 0) {
          printf("In child process:\n");
          execv(arg[0], arg);
          printf("I will never be called\n");
        }
        printf("Execution continues in parent process\n");
}
```

```
[shiyan@localhost examples-for-ia]$ ./exec1
In child process:
Execution continues in parent process
[shiyan@localhost examples-for-ia]$ exec1  exec1.c
fork1  fork1.c  fork2  fork2.c
```

若改成 "/bin/pwd"...
则执行pwd (子进程改变了)

Basic concepts – *file descriptor*

- A file descriptor (an integer) is returned when a file is opened or created, and is used as an argument when later the file is read or written

- File descriptors are assigned by the kernel when the following system calls are successful
  - open
  - creat
  - dup
  - pipe
  - fcntl

fd=0 : stdin
fd=1 : stdout
fd=2 : stderr

Basic concepts – *file descriptor*

- Related system calls
  - open
    - Be used to open or create a file
    - int open (char *pathname, int oflag, int mode);
  - close
    - Be used to close a file
    - int close (int filedes);
  - read
    - Be used to reading the data from an opened file
    - int read (int filedes, char *buff, unsigned int nbytes);
  - write
    - Be used to writing data into an opened file
    - int write (int filedes, char *buff, unsigned int nbytes);

File descriptor or -1

0 or -1

Bytes of data that are read, 0 or -1

Bytes of data that are written or -1

Name of the file

O_RDONLY
O_WRONLY
O_RDWR
...
(<fcntl.h>)

Only used when creating a file to indicate the access authority

Basic concepts – *file descriptor*

- Related system calls
  - lseek
    - Be used to locate in a file
    - long lseek (int filedes, long offset, int whence);
  - dup
    - Be used to duplicate a file descriptc
    - int dup (int filedes);
    - int dup2 (int filedes, int filedes2);
  - fcntl
    - Be used to change the properties o… already open
    - int fcntl (int filedes, int cmd, int arg);

The new offset in the file or -1

The new file descriptor or -1

Depending on cmd or -1

SEEK_SET
SEEK_CUR
SEEK_END

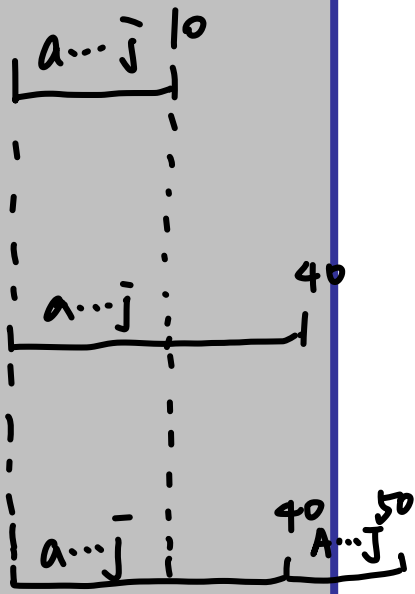
Indicate the different functions of fcntl():
F_DUPFD,
F_GETFD/F_SETFD,
F_GETFL/F_SETFL,
F_GETOWN/F_SETOWN,
F_GETLK/F_SETLK/F_SETLKW

```
dup(filedes); ⟷ fcntl(filedes,F_DUPFD,0);
```

Basic concepts – *file descriptor*

- Sample program of lseek() – lseek1.c

```
        if (write(fd,buf1,10)!=10)
        {       printf("buf1 write error\n");
                exit(1);}
         /*offset now = 10 */
        if (lseek(fd,40,SEEK_SET)==-1)
        {       printf("lseek error\n");
                exit(1);}
         /*offset now = 40 */
        if (write(fd,buf2,10)!=10)
        {       printf("buf2 write error\n");
                exit(1);}
         /*offset now = 50 */
        exit(0);
}
```



Basic concepts – *file descriptor*

- Sample program of read() and write() – readwrite1.c

```
#include <fcntl.h>
#include <unistd.h>
int main(void)
{
        char quit='.';
        char buf[10];
        int fd;
        if((fd = open("out.out",O_RDWR | O_CREAT,0))==-1)
                printf("Error in opening\n");
        while(buf[0]!=quit)
        {
                read(0,buf,1);
                write(fd,buf,1);
                write(1,buf,1);
        }
        close(fd);
}
```

可读可写 flag1　flag2　权限为0　创建

标准输入(从键盘)　写入文件　标准输出(从屏幕)

若输入"."输出"."然后退出

Basic concepts – *signal*

- Signals are some time called "Software interrupts"
- Signals can be sent from one process to another or from kernel to a process
- Header file: <signal.h>
- The names of the signals begin with SIG
  - SIGALRM: alarm clock timeout 定时器.
  - SIGINT: Interrupt character (Ctrl-C) is typed

Basic concepts – *signal*

Five conditions that generate signals

1. **Kill system call**
   - the system call ***kill*** allows a process to send a signal to another process or to itself
2. **Kill command**
   - the command ***kill*** is also used to send a signal
   - often used to terminate a background process out of control
3. **Certain terminal characters**
   - e.g., the interrupt character (typically control-C or Delete) terminates a process that is running - it generates a SIGINT signal
4. **Certain hardware conditions**
   - the hardware detects these conditions and then notifies the kernel. E.g., invalid storage access - SIGEGV
5. **Certain software conditions**
   - the kernel notices these conditions and generates the signal. E.g., SIGALRM

Basic concepts – *signal*

What can a process do with a signal?

1. **Catch the signal**
   - A process can provide a function that is called whenever a specific type of signal occurs. This function is called **handler**.

2. **Ignore the signal**
   - A process can choose to ignore a signal
   - Two signals that can not be ignored: SIGKILL, SIGSTOP

3. **Execute the default action**
   - A process can allow the default to happen
   - default actions of most signals are to terminate the process

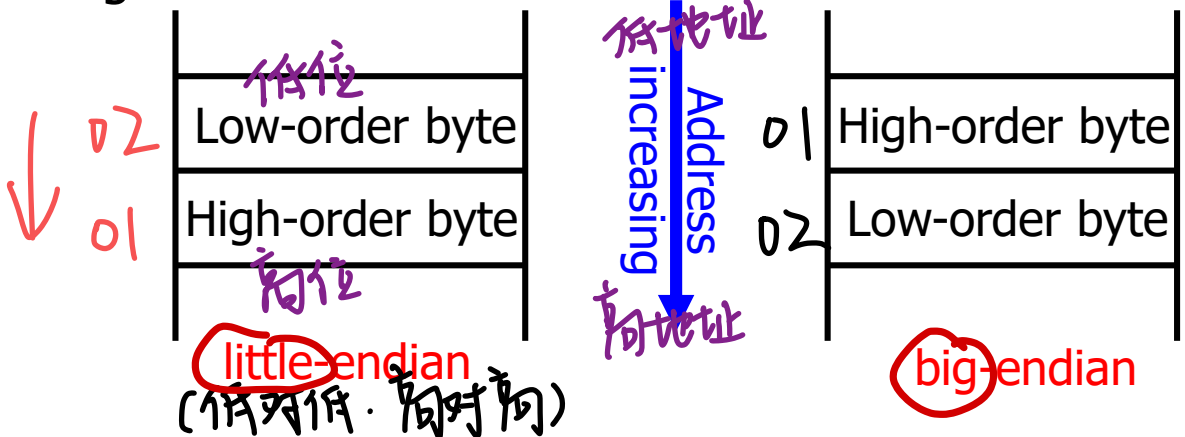
IP Addresses (1)

- 32-bit IP addresses are stored in an *IP Address structure*
  - <netinet/in.h>

```
/* Internet address.  */
typedef uint32_t in_addr_t;
struct in_addr {
    in_addr_t s_addr;
  };
```

```
/*Defined in <stdint.h>*/
typedef unsigned int uint32_t;
```

- Two ways to store multi-byte integers
  - Big-endian vs. little-endian



IP Addresses (2): Host byte order vs. network byte order

- Host byte order is machine-dependent
  - You can see it in <bits/endian.h>
  - A program used to output the host byte order
- Network byte order is machine-independent (*big-endian*)
- Byte order conversion functions
  - **htonl**: host byte order → network byte order for ***long int***
  - **htons**: host byte order → network byte order for ***short int***
  - **ntohl**: network byte order → host byte order for ***long int***
  - **ntohs**: network byte order → host byte order for ***short int***

Domain Name System (1)

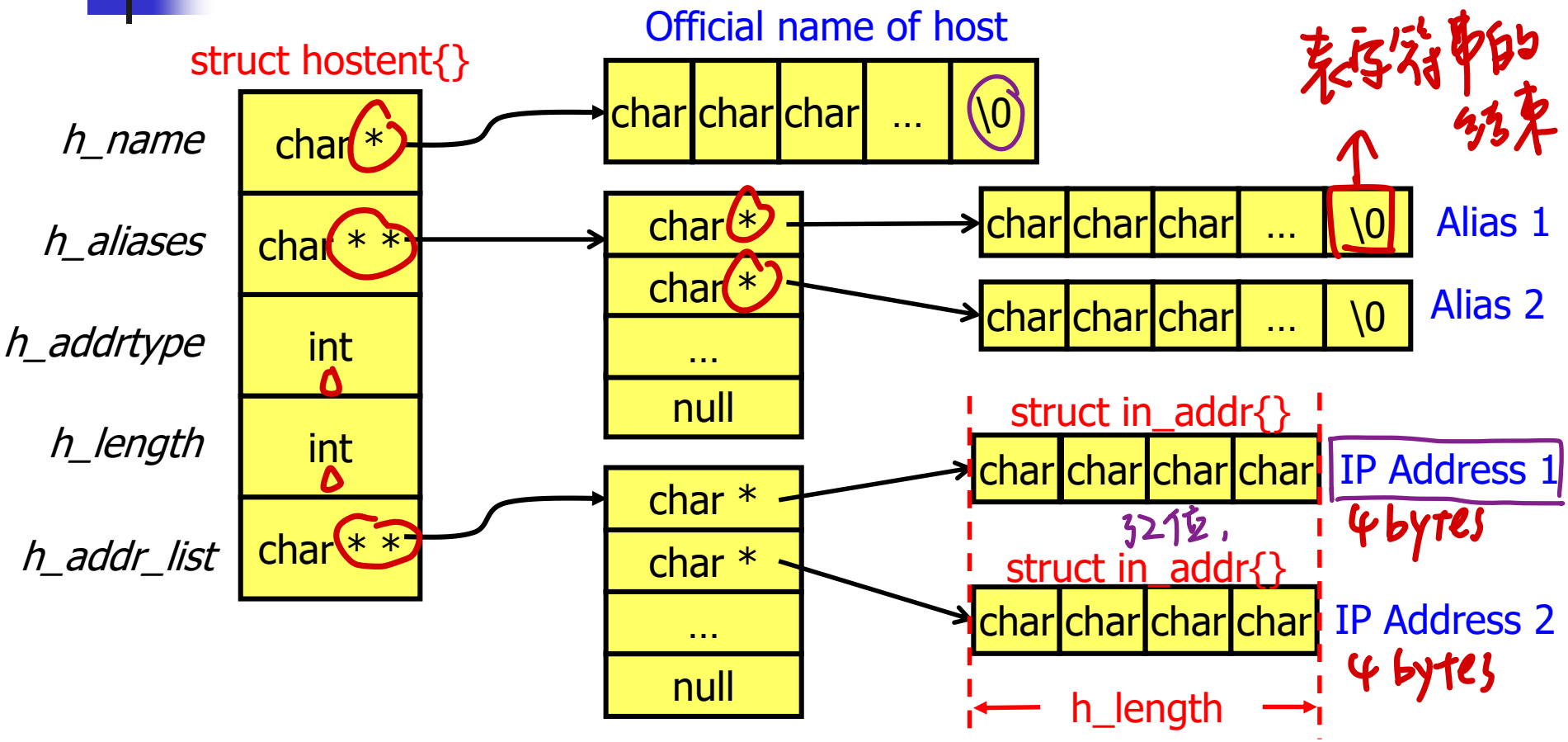
- The Internet maintains a mapping between IP addresses and domain names in a huge worldwide distributed database called *DNS*.
  - Conceptually, programmers can view the DNS database as a collection of millions of *host entry structures*
  - <netdb.h>

Domain Name System (2): Host Entry Structure

h_addr_list:
An array of pointers to IP addresses for the host (in network byte order), terminated by a NULL pointer.

```
/* Description of data base entry for a single host. */
struct hostent
{
  char *h_name;            /* Official name of host.   */
  char **h_aliases;        /* Alias list.              */
  int h_addrtype;          /* Host address type.       */
  int h_length;            /* Length of address.       */
  char **h_addr_list;      /* List of addresses from name
                              server.*/
#define h_addr  h_addr_list[0]  /* The first address in
                                   the address list.   */
};
```

Domain Name System (3): Host Entry Structure



Domain Name System (5)

- Functions for retrieving host entries from DNS
  - gethostbyname: query key is a DNS domain name.

通过域名查IP

```
#include <netdb.h>
struct hostent *gethostbyname (const char *hostname);
```

返回值为结构体

  - gethostbyaddr: query key is an IP address.

```
#include <netdb.h>
struct hostent * gethostbyaddr (const char *addr, int len, int family );
```

AF_INET for IPv4

Sockets Interface – definitions(1)

- What is a socket?
  - To the kernel, a socket is an endpoint of communication.
  - To an application, a socket is a **file descriptor** that lets the application read/write from/to the network.
    - Remember: All Unix I/O devices, including networks, are modeled as files.
- Clients and servers communicate with each other by reading from and writing to socket descriptors.
- The main distinction between regular file I/O and socket I/O is how the application "opens" the socket descriptors.

File Model in Unix/Linux

- In Unix/Linux, all I/O devices are treated as files
  - Identified with File Descriptors
  - File operations
    - open
    - close
    - lseek
    - read
    - write
    - …

Sample File Descriptor Table
(One per Process)

0	stdin
1	stdout
2	stderr
3	file
4	device
5	socket
..	…

Sockets Interface – definitions(2)

- **Internet-specific socket address** (bits/socket.h)

```
struct sockaddr_in {
   unsigned short sin_family; /* address family (always AF_INET) */
   unsigned short sin_port; /* port num in network byte order */
   struct in_addr sin_addr; /* IP addr in network byte order */
   unsigned char sin_zero[8]; /* pad to sizeof(struct sockaddr) */
};
```

  - **Address family**: Domains refer to the area where the communicating processes exist. Commonly used domains include:
    - AF_UNIX: for communication between processes on one system;
    - AF_INET (IPv4): for communication between processes on the same or different systems using the DARPA standard protocols (IP/UDP/TCP)
    - AF_INET6 (IPv6)
    - AF_LOCAL (Unix domain)
    - AF_UNSPEC (the importance will be explained later)
    - …

Sockets Interface – definitions(3)

- **Generic socket address** (<sys/socket.h>)

```
struct sockaddr {
   unsigned short sa_family; /* protocol family */
   char sa_data[14]; /* protocol-specific address,
                        up to 14 bytes. */
};
```

- **Protocol family**
  - PF_LOCAL: Local to host, pipes and file-domain
  - PF_UNIX: Old BSD name for PF_LOCAL
  - **PF_INET: IP protocol family**
  - PF_AX25: Amateur radio AX.25
  - PF_IPX: Novell internet protocol
  - PF_INET6: IP version 6
  - PF_ATMSVC: ATM SVCs
  - PF_APPLETALK: Appletalk DDP
  - …

Sockets Interface – definitions(4)

- Generic socket address and Internet-specific socket address
  - Pointer to generic socket address is used for address arguments to **connect(), bind() and accept()**
  - Must cast Internet-specific socket address (`sockaddr_in *`) to generic socket address (`sockaddr *`) for connect, bind, and accept

```
struct sockaddr_in serv;
/* fill in serv{}*/
bind (sockfd, (struct sockaddr *)&serv , sizeof(serv));
```

Sockets Interface – types(1)

- Stream Socket
  - Service: reliable (i.e. sequenced, non-duplicated, non-corrupted) bidirectional delivery of byte-stream data
  - Metaphor: a phone call
  - int s = socket (PF_INET, SOCK_STREAM, 0);
- Datagram Socket
  - Service: unreliable, unsequenced datagram
  - Metaphor: sending a letter
  - int s = socket (PF_INET, SOCK_DGRAM, 0);
- Raw Sockets Service
  - allows user-defined protocols that interface with IP
  - Requires `root` access
  - Metaphor: playing with an erector set
  - int s = socket (PF_INET, SOCK_RAW, protocol);
- **SOCK_STREAM and SOCK_DGRAM are the most common types of sockets used within UNIX/Linux**

System Calls – Socket Operation

- socket()
  - returns a socket descriptor
- bind()
  - What address I am on / what port to attach to
- connect()
  - Connect to a remote host
- listen()
  - Waiting for someone to connect to my port
- accept()
  - Get a socket descriptor for an incoming connection
- send() and recv()
  - Send and receive data over a connection
- read(), write()
  - Read from / Write to a particular socket, similar to recv()/ send()
- sendto() and recvfrom()
  - Send and receive data without connection
- close() and shutdown()
  - Close a connection Two way / One way

只有TCP有 UDP无

UDP

用shutdown 关单一方向(自己发送的方向)

不发也不收

System Calls – Byte Order Conversion

- htonl()
  - host byte order → network byte order for ***long int***
- htons()
  - host byte order → network byte order for ***short int***
- ntohl()
  - network byte order → host byte order for ***long int***
- ntohs()
  - network byte order → host byte order for ***short int***

System Calls – Address Formats Conversion

- **inet_aton()**
  - IP address in numbers-and-dots notation (ASCII string) → IP address structure in network byte order
- **inet_addr()**
  - same function with inet_aton()
- **inet_ntoa()**
  - IP address structure in network byte order → IP address in numbers-and-dots notation (ASCII string)
- **inet_pton()**
  - Similar to inet_aton() but working with IPv4 and IPv6
- **inet_ntop()**
  - Similar to inet_ntoa() but working with IPv4 and IPv6

System Calls – Name and Address Operation

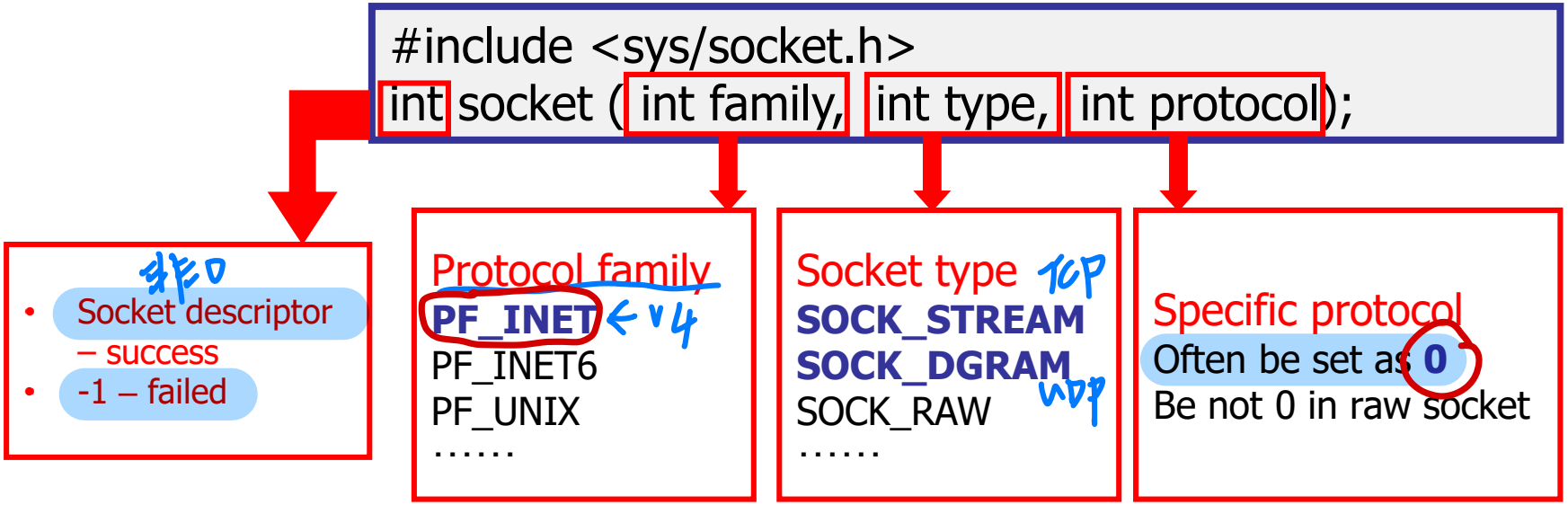
- **gethostbyname()**
  - retrieving host entries from DNS and the query key is a DNS domain name
- **gethostbyaddr()**
  - retrieving host entries from DNS and the query key is an IP address
- gethostname()
  - Obtaining the name of a host
- getservbyname()
  - Mapping a named service onto a port number
- getservbyport()
  - Obtaining an entry from the services database given the port number assigned to it

Using UDP to query Local DNS Server

Returns a pointer to struct hostent (host entry structure) on success

System Calls – socket()

- An application calls *socket()* to create a new socket that can be used for network communication
- The call returns a descriptor for the newly created socket



System Calls – bind()

- An application calls bind() to specify the local endpoint address (a **local IP address and protocol port number**) for a socket
- For TCP/IP, the endpoint address uses the ***sockaddr_in*** structure.
- Must cast Internet-specific socket address (`struct sockaddr_in *`) to generic socket address (`struct sockaddr *`) for bind
- **Servers** use *bind()* to specify the well-known port at which they will await connections

```
#include <sys/socket.h>
int bind (int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
```

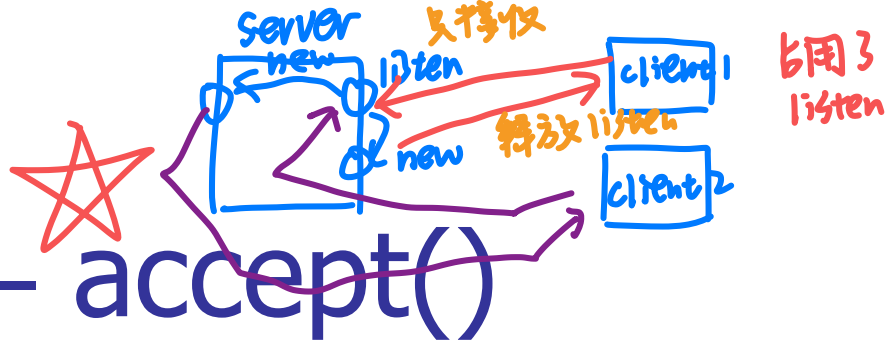
- 0 – success; -1 – failed
- Socket descriptor to be bound
- The local address to which the socket should be bound
- The length of the structure (bytes)

System Calls – listen()

- Connection-oriented **servers** call *listen()* to place a socket in *passive mode* and make it ready to accept incoming connections
- *listen()* also sets the number of incoming connection requests that the protocol software should enqueue for a given socket while the server handles another request
- It only applies to socket used with **TCP**

```
#include <sys/socket.h>
int listen (int sockfd,        int qlength);
```

- 0 – success, -1 – failed
- Socket that should be prepared for use by a server
- The length of the request queue for that socket



System Calls – accept()

- The **server** calls accept() to extract the next incoming request
- *accept()* creates a new socket for each new connection request, and returns the descriptor of the new socket to its caller
- *accept()* fills in the structure (*sockaddr*) with the IP address and protocol port number of the remote machine
- Must cast Internet-specific socket address (`struct sockaddr_in *`) to generic socket address (`struct sockaddr *`) for accept()

```
#include <sys/socket.h>
int accept (int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
```

- Socket Descriptor (non-zero) – success
  -1 – failed
- Socket on which to wait
- The address of the client that placed the request
- The length of the client address

System Calls – connect()

- After creating a socket, a **client** calls *connect()* to establish an active connection to a remote server
- Must cast Internet-specific socket address (`struct sockaddr_in *`) to generic socket address (`struct sockaddr *`) for connect

```
#include <sys/socket.h>
int connect (int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

- 0 – success, -1 – failed
- Socket to connect
- The server's address to which the socket should connect
- The length of the server's address (bytes)

System Calls – send()

- Both clients (to transmit request) and servers (to transmit replies) used *send()* to transfer data across a **TCP connection**
- The application passes the descriptor of a socket to which the data should be sent, the address of the data to be sent, and the length of the data
- Usually, send copies outgoing data into buffers in the OS kernel

```
#include <sys/socket.h>
ssize_t send (int sockfd, const void *buff, size_t nbytes, int flags);
```

- ssize_t: Number of bytes that are sent successfully; -1 – failed
- int sockfd: socket descriptor you want to send data to
- const void *buff: The address of the data to be sent
- size_t nbytes: The number of bytes to be sent
- int flags: The flag controlling the connection, usually 0

System Calls – recv()

- Both clients (to receive a reply) and servers (to receive a request) use *recv* to receive data from a **TCP connection**
- If the buffer cannot hold an incoming user datagram, *recv* fills the buffer and discards the remainder

```
#include <sys/socket.h>
ssize_t recv (int sockfd, void *buff, size_t nbytes, int flags);
```

- Number of bytes that are received successfully
  -1 – failed
- socket descriptor to read from
- The address in memory into which the data to be placed
- The length of the buffer area
- The flag controlling the reception, usually 0

receive完写入.
之前为空.

System Calls – sendto() & recvfrom()

- Allow the caller to send or receive a message over **UDP**
- sendto() requires the caller to specify a destination
- recvfrom() uses an argument to specify where to record the sender's address

```
#include <sys/socket.h>
ssize_t sendto (int sockfd, const void *buff, size_t nbytes, int flags,
                const struct sockaddr *to, socklen_t addrlen);
```

Destination address

The length of the destination address

```
#include <sys/socket.h>
ssize_t recvfrom (int sockfd, void *buff, size_t nbytes, int flags,
                  struct sockaddr *from, socklen_t *addrlen );
```

Number of bytes that are sent or received successfully
-1 – failed

Where to record the sender's address

The length of the sender's address

System Calls – close()

- Once a client or server finishes using a socket, it calls *close* to deallocate it
- Any unread data waiting at the socket will be discarded

```
#include <unistd.h>
int close ( int sockfd);
```

0 – success
-1 – failed

Socket to be closed

System Calls – inet_aton() & inet_addr()

- Converts an IP address in numbers-and-dots notation into unsigned long in network byte order

```
#include <arpa/inet.h>
int inet_aton (const char *string, struct in_addr *address);
```

- 1 – success
  0 – error
- Pointer to the string that contains the numbers-and-dots notation
- Pointer to IP address structure

```
#include <arpa/inet.h>
in_addr_t inet_addr (const char *string,);
```

- When success: return the 32-bit address in network byte order
  When failed: return INADDR_NONE
- Pointer to the string that contains the numbers-and-dots notation

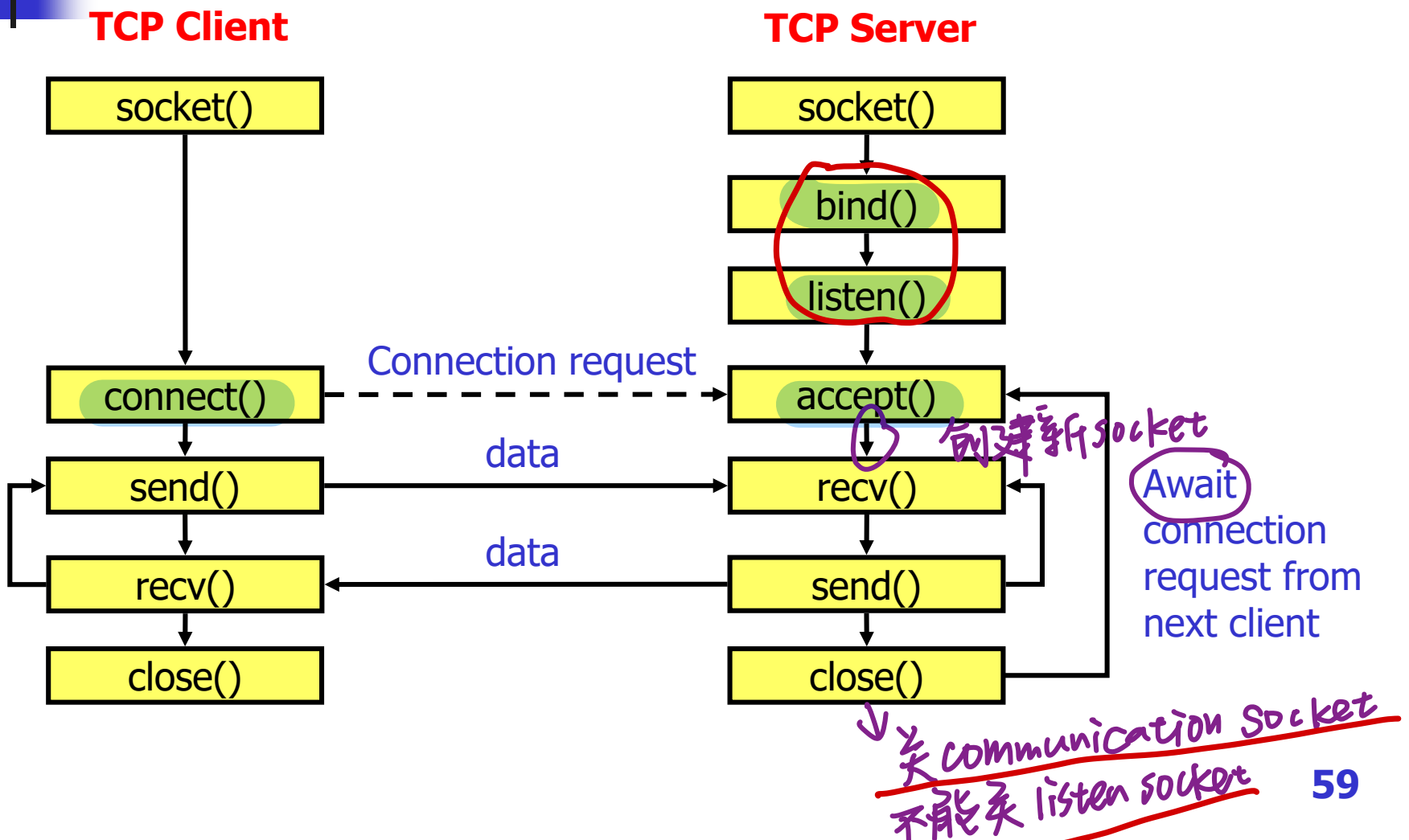
System Calls – inet_ntoa()

- Mapping a 32-bit integer (an IP address in network byte order) to an ASCII string in dotted decimal format

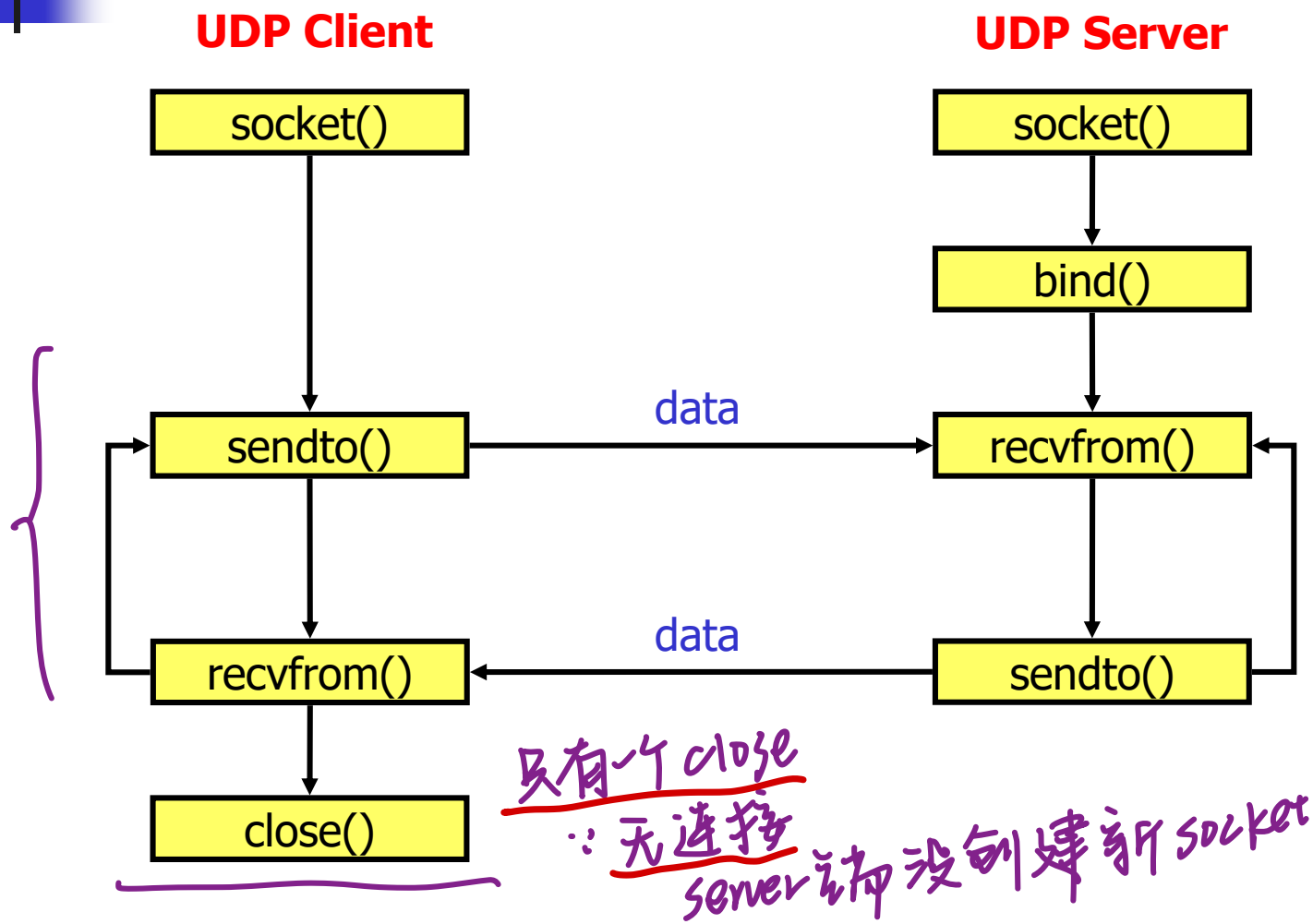
```
#include <arpa/inet.h>
char * inet_ntoa ( struct in_addr inaddr );
```

- `char *`: Pointer to the string contains the numbers-and-dots notation
- `struct in_addr inaddr`: IP address structure in network byte order

Overview of TCP-based sockets API



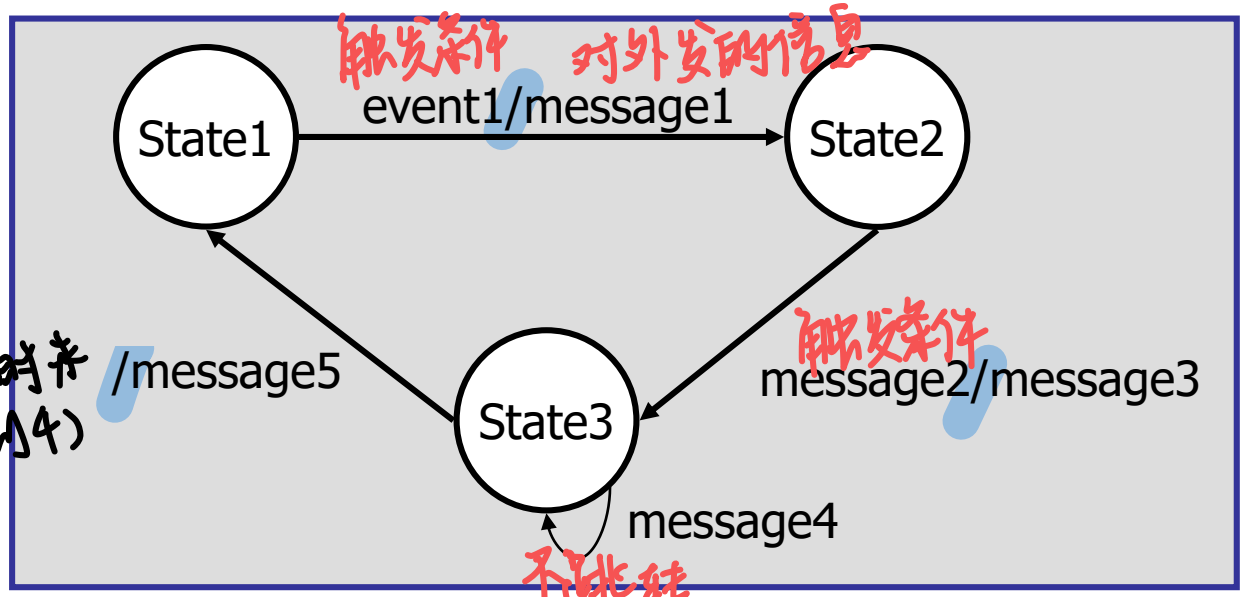
Overview of UDP-based sockets API



DHCP

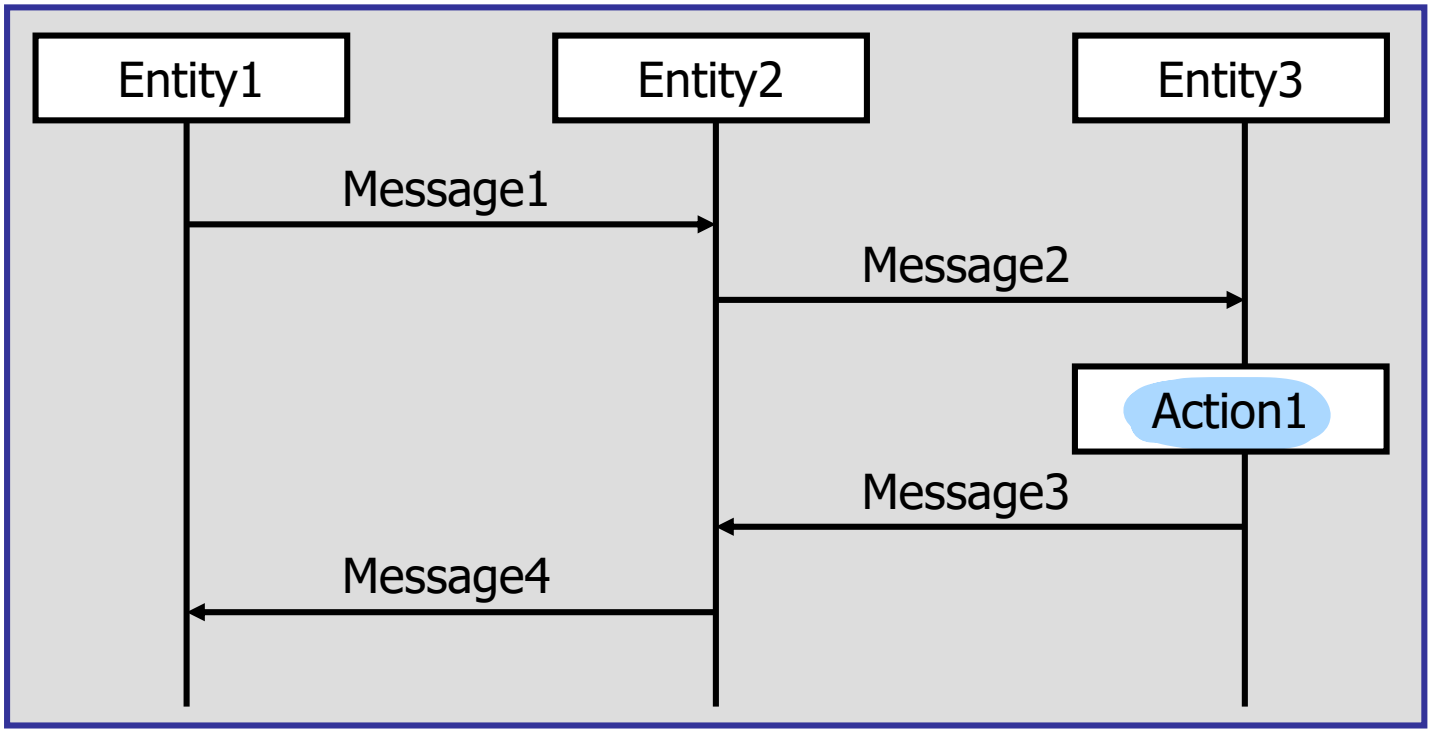
STD (1) 状态迁移图

- State Transition Diagram 单看某实体的状态
- Define the state transition relationship in a protocol
- The state transition may be triggered by an event or an incoming message, and may be together with an outgoing message
- Basic format: state, event, incoming message, outgoing message



MSC (1)

- Message Sequence Chart
- Define the message exchange sequences between different network elements to complete a specific function
- Basic format: network entity, message, action



What is BOOTP?

- Bootstrap Protocol
- used by a network client to obtain an IP address from a configuration server
- Used by diskless workstation to obtain boot information such as boot file name (TFTP is used to transfer boot file)
- originally defined in RFC 951

What is DHCP?

- Dynamic Host Configuration Protocol
- Provides automatic configuration of remote hosts
- An extension to BOOTP
- Using UDP as its transport protocol
- Following client-server paradigm
  - Using port 67 (for server) and 68 (for client)

DHCP Client & DHCP Server

2 entity

- DHCP client
  - A DHCP client is an Internet host using DHCP to obtain configuration parameters such as a network address
- DHCP server
  - A DHCP server is an Internet host that returns configuration parameters to DHCP clients

DHCP Lease 使用有效期

- A DHCP lease 租约 is the amount of time that the DHCP server grants to the DHCP client permission to use a particular IP address
- A typical server allows its administrator to set the lease time
- The optimal time for a lease depends on the particular network and the needs of a particular host 可调整

DHCP Features

- *Simple administration*
  - **IP addresses**, **subnet masks**, and the **default gateway** do not need to be manually entered on every client machine
- *Moving machines*
  - When moving a machine to a different subnet, you simply move the machine and it acquires a new IP and subnet mask from the DHCP server on that subnet.
- *Eliminating erroneous IP information*
  - Taking the human factor out of the equation reduces problems, such as duplicate IP addresses
- *Eliminating additional configuration information*
  - **DNS servers**, default gateways and WINS servers are some of the widely used configuration option settings possible with DHCP.
- *Efficient use of IP addresses*
  - DHCP can help make the best use of limited IP addresses. All available IP address are put into a **pool** and used by active clients. Machines that are inactive do not use an IP address.

DHCP vs. BOOTP - Similarities

- The format structure each uses to exchange messages
- Use of well-known UDP ports for client/server communication
- IP address distribution as an integral part of configuration service

DHCP vs. BOOTP - Differences

- BOOTP
  - Prior to DHCP
  - Configure diskless workstations
  - Do not rebind or renew configuration

无法续约
无法回收

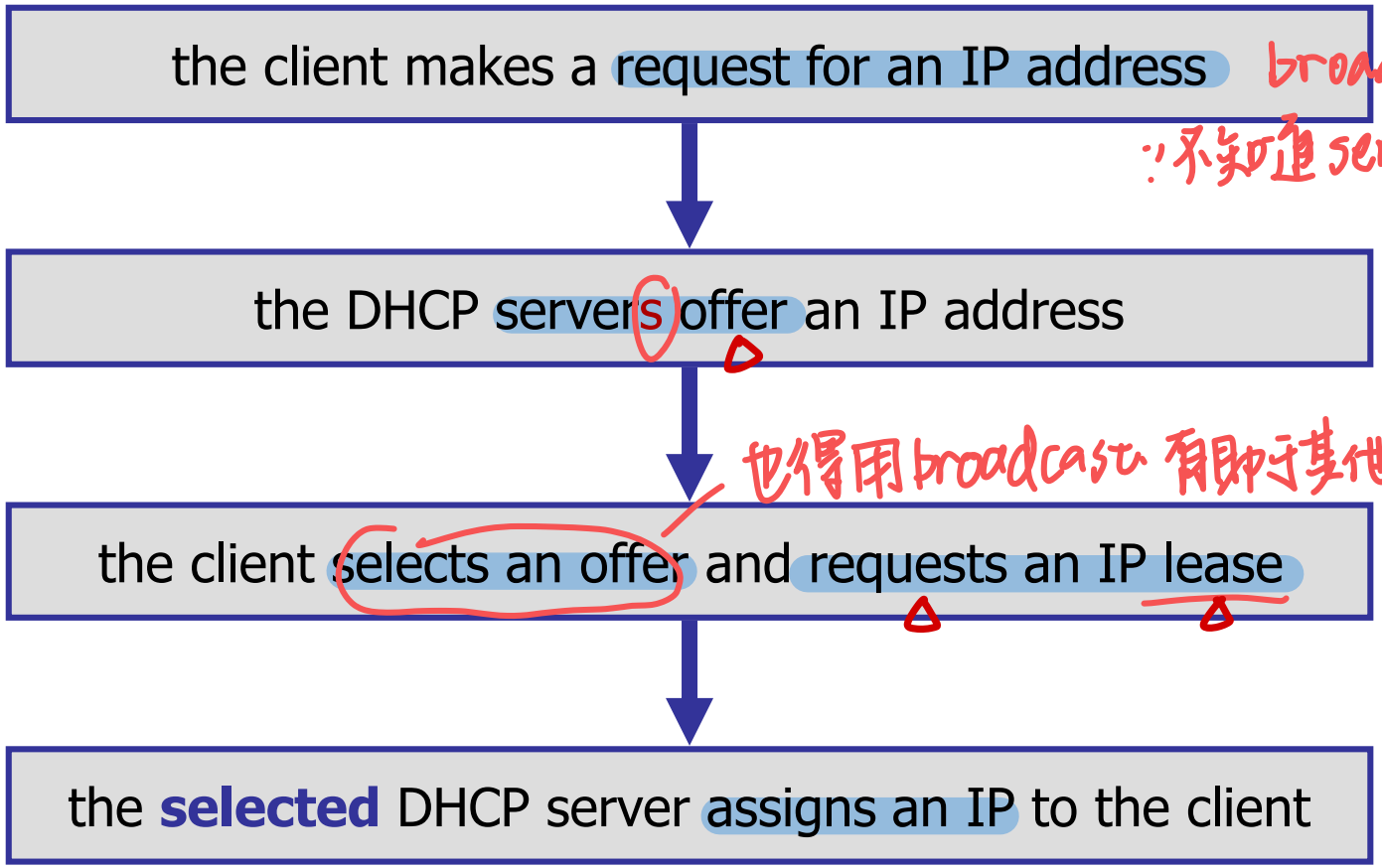
- DHCP
  - After BOOTP
  - Configure frequently relocated networked computers with local hard drives and full boot capabilities

支持联网的各种设备

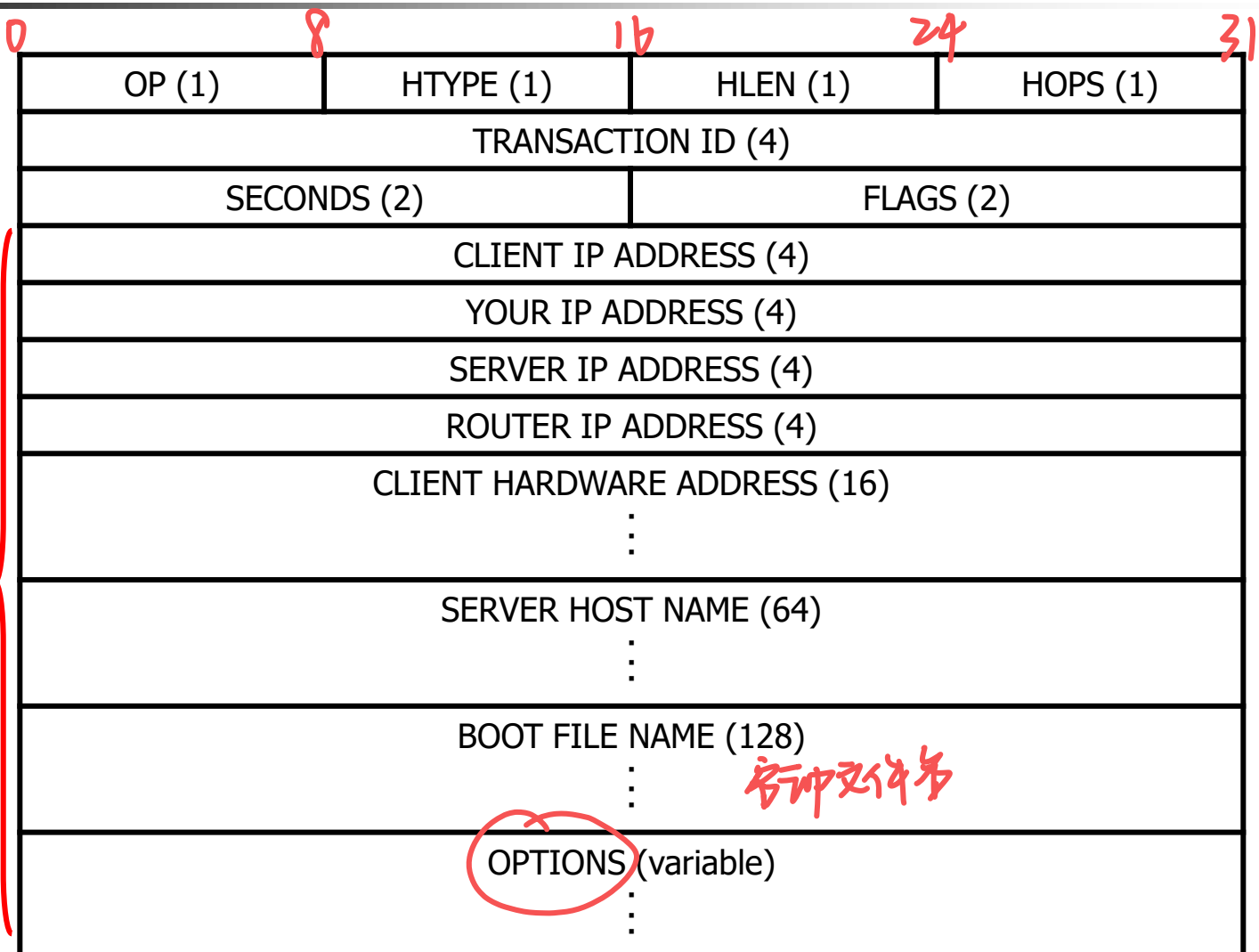
  - Clients automatically enter a rebinding state at set timed intervals to renew their leased address allocation

自动续约

Phases of IP Assignment with DHCP



DHCP Message Format



Clients fill in as much as they know and leave remaining fields set to zero

DHCP Message Format

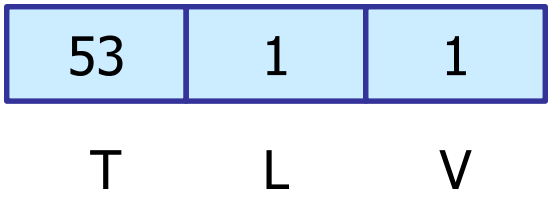
- **OP**: 1 – request, 2 – reply
- **HTYPE**: Hardware Address Type, Defined in the ARP section in the Assigned Numbers RFC, E.g. 1-Ethernet (10Mb)
- **HLEN**: Hardware Address Length, E.g. 6-Ethernet (10Mb)
- **HOPS**: Client sets to zero, optionally used by relay agents when booting via a relay agent
- **TRANSACTION ID**: an integer, Used by the client to match responses with requests
- **SECONDS**: seconds elapsed since client began address acquisition or renewal process
- **FLAGS**: Allows control of the request and response. A client uses the leftmost bit to control weather the server sends the response via unicast or broadcast. The remaining bits are reserved for future use
- **CLIENT IP ADDRESS**: Only filled in if client is in BOUND, RENEW or REBIND state
- **YOUR IP ADDRESS**: Client IP address
- **OPTIONS**: TLV encoding style
- SERVER IP ADDRESS:

- ROUTER IP ADDRESS: 负责实现request转发的路由
- Server host name: 可选. sever name. 以0x00结尾
- Boot file name: 启动文件名
- client hardware addr: client的硬件地址

Message type option

携带包类型

- TLV encoding
- Type=53 (type of the DHCP message)
- Length=1 byte
- Value
  - 1: DHCPDISCOVER
  - 2: DHCPOFFER
  - 3: DHCPREQUEST
  - 4: DHCPDECLINE
  - 5: DHCPACK
  - 6: DHCPNAK
  - 7: DHCPRELEASE
  - 8: DHCPINFORM



DHCP Messages (1)

- **DHCPDISCOVER** - Client broadcast to locate available servers.
- **DHCPOFFER** - Server to client in response to DHCPDISCOVER with offer of configuration parameters.
- **DHCPREQUEST** - Client message to servers either
  - (a) requesting offered parameters from one server and implicitly declining offers from all others,
  - (b) confirming correctness of previously allocated address after, e.g., system reboot
  - (c) extending the lease on a particular network address.
- **DHCPACK** - Server to client with configuration parameters, including committed network address.

DHCP Messages (2)

- **DHCPNAK** - Server to client indicating client's notion of network address is incorrect (e.g., client has moved to new subnet) or client's lease as expired.
- **DHCPDECLINE** - Client to server indicating network address is already in use.
- **DHCPRELEASE** - Client to server relinquishing network address and cancelling remaining lease.
- **DHCPINFORM** - Client to server, asking only for local configuration parameters; client already has externally configured network address. For example, it can be used to obtain tunnel endpoint address.

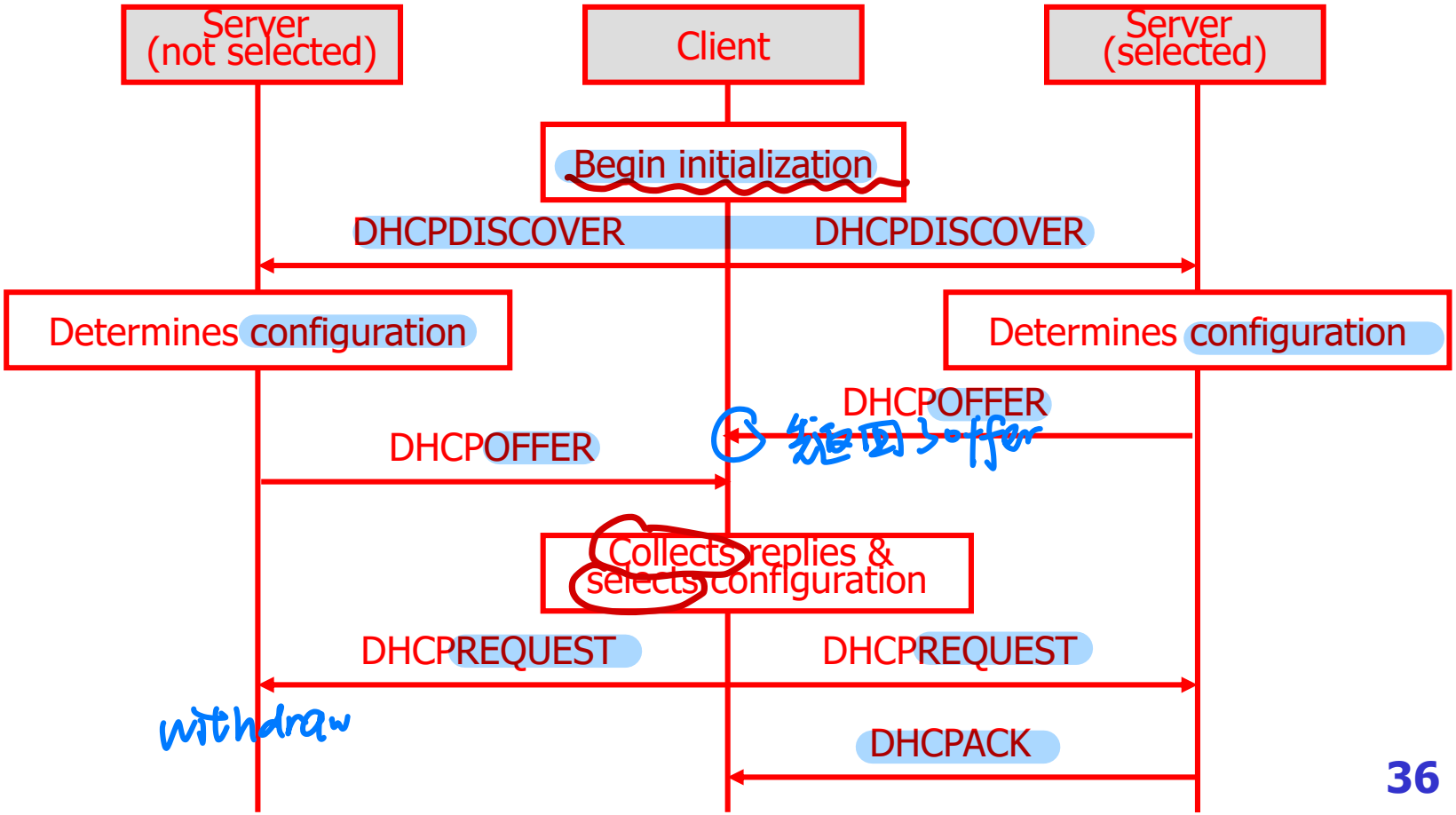
Address Acquisition: phases

4个phase需全部完成

- ***IP lease request***
  - To boot up, the client sends a DHCPDISCOVER broadcast message, requesting the location of a DHCP server with IP address information
  - The DHCPDISCOVER packet is encapsulated in a UDP/IP packet and is sent to the local subnet broadcast address of 255.255.255.255
- ***IP lease offer***
  - After requesting a lease, the DHCP client waits for a response and is said to be in a SELECT state
  - Any available DHCP servers with IP addresses to offer respond to the client request with a DHCPOFFER message
- ***IP lease selection***
  - The client chooses one DHCPOFFER from all the offers it receives, regardless of which subnet the DHCP server is located in
  - The client then sends a broadcast DHCPREQUEST message, requesting a lease
- ***IP lease acknowledgment***
  - The DHCP server that made the offer responds to the client with a DHCPACK message while any other DHCP servers that made an offer withdraw
  - The IP address is assigned to the client

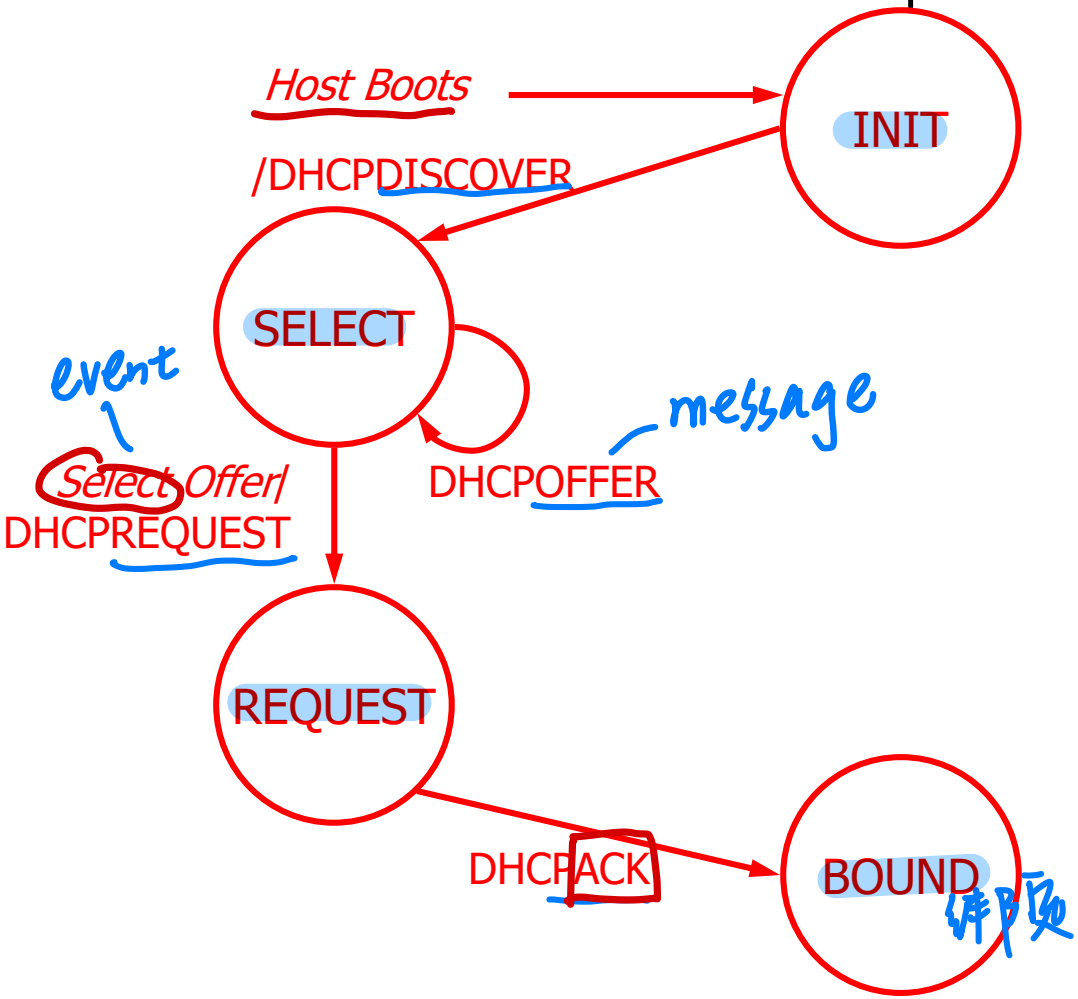
Address Acquisition (2)

- MSC for Address Acquisition



Address Acquisition (3)

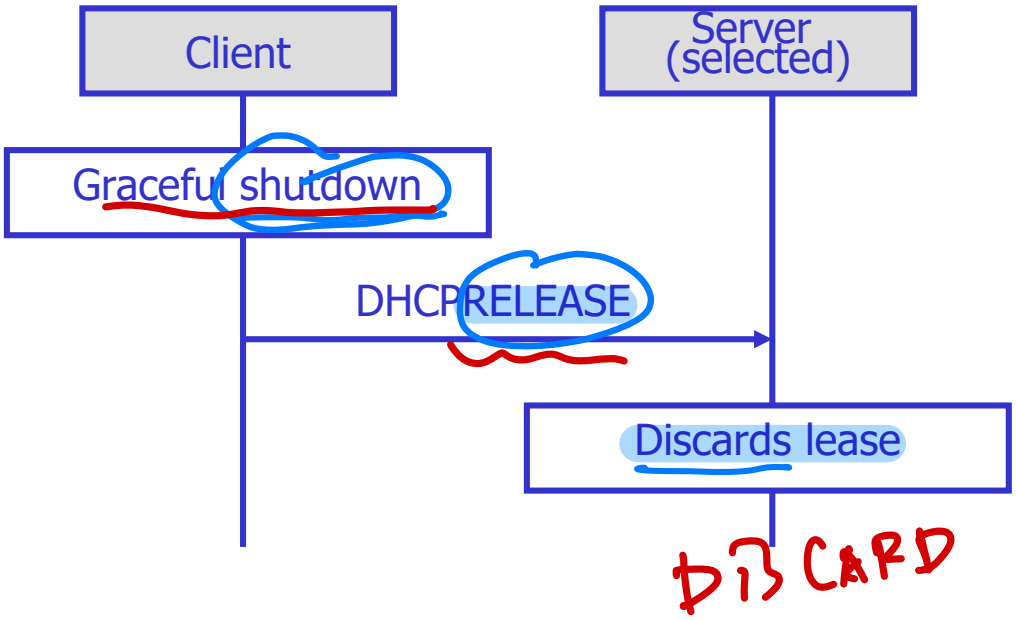
- STD of client for Address Acquisition



INIT
SELECT
REQUEST
BOUND

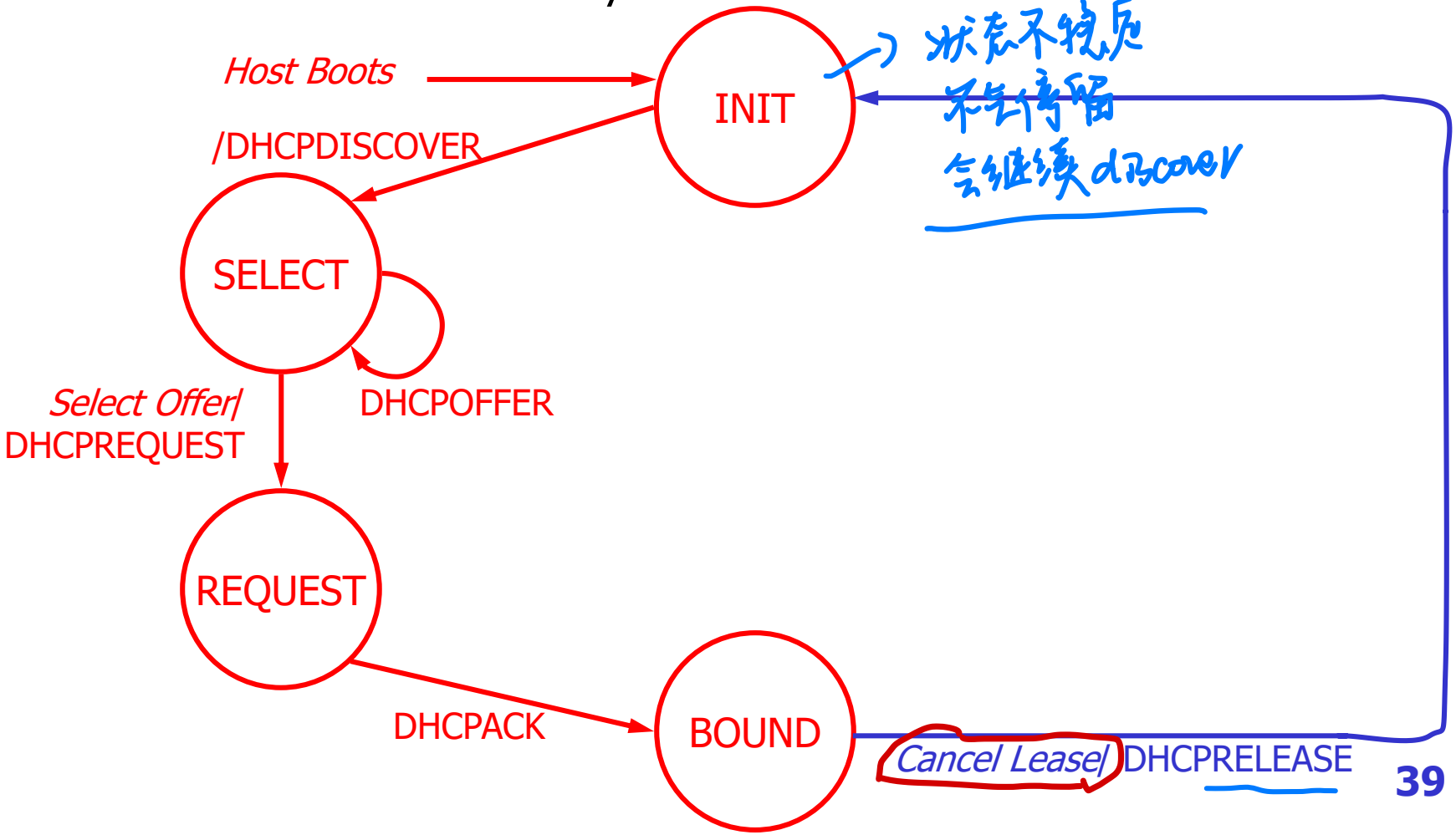
Early Lease Termination (1)

- MSC for Early Lease Termination 提前解約



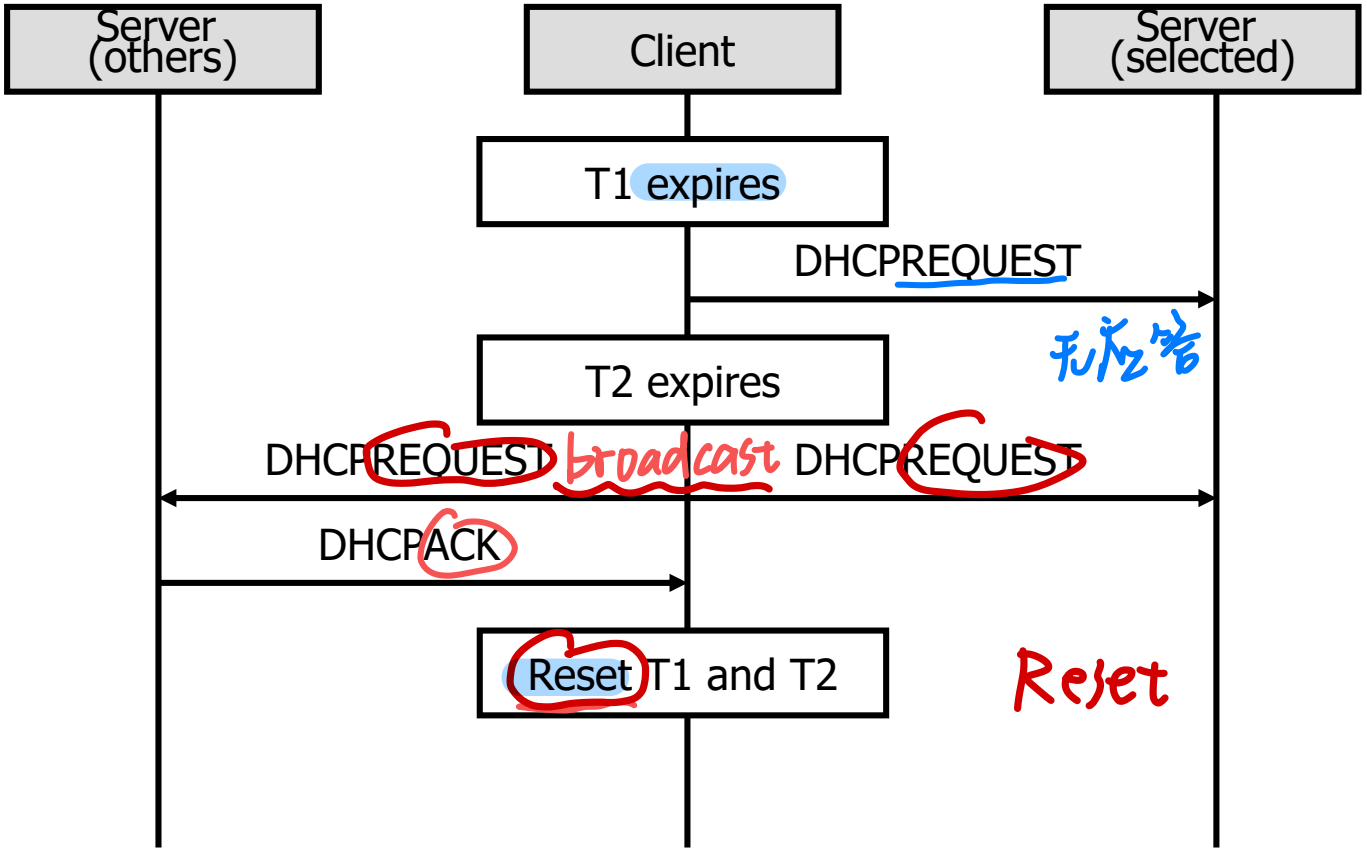
Early Lease Termination (2)

- STD of client for Early Lease Termination



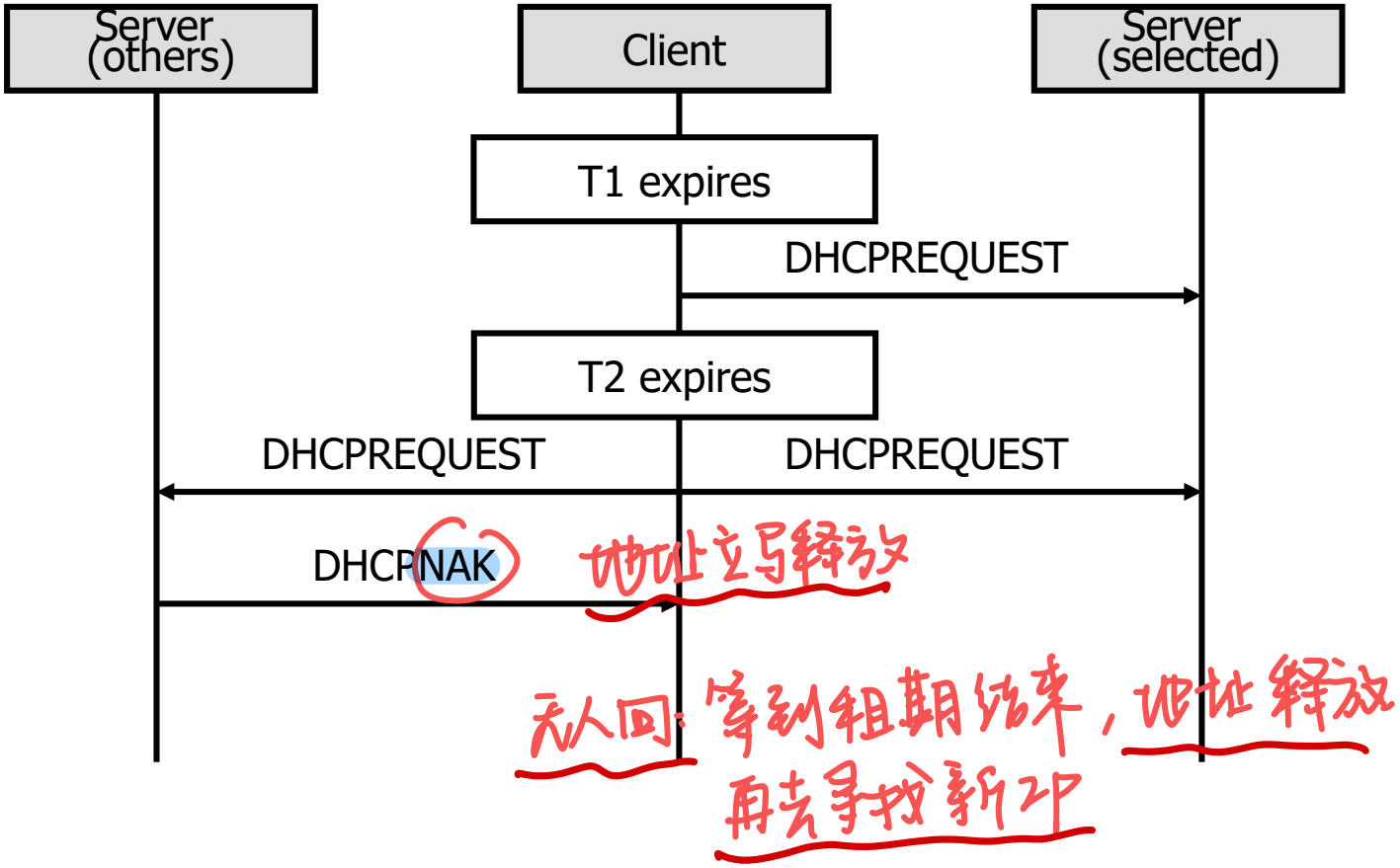
Lease Renewal (2)

- MSCs for lease renewal



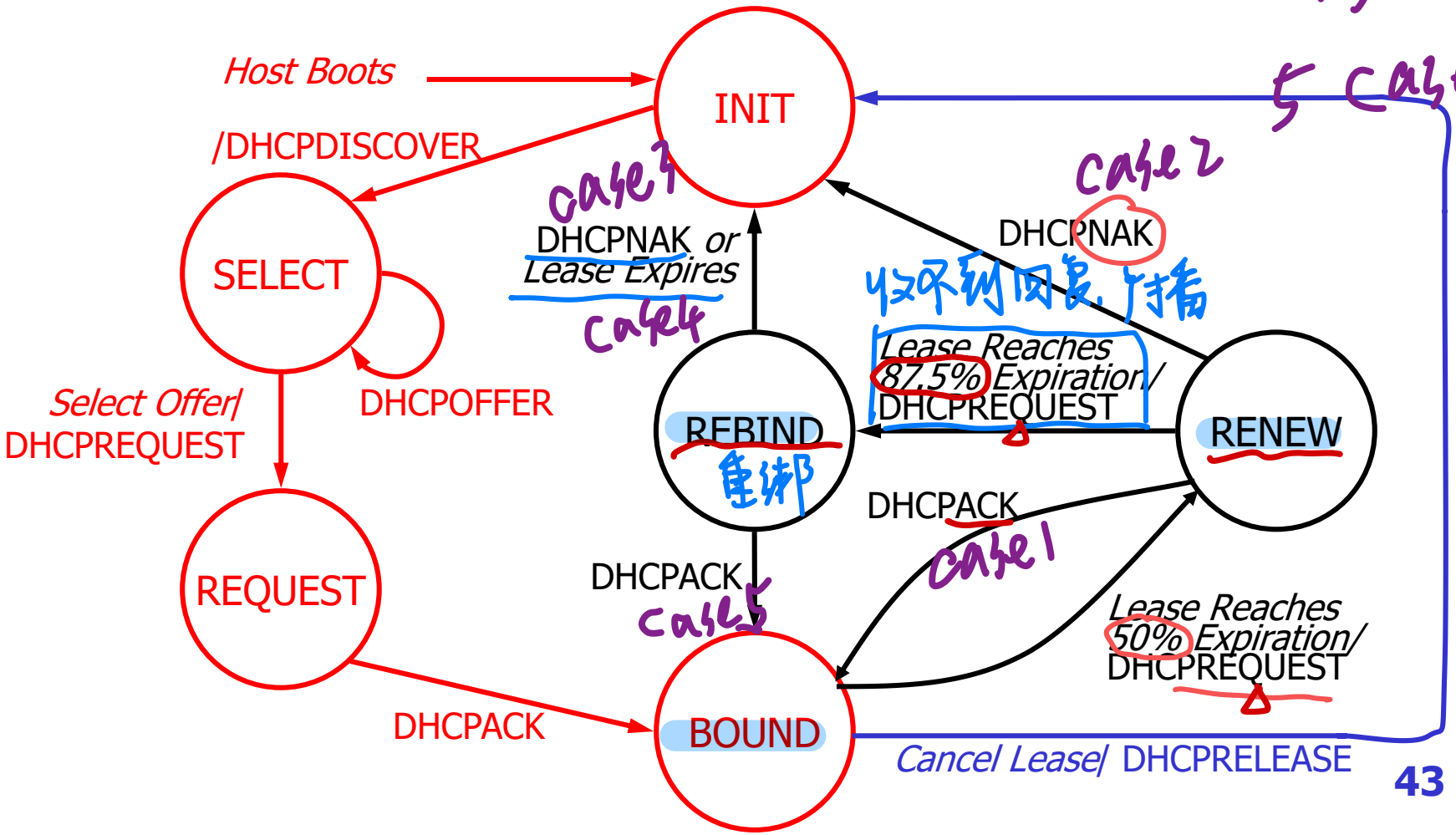
Lease Renewal (3)

- MSCs for lease renewal



Lease Renewal (4)

- STD of client for lease renewal



DHCP Relay

所处子网无DHCP，用relay发送到别的子网

- Used to interconnect hardware and software on different physical network segments called subnets and forward IP packets between each of the subnets.
- Router must comply with DHCP/BOOTP relay agent capabilities.
- A computer that can function as a relay agent if router cannot function as a relay agent.

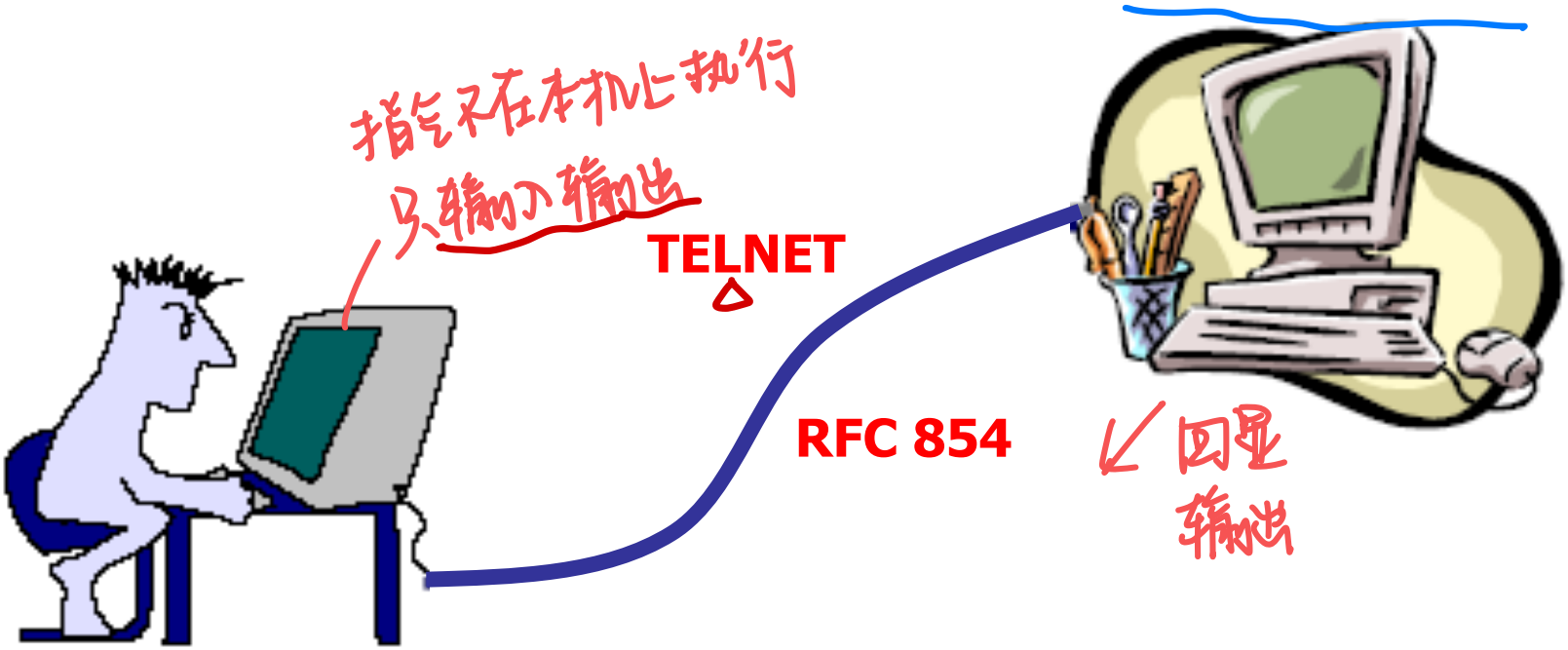
是router or computer

TELNET

What Is TELNET? (1)

建立连接

- A protocol used to establish a dumb terminal session to another computer on the Internet
  哑终端.
- An important Internet application for remote access



What Is TELNET? (2)

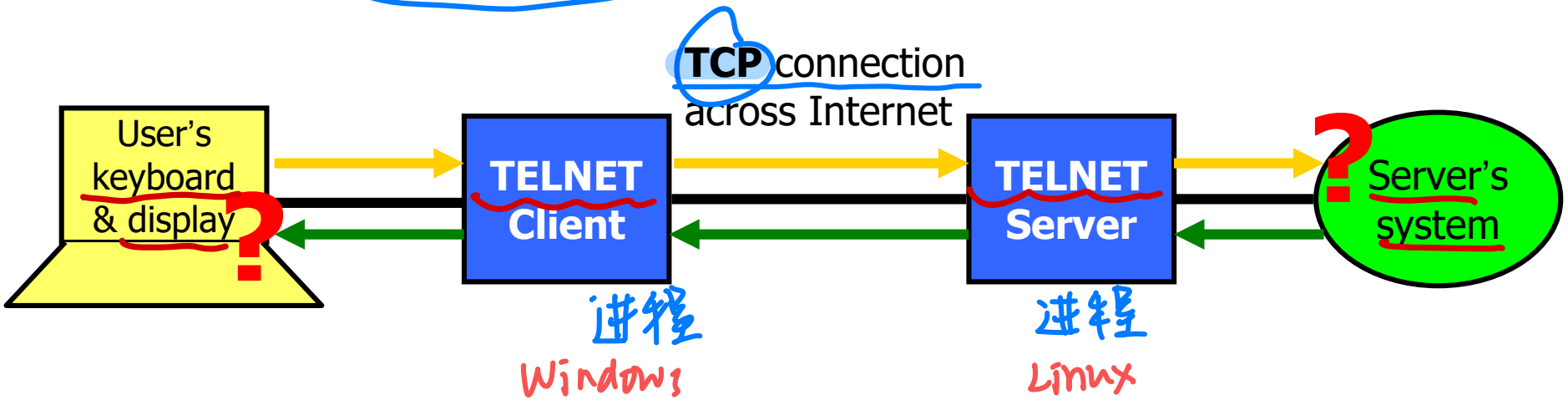
- Definition in RFC854
  - The purpose of the TELNET Protocol is to provide a general, bi-directional, byte oriented communications facility.
  - Its primary goal is to allow a standard method of interfacing terminal devices and terminal-oriented processes to each other.
  - It is envisioned that the protocol may also be used for terminal-terminal communication ("linking") and process-process communication (distributed computation).

TELNET vs. telnet

- TELNET is a *protocol* that provides "a general, bi-directional, eight-bit byte oriented communications facility"
- telnet is a *program* that supports the TELNET protocol over TCP
- Many application protocols are built upon the TELNET protocol

Problem

- Lack of common language between the terminal and the remote host



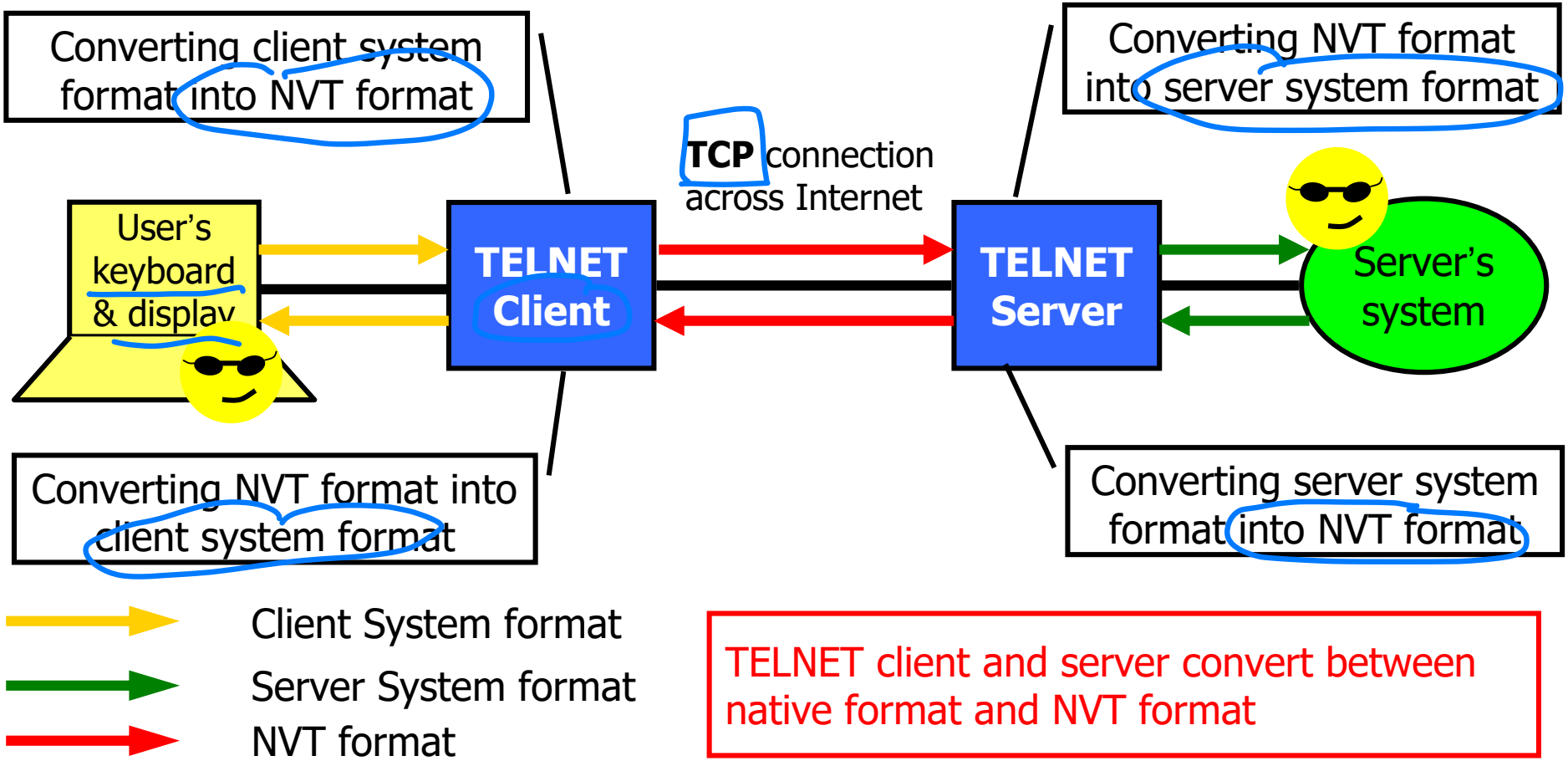
provide a standard interface to remote systems

Network Virtual Terminal

- Transform local characteristics into standardized form
  - Network virtual terminal (NVT)
- Imaginary device
  - Well defined set of characteristics
- Both sides generate data and control signals in native language but translates them to NVT form
  - The sending side translates native data and control signals into NVT form before sending out
  - The receiving side gets the NVT data and signals and translates into its native form

NVT Operation

- Accommodating heterogeneity 异质



TELNET client and server convert between native format and NVT format

TELNET Operations

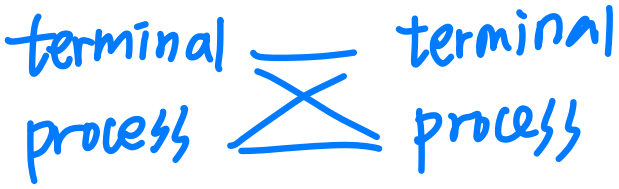
- Connection management
  - Connection request, establish and terminate
  - Telnet uses TCP (port 23) by default
- Option Negotiation
  - To determine mutually agreeable set of characteristics and options
- Exchange of control information (e.g. end of line), commands and transfer of data between two correspondents
- A typical telnet session is to exchange of data between terminal and host
  - Multiple rounds
  - Not only for accessing remote accounts; was also used for interactive system
    - Try “telnet bbs.byr.cn”

Some Features

data sent as stream of 8-bit bytes

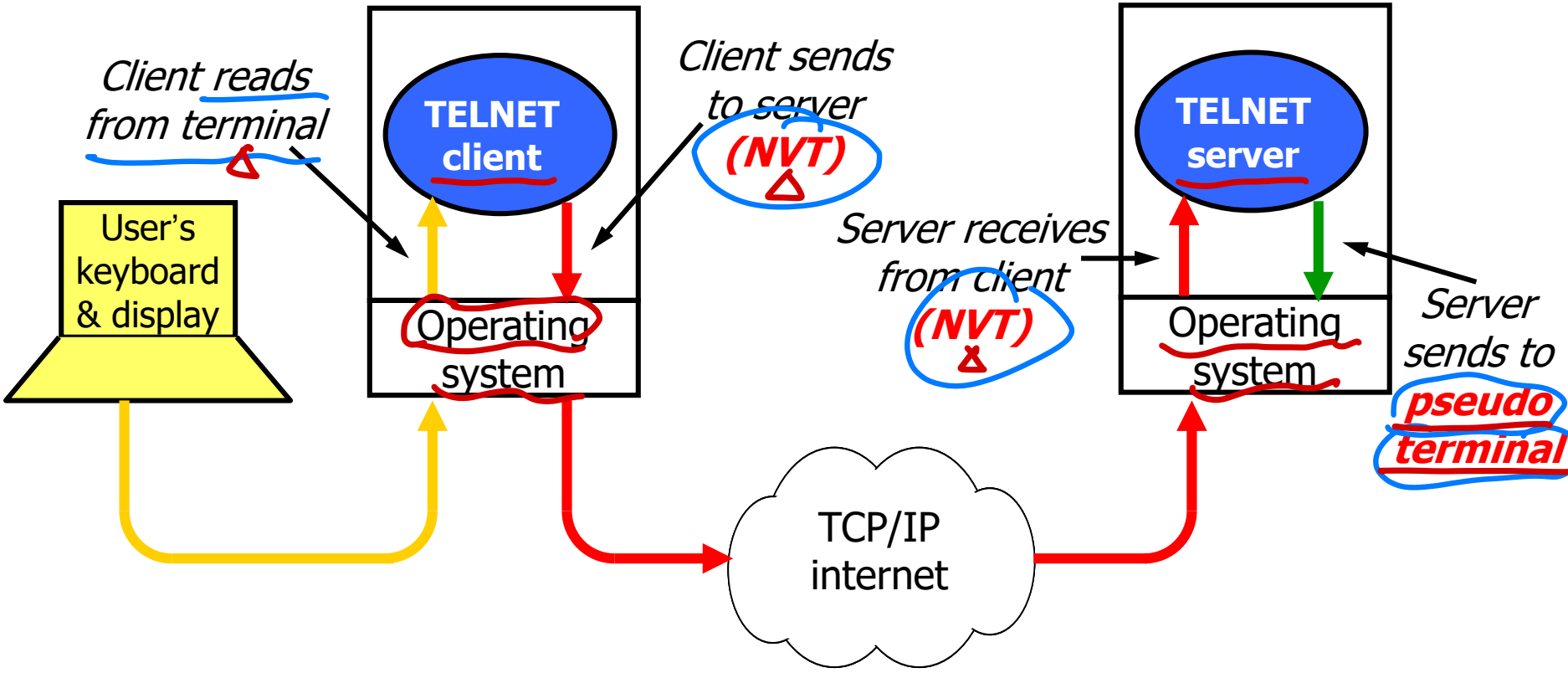
- TCP connection: directed toward port 23 of the server being asked to perform a service
- Data and control multiplexed over the same connection
- NVT - representation of a generic terminal
- Negotiated options - Enabling Telnet to evolve to meet new demands without endless new versions of basic protocol
- A symmetric view of terminals and processes

与ftp不同，同一连接多路传数据+控制



Transmission Of Data (1)

- Data path from the user's keyboard to the remote system



Transmission Of Data (2)

- Underlying TCP full duplex
  - The underlying network is intrinsically full duplex
- Data sent half duplex 半双工
  - The communication between terminal and process is one direction at a time.
- Data sent as stream of 8-bit bytes
  - No other formatting
- Control signals and other non-data information sent as Telnet commands
  - Byte strings embedded in data stream
  - User control signals, commands between Telnet processes as part of protocol and option negotiation and subnegotiation

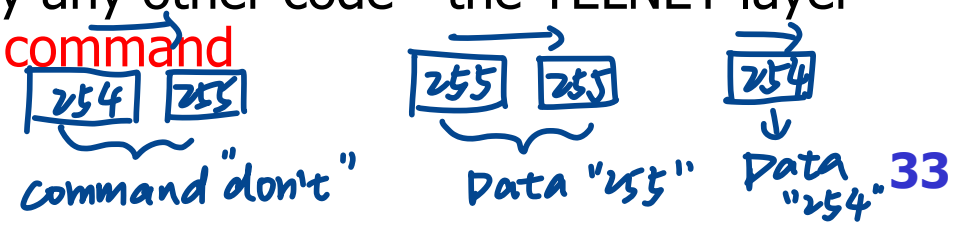
任何一个字符都1 byte

Control Functions (3) – delivery

Command	Decimal Codes	Description
IAC	255 "11111111"	Interpret next octet as command
DONT	254	Denial of request to perform specific option
DO	253	Approval to allow specific option
WONT	252	Refusal to perform specific option
WILL	251	Agreement to perform specific option
SB	250	Start of option subnegotiation
GA	249	Go ahead
EL	248	Erase line
EC	247	Erase character
AYT	246	Are you there
AO	245	Abort output
IP	244	Interrupt process
BRK	243	Break
DMARK	242	Data mark
NOP	241	No operation
SE	240	End of subnegotiation
EOR	239	End of record

Control Functions (4) – IAC

- TELNET command structure
  - at least a two byte sequence: the IAC (Interpret as Command) escape character followed by the code for the command
- The IAC code is 255
  - If a 255 is sent as data - it must be followed by another 255
- Looking for a command
  - Each receiver must look at each byte that arrives and look for an IAC
  - If IAC is found and the next byte is "IAC" - a single data byte (value 255) is presented to the application/ terminal
  - If IAC is followed by any other code - the TELNET layer interprets this as a command



Control Functions (5) – DO, DONT, WILL, WONT

- Used for options negotiation
- Examples

Sender	Receiver	Meaning
WILL →	← DO	Sender wants to active a option, and receiver agrees
WILL →	← DON'T	Sender wants to active a option, and receiver refuses
DO →	← WILL	Sender wants receiver to active a option, and receiver agrees
DO →	← WONT	Sender wants receiver to active a option, and receiver refuses

∴基本部分不变，不断增加options
∴优点：使Telnet能够不断发展以满足新的需求，而无需不断更新基本协议版本。

Motivations

- All NVTs support a minimal set of capabilities
- Some terminals have more capabilities than the minimal set
- The two endpoints negotiate a set of mutually acceptable options (character set, echo mode, etc.)
- The set of options is not part of the TELNET protocol, so that new terminal features can be incorporated without changing the TELNET protocol

新option不会影响TELNET协议

Option Examples

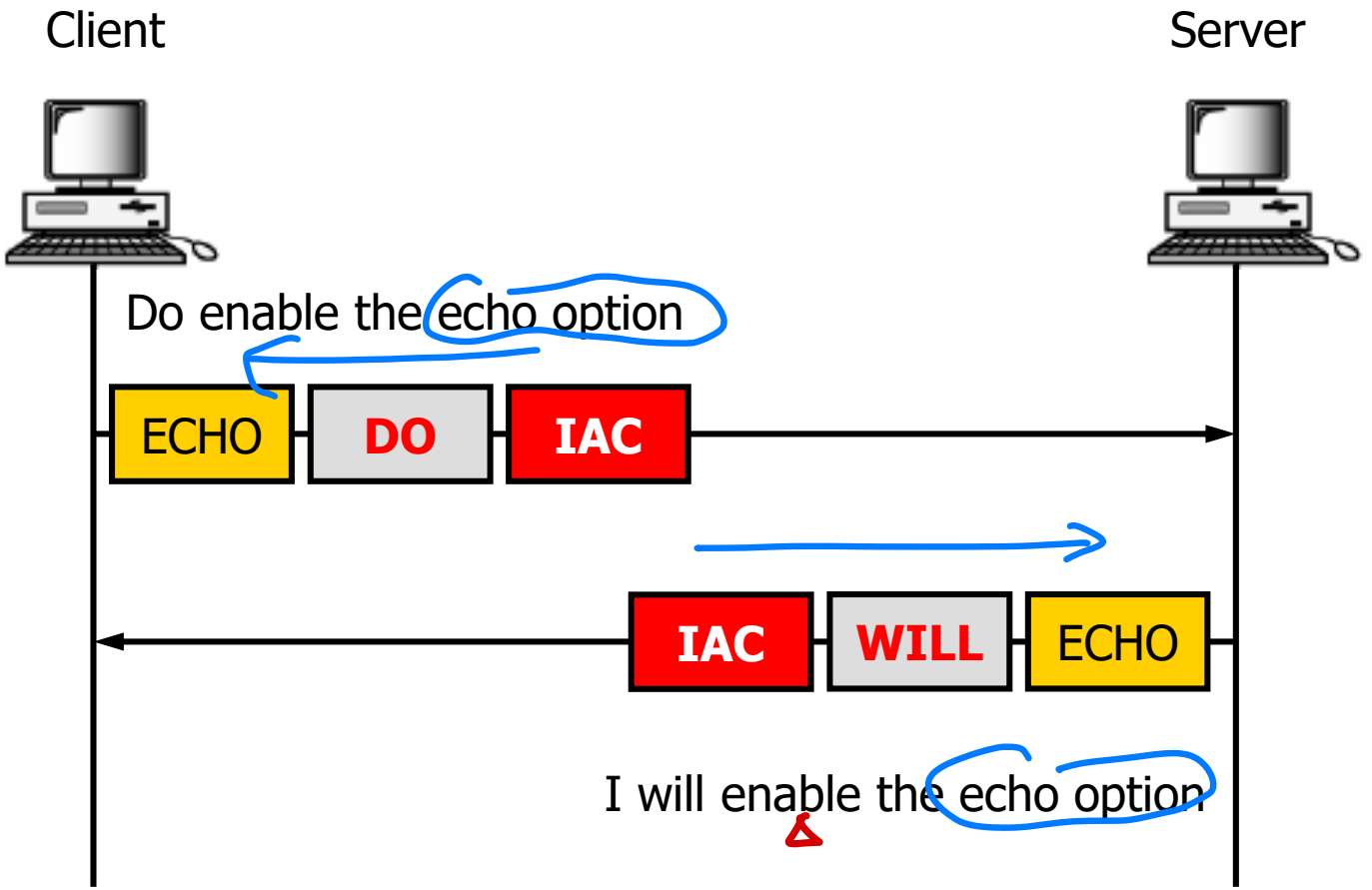
- echo modes
  - Keyboard input be echoed on the terminal side or not
- Line mode vs. character mode
  - One line or one character per transmission
- character set (EBCDIC vs. ASCII)
  - EBCDIC - Extended Binary-Coded Decimal Interchange Code
  - ASCII - American Standard Code for Information Interchange

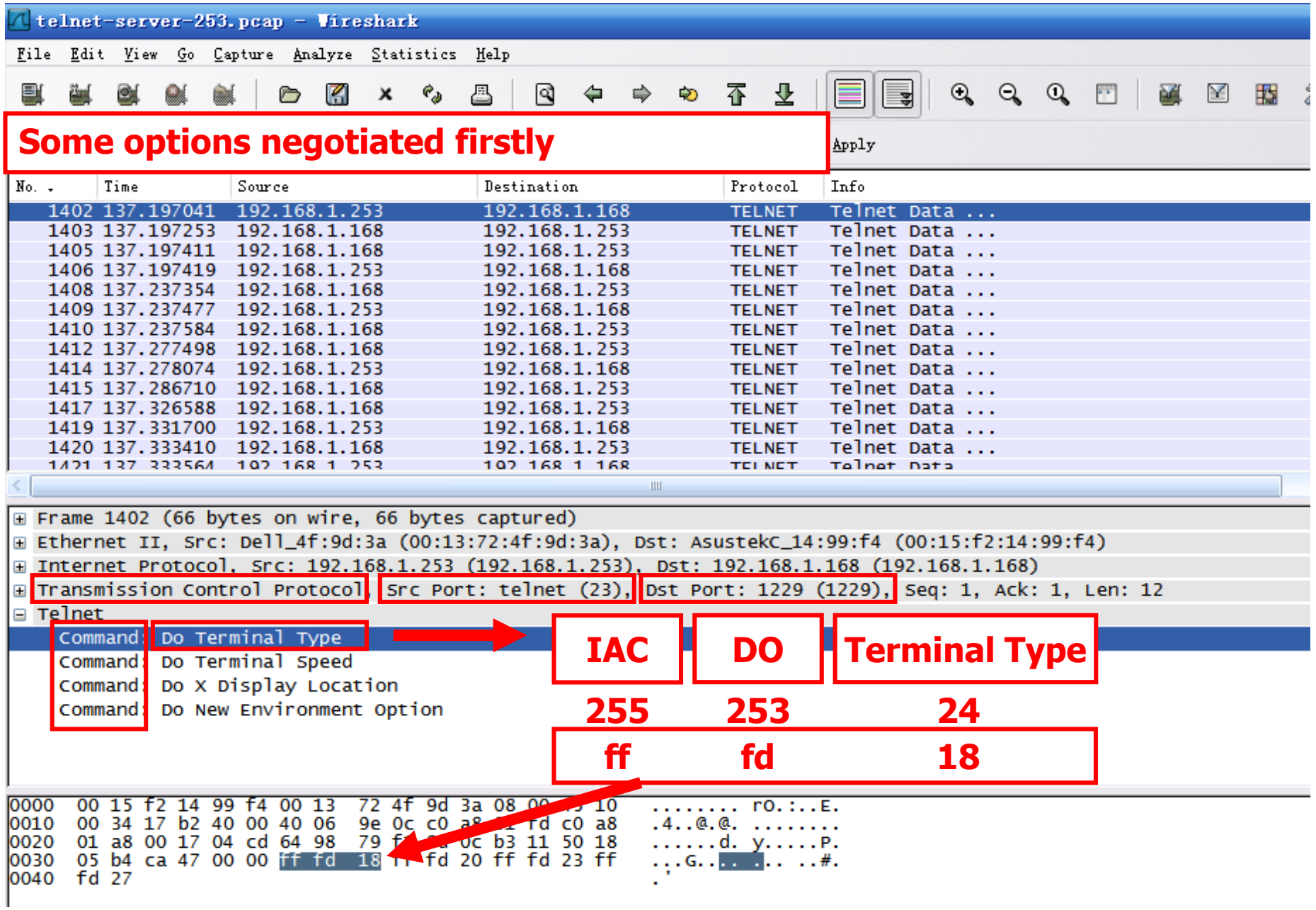
Options Negotiation

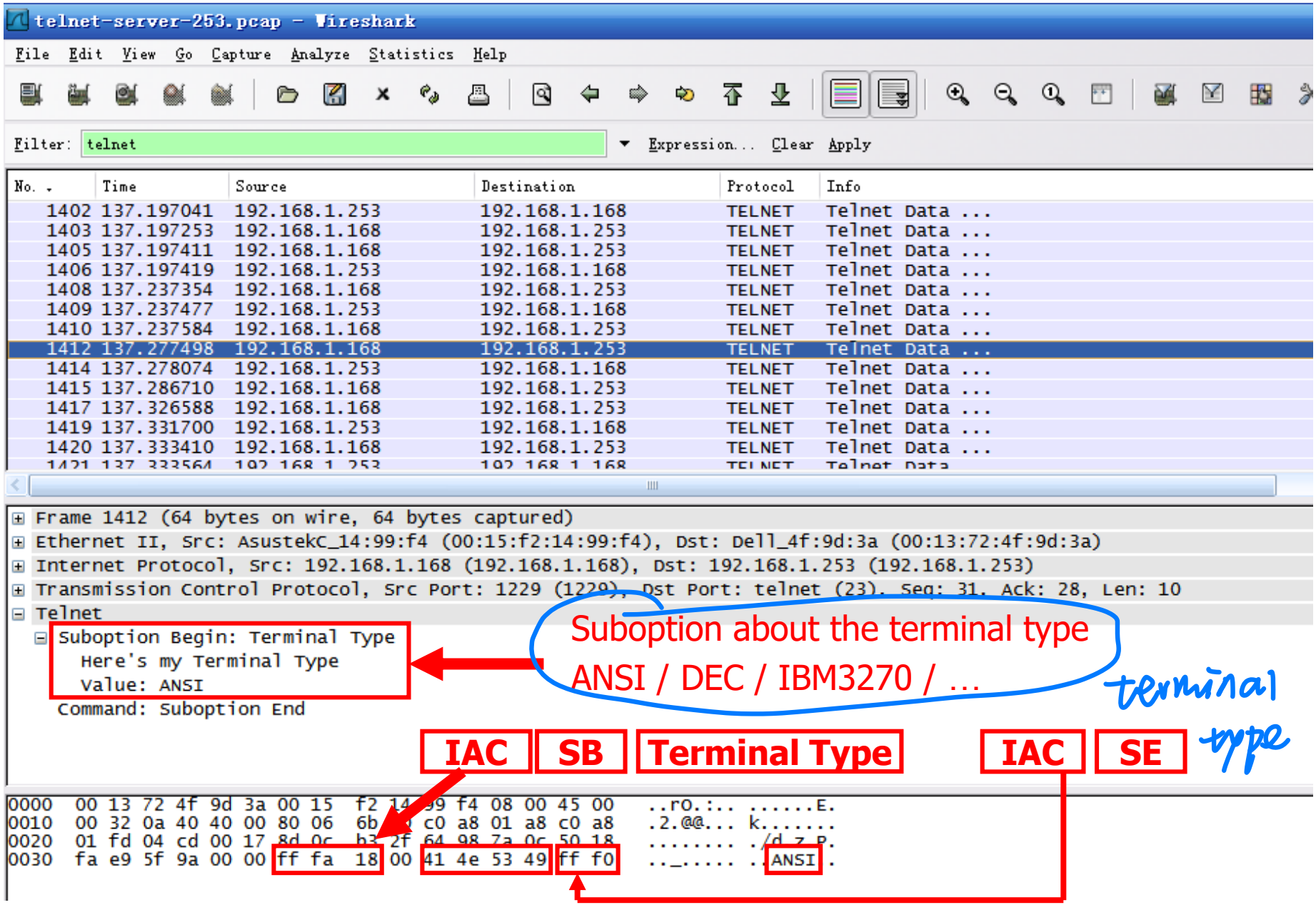
- Each option is assigned a byte value
- The DO, DONT, WILL, and WONT commands are used to negotiate options
- Options negotiation is symmetric
- Steps must be taken to avoid option processing loops
- Subnegotiations are used when more information is needed, such as when negotiating terminal type, window size, etc

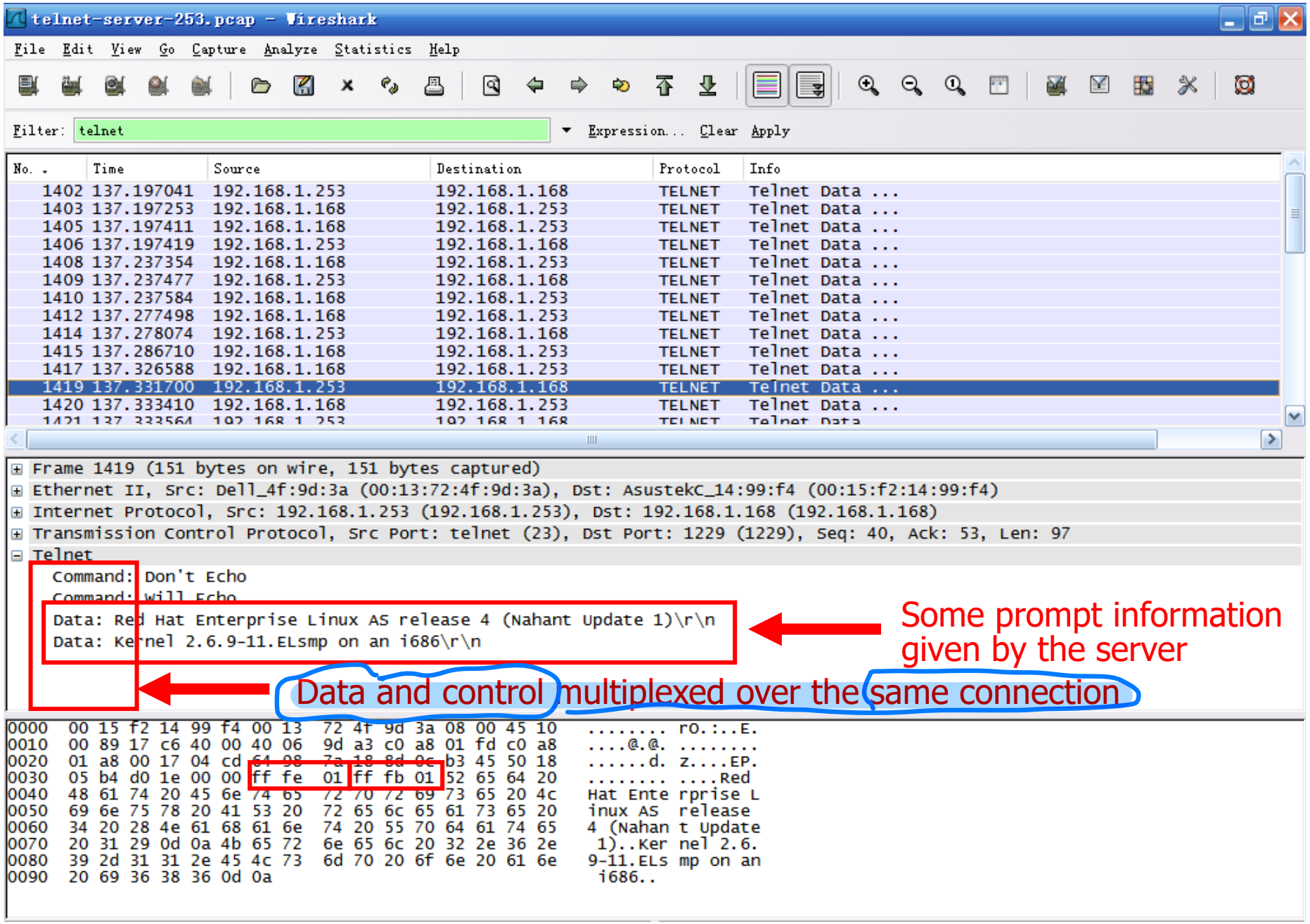
subnegotiation

Example: Negotiation of Echo Option




telnet-server-253.pcap - Wireshark
File Edit View Go Capture Analyze Statistics Help
Some options negotiated firstly
Apply
No. Time Source Destination Protocol Info
1402 137.197041 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1403 137.197253 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1405 137.197411 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1406 137.197419 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1408 137.237354 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1409 137.237477 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1410 137.237584 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1412 137.277498 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1414 137.278074 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1415 137.286710 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1417 137.326588 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1419 137.331700 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1420 137.333410 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
Frame 1402 (66 bytes on wire, 66 bytes captured)
Ethernet II, Src: Dell_4f:9d:3a (00:13:72:4f:9d:3a), Dst: AsustekC_14:99:f4 (00:15:f2:14:99:f4)
Internet Protocol, Src: 192.168.1.253 (192.168.1.253), Dst: 192.168.1.168 (192.168.1.168)
Transmission Control Protocol, Src Port: telnet (23), Dst Port: 1229 (1229), Seq: 1, Ack: 1, Len: 12
Telnet
Command: Do Terminal Type
Command: Do Terminal Speed
Command: Do X Display Location
Command: Do New Environment Option
IAC
DO
Terminal Type
255
253
24
ff
fd
18
0000 00 15 f2 14 99 f4 00 13 72 4f 9d 3a 08 00 45 10 .... rO.:..E.
0010 00 34 17 b2 40 00 40 06 9e 0c c0 a8 01 fd c0 a8 .4..@.@. ....
0020 01 a8 00 17 04 cd 64 98 79 f1 [illegible] 0c b3 11 50 18 ......d. y.....P.
0030 05 b4 ca 47 00 00 ff fd 18 ff fd 20 ff fd 23 ff ...G..... ... ..#.
0040 fd 27 .'

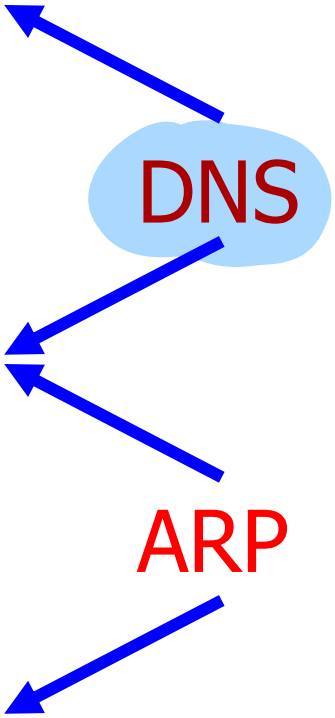

telnet-server-253.pcap - Wireshark
File Edit View Go Capture Analyze Statistics Help
Filter: telnet
Expression... Clear Apply
No. Time Source Destination Protocol Info
1402 137.197041 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1403 137.197253 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1405 137.197411 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1406 137.197419 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1408 137.237354 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1409 137.237477 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1410 137.237584 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1412 137.277498 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1414 137.278074 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1415 137.286710 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1417 137.326588 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1419 137.331700 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1420 137.333410 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
Frame 1412 (64 bytes on wire, 64 bytes captured)
Ethernet II, Src: AsustekC_14:99:f4 (00:15:f2:14:99:f4), Dst: Dell_4f:9d:3a (00:13:72:4f:9d:3a)
Internet Protocol, Src: 192.168.1.168 (192.168.1.168), Dst: 192.168.1.253 (192.168.1.253)
Transmission Control Protocol, Src Port: 1229 (1229), Dst Port: telnet (23), Seq: 31, Ack: 28, Len: 10
Telnet
Suboption Begin: Terminal Type
Here's my Terminal Type
Value: ANSI
Command: Suboption End
Suboption about the terminal type
ANSI / DEC / IBM3270 / ...
terminal type
IAC
SB
Terminal Type
IAC
SE
0000 00 13 72 4f 9d 3a 00 15 f2 14 99 f4 08 00 45 00 ..ro.:.. ......E.
0010 00 32 0a 40 40 00 80 06 6b c0 a8 01 a8 c0 a8 .2.@@... k.......
0020 01 fd 04 cd 00 17 8d 0c b3 2f 64 98 7a 0c 50 18 ..../d.z.P.
0030 fa e9 5f 9a 00 00 ff fa 18 00 41 4e 53 49 ff f0 .._..... ..ANSI..


telnet-server-253.pcap - Wireshark
File Edit View Go Capture Analyze Statistics Help
Filter: telnet
Expression... Clear Apply
No. Time Source Destination Protocol Info
1402 137.197041 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1403 137.197253 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1405 137.197411 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1406 137.197419 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1408 137.237354 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1409 137.237477 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1410 137.237584 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1412 137.277498 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1414 137.278074 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1415 137.286710 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1417 137.326588 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
1419 137.331700 192.168.1.253 192.168.1.168 TELNET Telnet Data ...
1420 137.333410 192.168.1.168 192.168.1.253 TELNET Telnet Data ...
Frame 1419 (151 bytes on wire, 151 bytes captured)
Ethernet II, Src: Dell_4f:9d:3a (00:13:72:4f:9d:3a), Dst: AsustekC_14:99:f4 (00:15:f2:14:99:f4)
Internet Protocol, Src: 192.168.1.253 (192.168.1.253), Dst: 192.168.1.168 (192.168.1.168)
Transmission Control Protocol, Src Port: telnet (23), Dst Port: 1229 (1229), Seq: 40, Ack: 53, Len: 97
Telnet
Command: Don't Echo
Command: Will Echo
Data: Red Hat Enterprise Linux AS release 4 (Nahant Update 1)\r\n
Data: Kernel 2.6.9-11.ELsmp on an i686\r\n
Some prompt information given by the server
Data and control multiplexed over the same connection
0000 00 15 f2 14 99 f4 00 13 72 4f 9d 3a 08 00 45 10 .... rO.:..E.
0010 00 89 17 c6 40 00 40 06 9d a3 c0 a8 01 fd c0 a8 ....@.@. ....
0020 01 a8 00 17 04 cd 64 98 7a 18 8d 0c b3 45 50 18 ......d. z....EP.
0030 05 b4 d0 1e 00 00 ff fe 01 ff fb 01 52 65 64 20 ....Red
0040 48 61 74 20 45 6e 74 65 72 70 72 69 73 65 20 4c Hat Ente rprise L
0050 69 6e 75 78 20 41 53 20 72 65 6c 65 61 73 65 20 inux AS release
0060 34 20 28 4e 61 68 61 6e 74 20 55 70 64 61 74 65 4 (Nahan t Update
0070 20 31 29 0d 0a 4b 65 72 6e 65 6c 20 32 2e 36 2e 1)..Ker nel 2.6.
0080 39 2d 31 31 2e 45 4c 73 6d 70 20 6f 6e 20 61 6e 9-11.ELs mp on an
0090 20 69 36 38 36 0d 0a i686..

DNS

Basic Functions of DNS (2)

- Translating between addresses
  - Hostname (www.company-a.com)
    - Convert between Domain Name and IP
  - IP address (128.9.32.254)
    - Convert between IP and MAC
  - MAC address (C2-67-3E-95-C5-F1)



A Short History (3)

– from flat namespace to hierarchical namespace

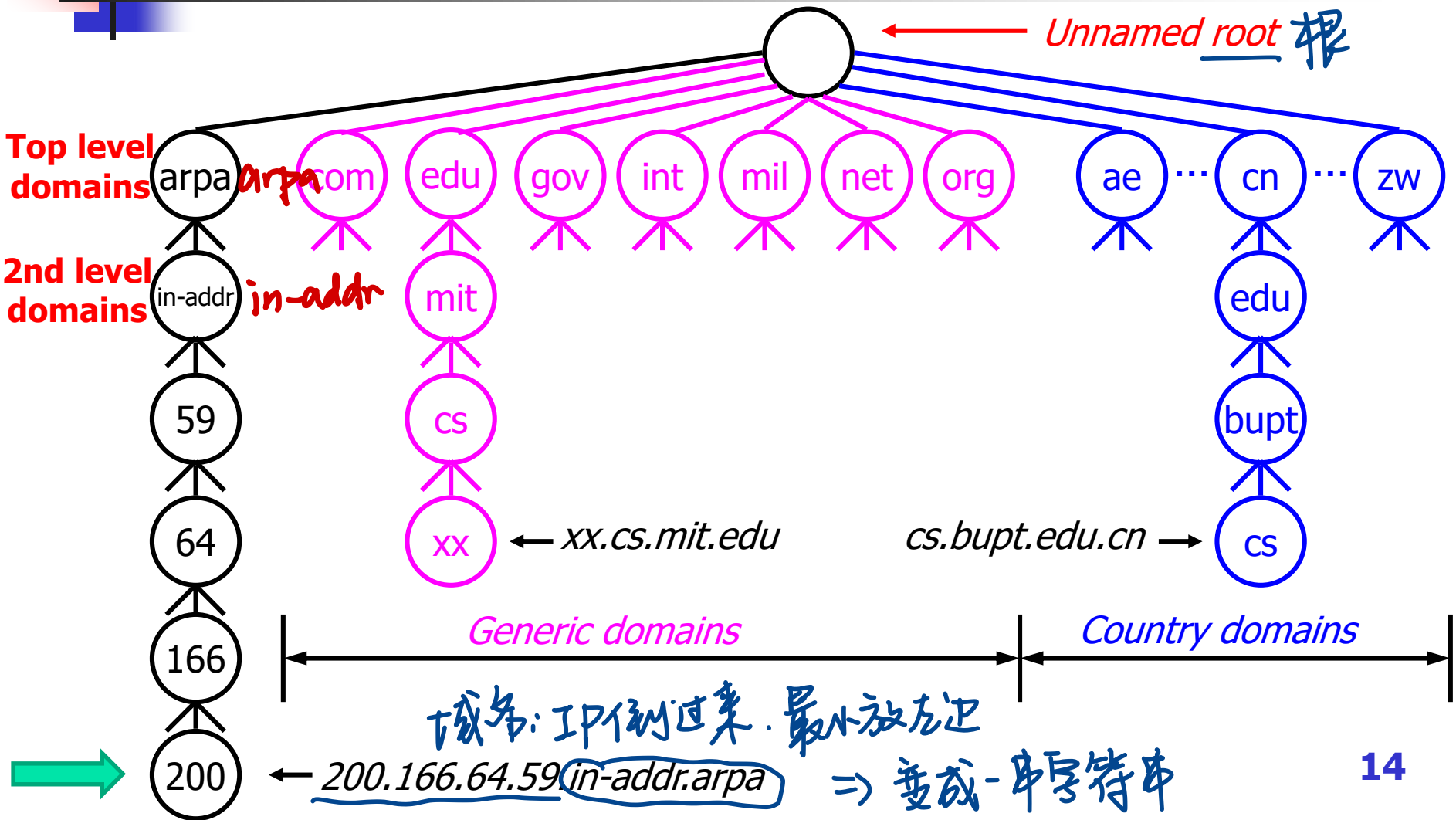
- Nowaday status
  1. ***Hierarchical structure***
  2. ***Distributed database***
  3. ***Efficient, reliable, general purpose***
  - It is primarily used for mapping host names and email destinations to IP addresses - but can be used for other purposes
  - It is a query / response protocol running on top of UDP/TCP, with default port number 53

Summary: What is DNS?

- Domain Name System
- A distributed database providing mapping between Domain name and IP address
  - Implemented in hierarchy of many name servers
- An application protocol
  - Used by hosts and name servers to communicate to resolve names

Domain Namespace (1)

– the hierarchical structure



Domain Namespace (2)

– organization of domains

- Each element of the hierarchy is referred to as a domain
- At the top of the hierarchy is the root domain, known as simply "."
- Subdomains directly underneath the root domain are called top-level domains
  - gTLD: .edu/.gov/.com/.org/.net/.mil/.int
  - other gTLD: .tv/.cc/.asia/.info/.store/.web/.biz/.tel/.mobi/.中国
  - country top level domain (ccTLD): .cn/.us/.eu/.kr/…
  - Infrastructure TLD: .arpa
  - New gTLD: under discussion
- Domains directly underneath top-level domains are called second-level domains, and so on

Domain Namespace (3)

– the domain names

- The domain name of a node is the list of the labels on the path from the node to the root of the tree separated by dots(“.”)
- Conventionally printed or read left to right, from the most specific (lowest, farthest from the root) to the least specific (highest, closest to the root)
  - eg. www.bupt.edu.cn
- Case insensitive
- Components can be up to 63 characters long, the full pathname must not be more than 255 characters
- The full name of a domain is also called its Fully Qualified Domain Name (FQDN)

Resource Record

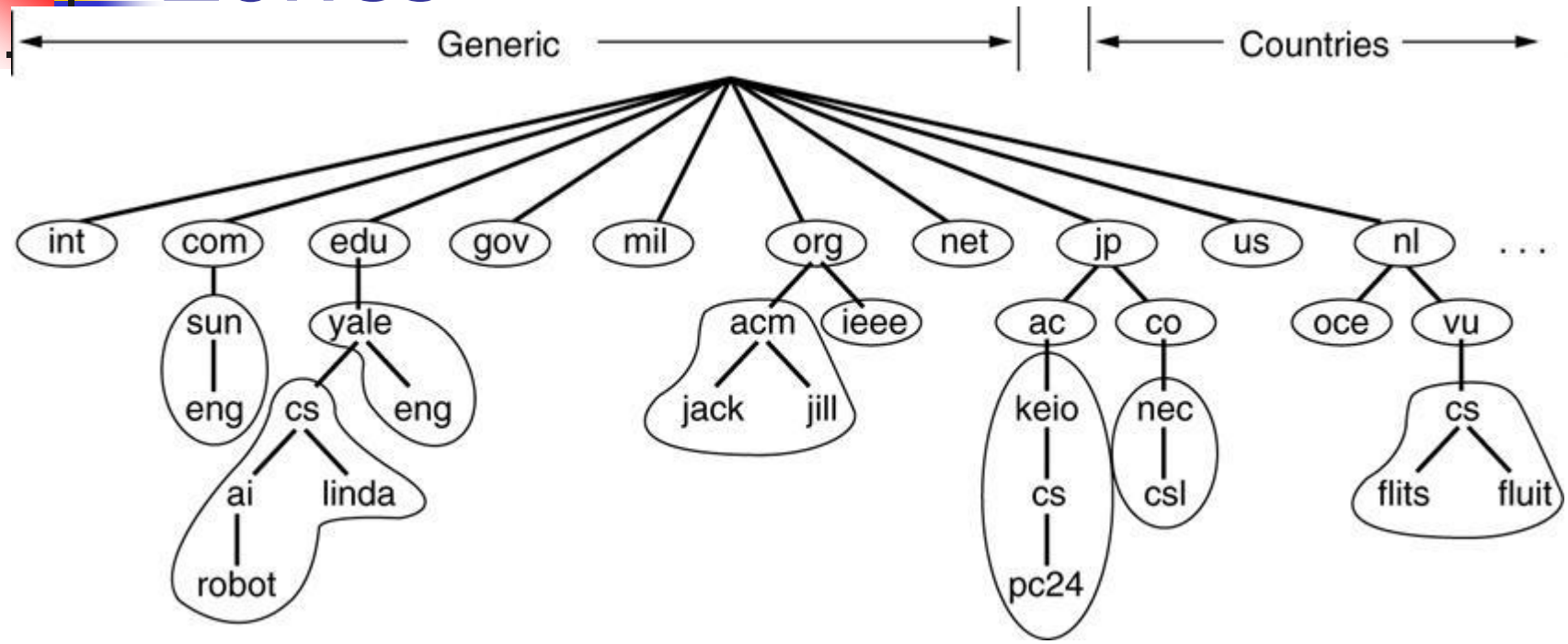
- Each domain in the DNS has one or more Resource Records (RRs), which are fields that contain information about that domain

- Each RR has the following information
  - Owner: the domain name where the RR is found
  - Type: specifies the type of the resource in this RR
    - A – Host IPv4 Address, AAAA - Host IPv6 Address
    - MX – Mail Exchanger
    - NS – Name Server
    - CNAME – Canonical Name
    - ...
  - Class: specifies the protocol family to use
    - IN – the Internet system
  - TTL: specifies the Time To Live (in unit of second) of the cached RRs
  - RDATA: the resource data

Name Servers (1)

- Name servers are the repositories of information that make up the domain database.
- The database is divided up into sections called zones, which are distributed among the name servers. A zone may be (one or more domains or even a sub-domain)
- Each name server handles one or more zones. And the essential task of a name server is to answer queries using data in its zones.
- Name servers can answer queries in a simple manner. The response can always be generated using only local data, and either contains the answer to the question or a referral to other name servers "closer" to the desired information.
- A given zone will be available from several name servers to ensure its availability.

Zones



- A zone corresponds to an administrative authority that is responsible for that portion of the hierarchy
- Eg. BUPT controls *x.bupt.edu.cn*

Name Servers (2)

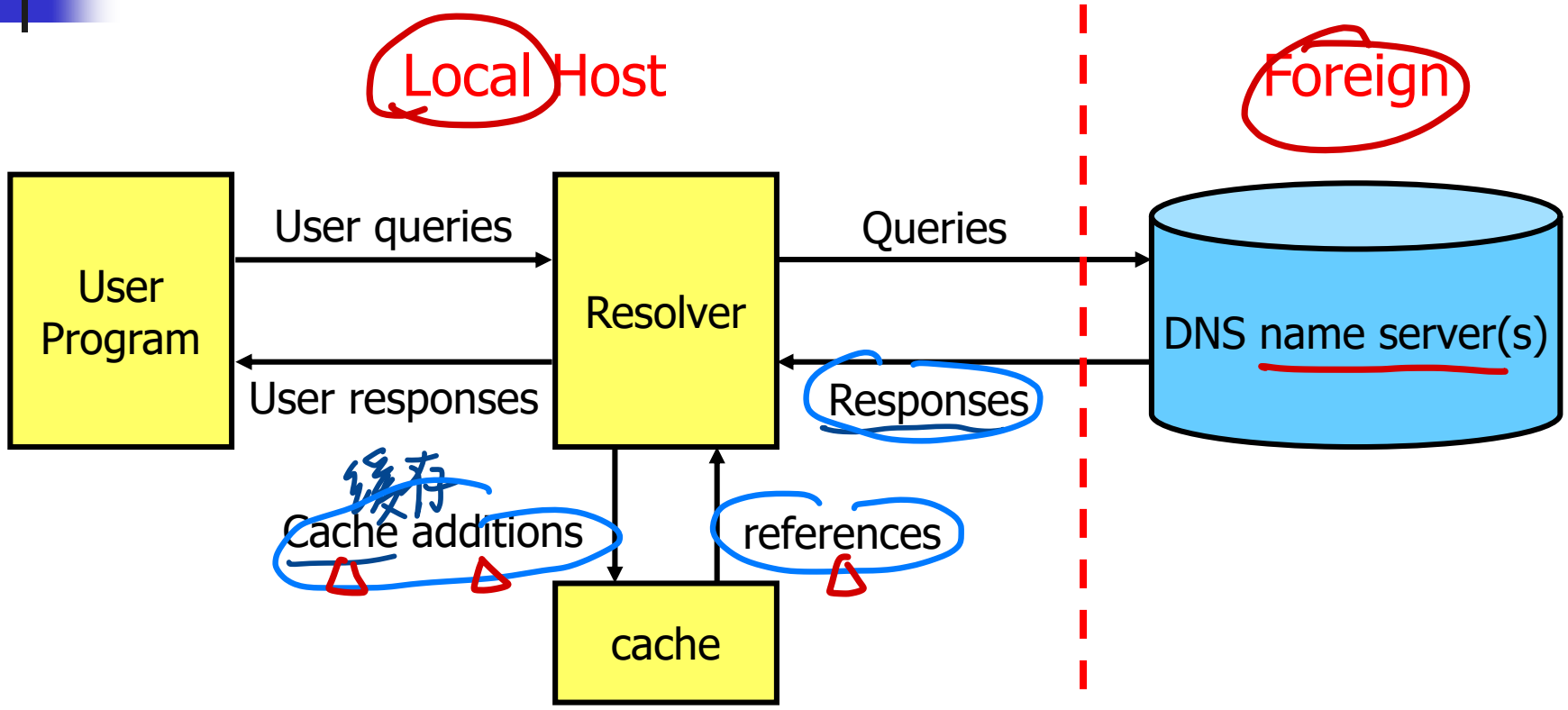
- Primary server / Authoritative server
  - holds in its database the name-to-address mappings for the group of hosts it administers
  - knows the official answer
- Secondary server
  - maintains a copy of the Primary Server's database
- Caching server
  - asks DNS queries to other servers but maintains a cache of the responses together with a "time to live" value
  - non-authoritative data about other parts of the tree

Name Resolvers (1)

- The client side of DNS

- A resolver is the interface between the user program and the domain name servers
  - A resolver receives a request from a user program (e.g., mail programs, TELNET, FTP)
  - asks questions to the DNS system on behalf of the application
  - returns the desired information

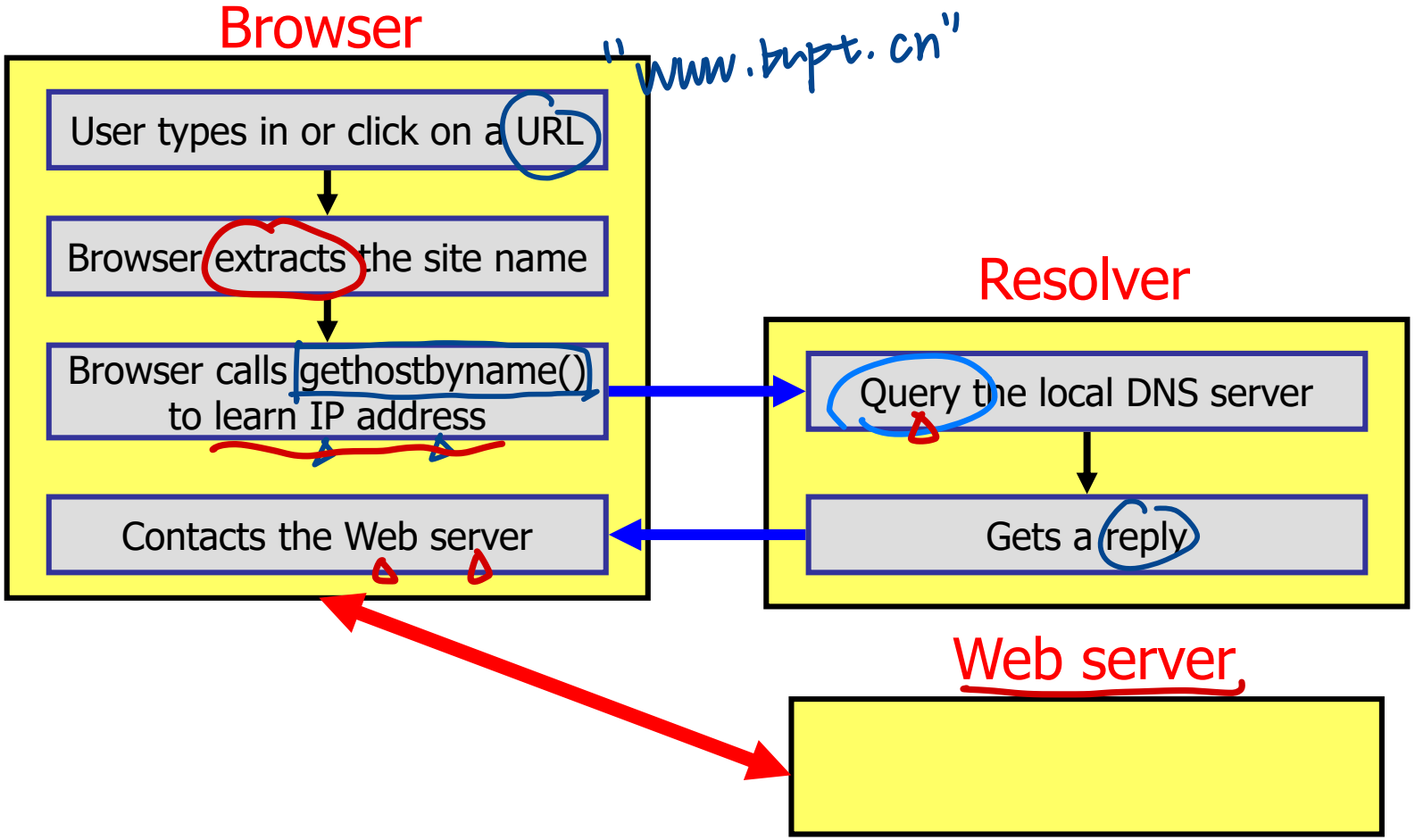
Name Resolvers (2)
– communication model



Resolvers are normally implemented in system calls.
E.g. gethostbyname(), gethostbyaddr()

Name Resolvers (3)
– an example

- DNS working together with HTTP application



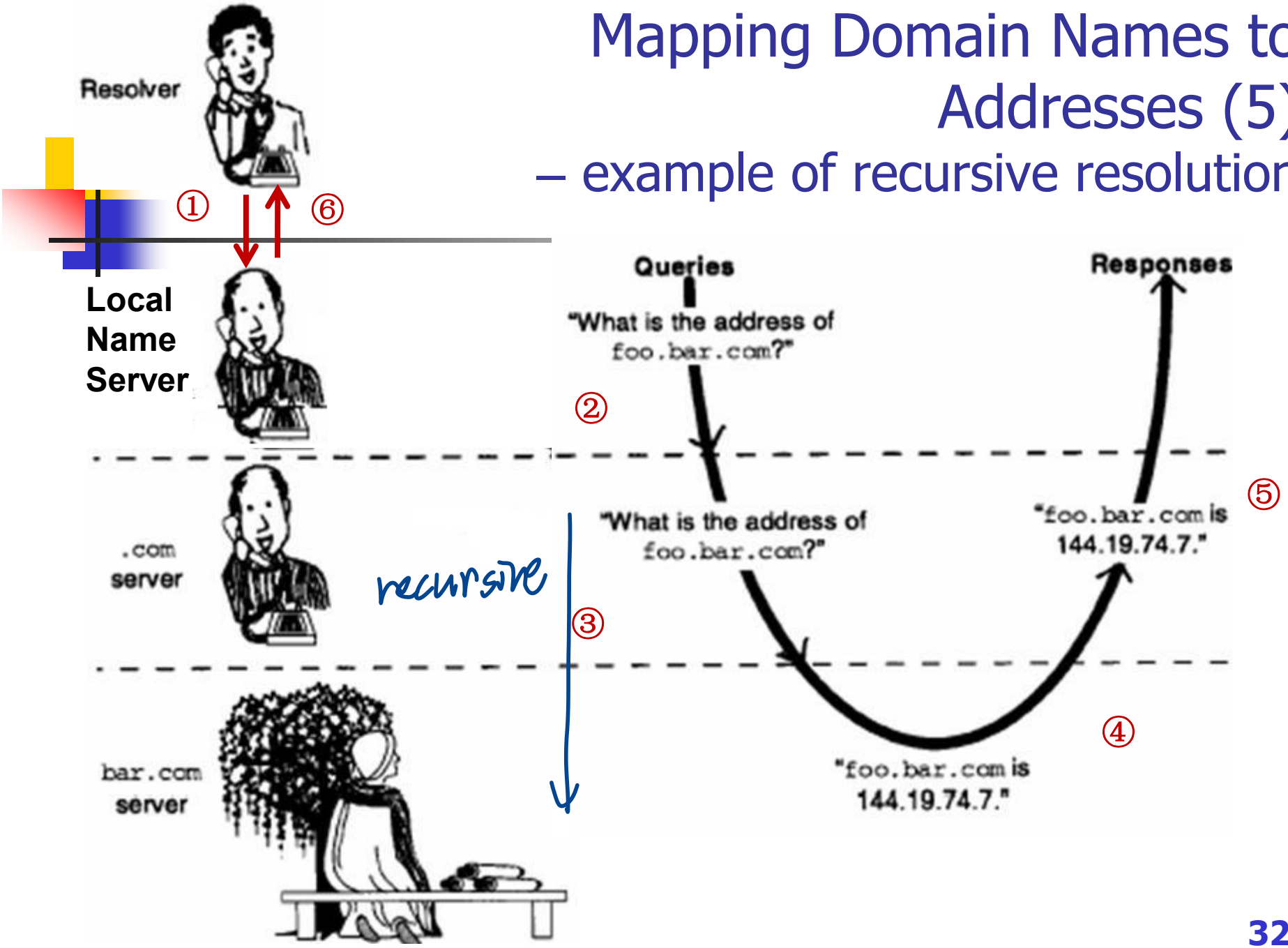
Mapping Domain Names to Addresses (3) – name resolution process

- Two-step name resolution process
  - Beginning with the local name server (default name server)
  - If the local server can not resolve a name, the query must be sent to another server in the domain system
- It can improve the query efficiency because most queries to name servers refer to local name.

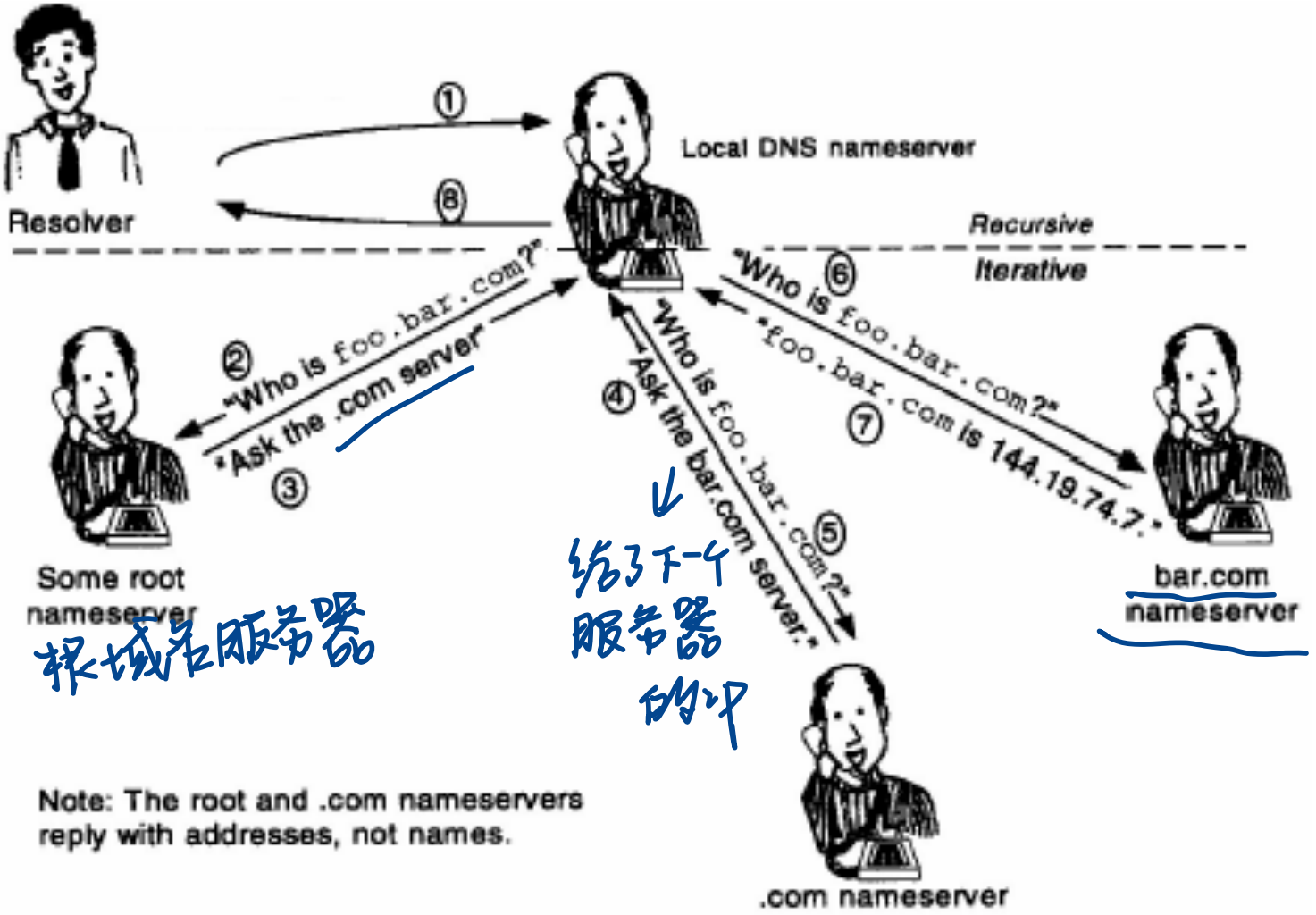
Mapping Domain Names to Addresses (4) – two name resolution methods

- A query is made to a local name server. If local name server can not resolve the name, then it will query another server.
- Recursive resolution:
  - If the queried server does not have the information, it must make the appropriate query or queries to get the information
  - Generally, a server fulfills a recursive query either with data in its own memory or by making another recursive query
- Iterative resolution:
  - If the queried server does not have the information, it may then respond with the address of another server; the local name server (on behalf of resolver)then queries that server (which might respond with the address of another server, and so on)
  - Commonly used by name servers on the Internet

Mapping Domain Names to Addresses (5)
– example of recursive resolution

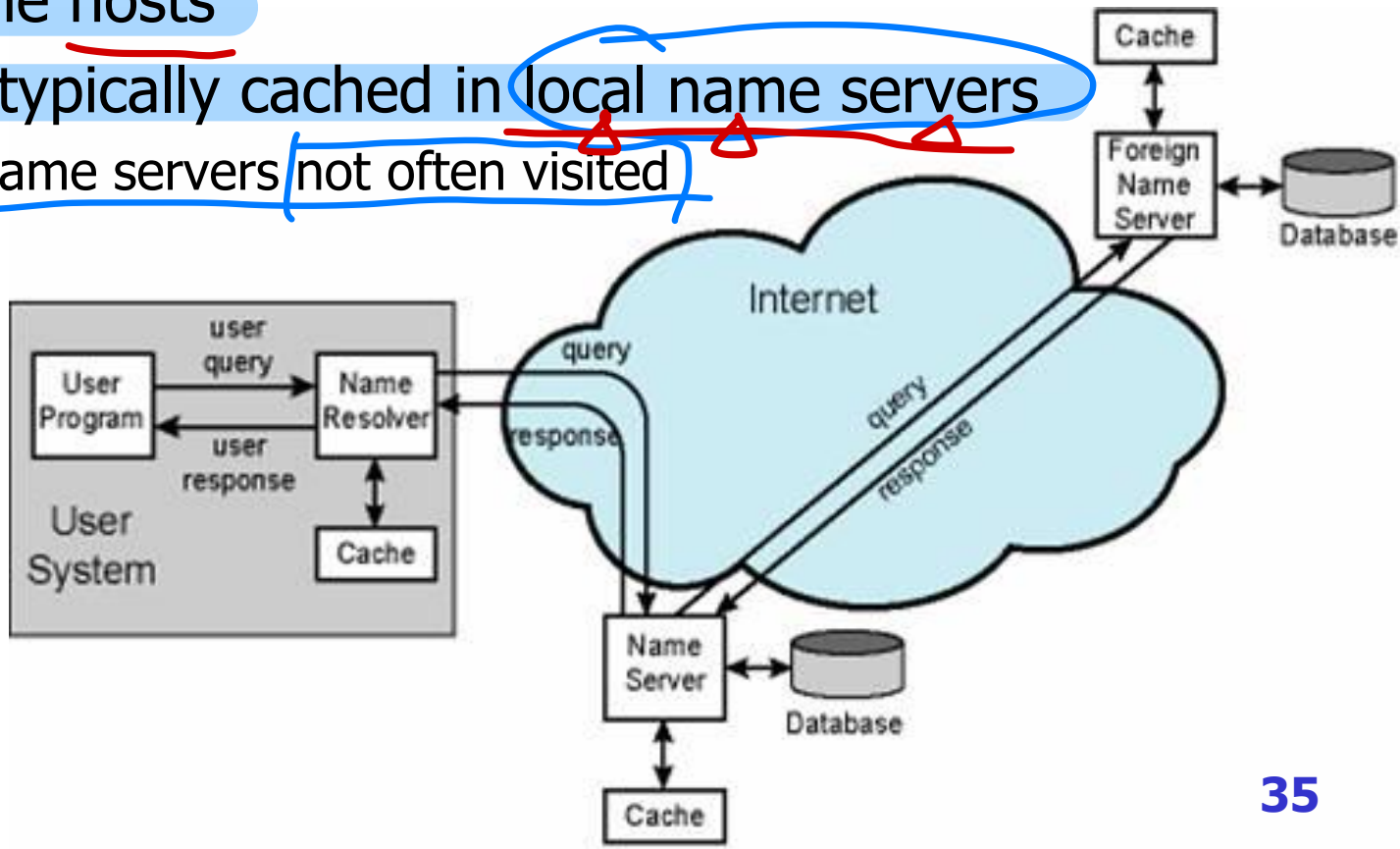


Mapping Domain Names to Addresses (6) – example of iterative resolution

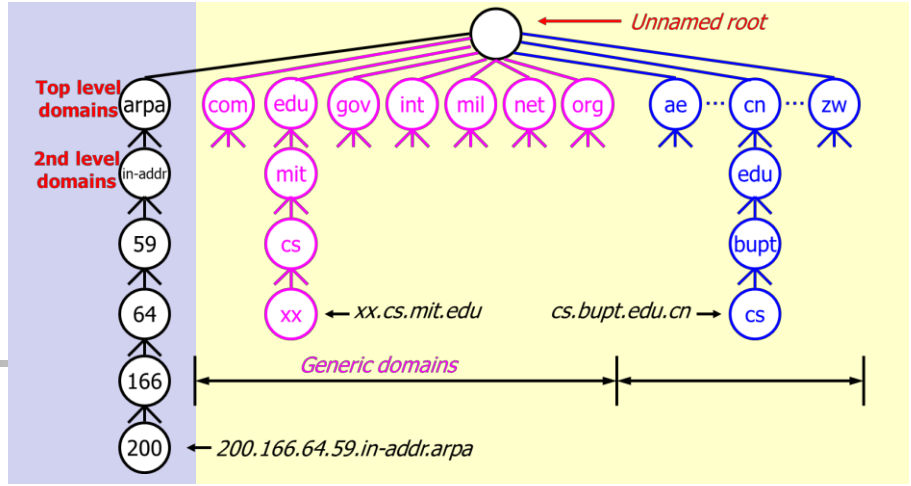


Mapping Domain Names to Addresses (7)
– caching mechanism to improve efficiency

- Caching at the name servers
- Caching at the hosts
- TLD servers typically cached in local name servers
  - Thus root name servers not often visited



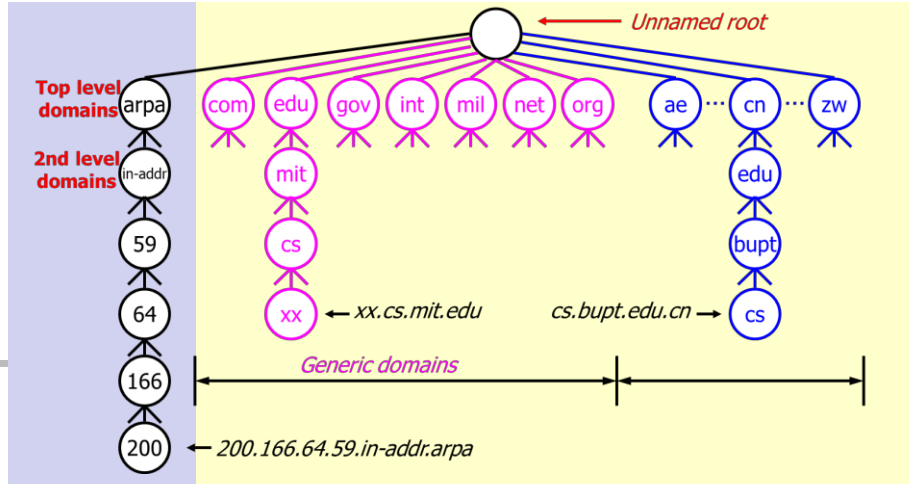
Inverse Queries



- Mapping a particular resource to a domain name or domain names that have that resource
  - Standard query: mapping a domain name to a resource
  - Inverse query: mapping a resource to a domain name
- Optional part of DNS
- Generally NOT used because there is often no way to find the server that can resolve the query without searching the entire set of servers
- NOT an acceptable method of mapping host addresses to host names, use the IN-ADDR.ARPA domain instead

建立新数据库，以IP地址为索引

Pointer Queries



- Using IN-ADDR.ARPA domain for address to host mapping
  - An IP address in dotted-decimal format is included in the query
  - The correct domain name for the machine with the specific IP address

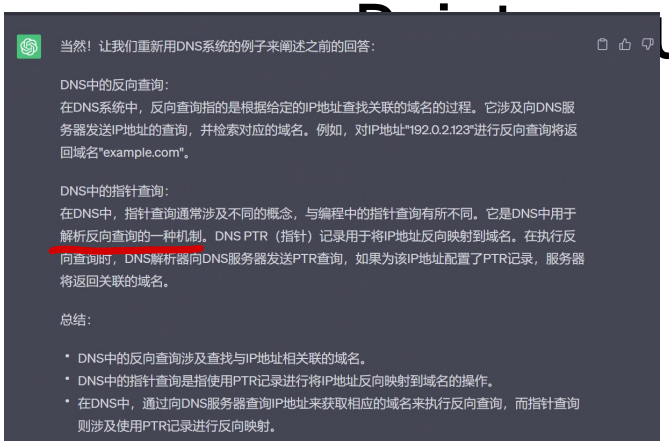
IPv4 地址 + .in-addr.arpa

- Note: Since the IN-ADDR.ARPA special domain and the normal domain for a particular host or gateway will be in different zones, the possibility exists that that the data may be inconsistent

例如，要对IP地址进行反向查找，将查找8.8.4.4域名的PTR记录4.4.8.8.in-addr.arpa，并找到指向该域名的PTR记录google-public-dns-b.google.com。如果A记录的google-public-dns-``b.google.com依次指向回8.8.4.4那么就可以说是向前证实。

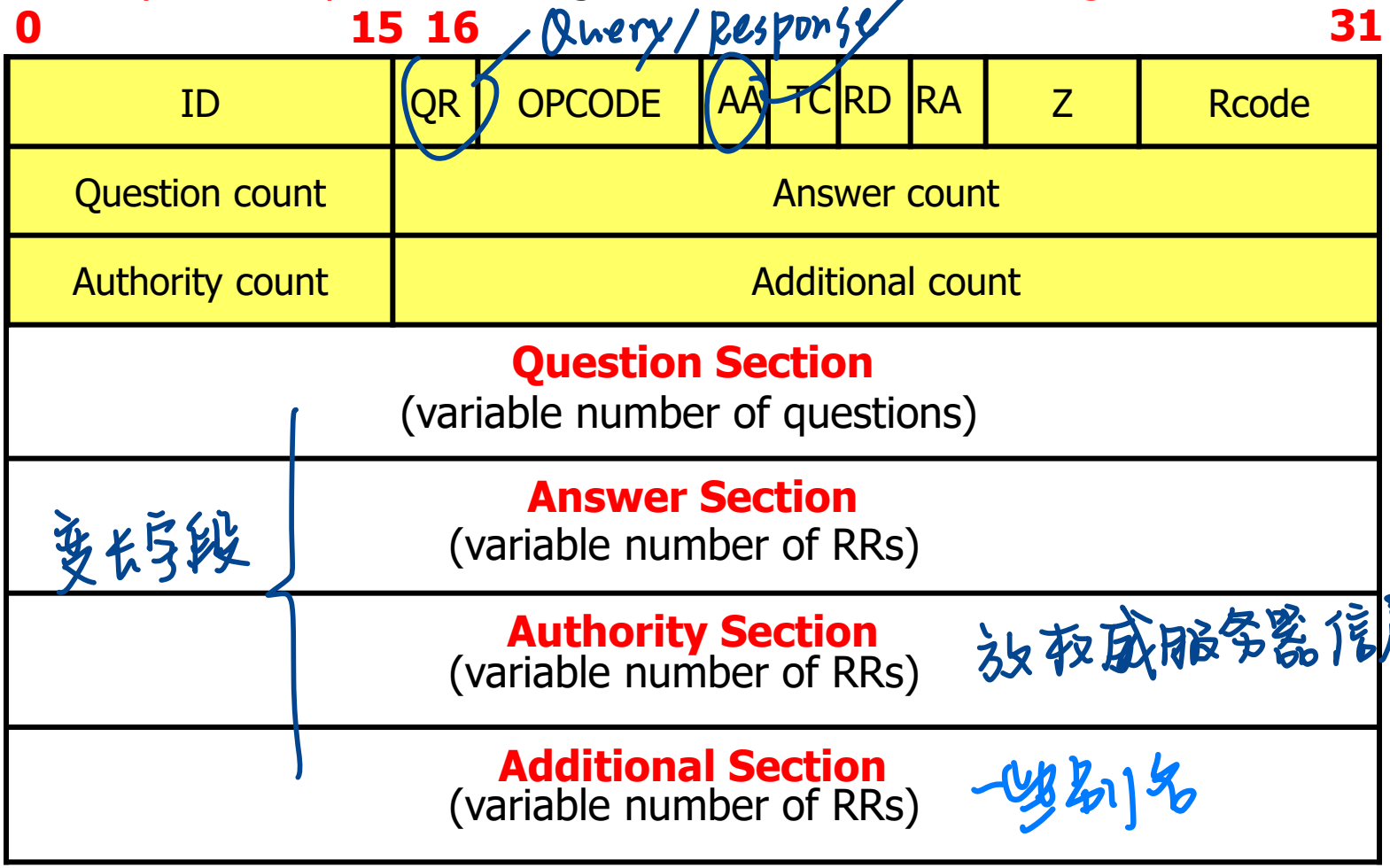
Inverse query vs. Pointer query

- Similarity: IP address → domain name
- Differences:
  - Inverse query uses the same domains as standard query, may need to search the entire set of servers
  - uery uses IN-ADDR.ARPA domain



DNS Message Format (from RFC 1035)

- *Query* and *Response* messages, both with *same message format*



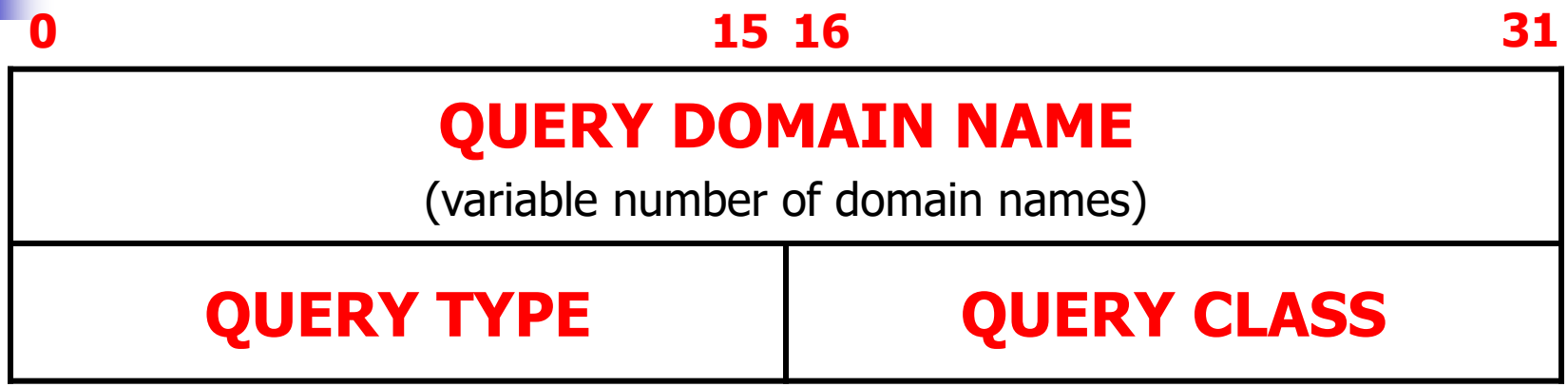
DNS Message Header Structure(1)

- ID: 16-bit field used to correlate queries and responses.
- QR: 1-bit field that identifies the message as a query (0) or response (1).
- OPCODE: 4-bit field that describes the type of query:
  - 0: Standard query (name to address). 1: Inverse query (address to name). 2: Server status request.
- AA: Authoritative Answer. 1-bit field. When set to 1, identifies this response is made by an authoritative name server.
- TC: Truncation. 1-bit field. When set to 1, indicates the message has been truncated due to length greater than that permitted.
- RD: Recursion Desired. 1-bit field. Set to 1 by the resolver to request recursive service by the name server.
- RA: Recursion Available. 1-bit field. Set to 1 by name server to indicate recursive query support is available.
- Z: 3-bit field. Reserved for future use. Must be set to 0.

DNS Message Header Structure(2)

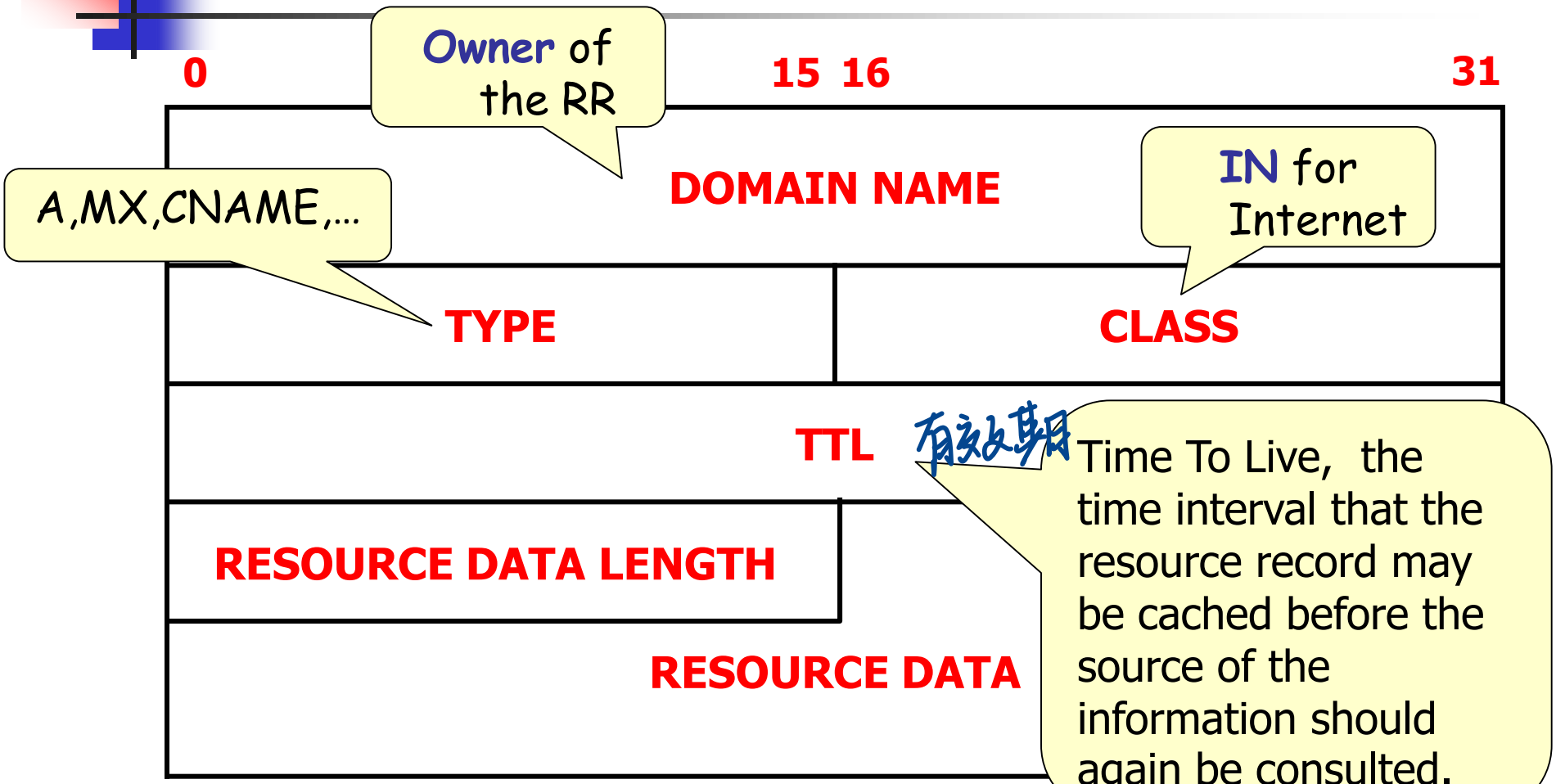
- RCODE: Response Code. 4-bit field that is set by the name server to identify the status of the query:
  - 0: No error condition. 1: Unable to interpret query due to format error.
  - 2: Unable to process due to server failure. 3: Name in query does not exist. 4: Type of query not supported. 5:Query refused for policy reasons.
- QDCOUNT(Question count): 16-bit field that defines the number of entries in the question section.
- ANCOUNT(Answer count): 16-bit field that defines the number of resource records in the answer section.
- NSCOUNT(Authority count): 16-bit field that defines the number of name server resource records in the authority section.
- ARCOUNT(Additional count): 16-bit field that defines the number of resource records in the additional records section.

Question Section Format



- QUERY TYPE: 16-bit field used to specify the type of the query
  - A – Host address
  - MX – Mail exchanger for the domain
  - …
- QUERY CLASS: 16-bit field used to specify the class of the query
  - IN – Internet system
  - …

Response Section: Resource Record Format



Resource Record Format (2) – type field

- SOA
  - Start Of Authority--identifies the domain or zone and sets a number of parameters
- NS
  - Maps a domain name to the name of a computer that is authoritative for the domain
- A
  - Maps the name of a system to its address. If a system (e.g., a router) has several addresses, then there will be a separate record for each.
- AAAA
  - Maps the name of a system to its IPv6 address. If a system (e.g., a router) has several addresses, then there will be a separate record for each.
- CNAME
  - Maps an alias name to the true, canonical name
- MX
  - Mail Exchanger. Identifies the systems that relay mail into the organization

Resource Record Format (3)
– type field

- TXT
  - Provides a way to add text comments to the database. For example, a txt record could map abc.com to the company's name, address, and telephone number
- WKS
  - Well Known Services. It can list the application services available at the host. Used sparingly, if at all
- HINFO
  - Host Information, such as computer type and model. Rarely used
- PTR
  - Maps an IP address to a system name. Used in address-to-name files.

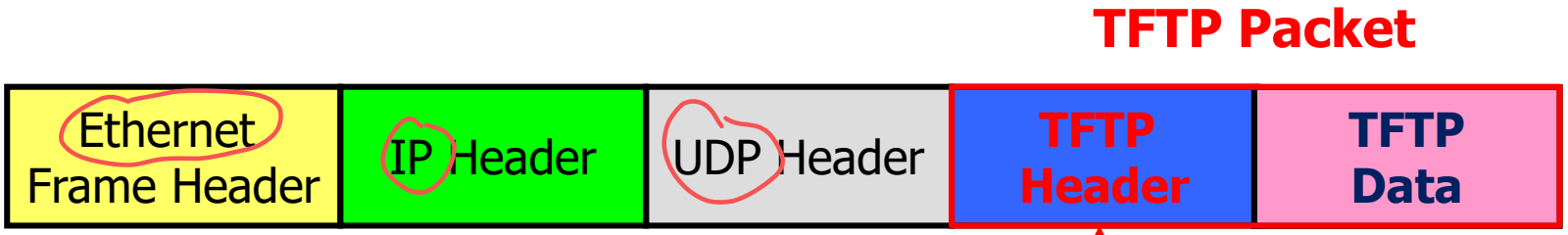
FTP

TFTP Features

- Read and write files from / to remote computers
- Minimal overhead (no security)
- Designed for UDP, although could be used with other transport protocols 功能较简化，占代码量小
- Easy to implement
- Small – possible to include in firmware
- Often uses to bootstrap workstations and network devices 引导
- No Access Control / No Directory Retrieval 无登录和密码 无目录

TFTP Protocols – Packet Format (1)

- Order of headers



TFTP header consists of a 2 byte opcode field indicating the packet's type:

1	Read request (RRQ) 下载请求
2	Write request (WRQ) 上传请求
3	Data (DATA)
4	Acknowledgment (ACK) 确认
5	Error (ERROR) 差错提示

RRQ
WWQ

TFTP Operations – Transfer Mode

- Netascii - for transferring text files
  - all lines end with \r\n
  - provides standard format for transferring text files
  - both ends responsible for converting to/from netascii format
- Octet - for transferring binary files
  - no translation done

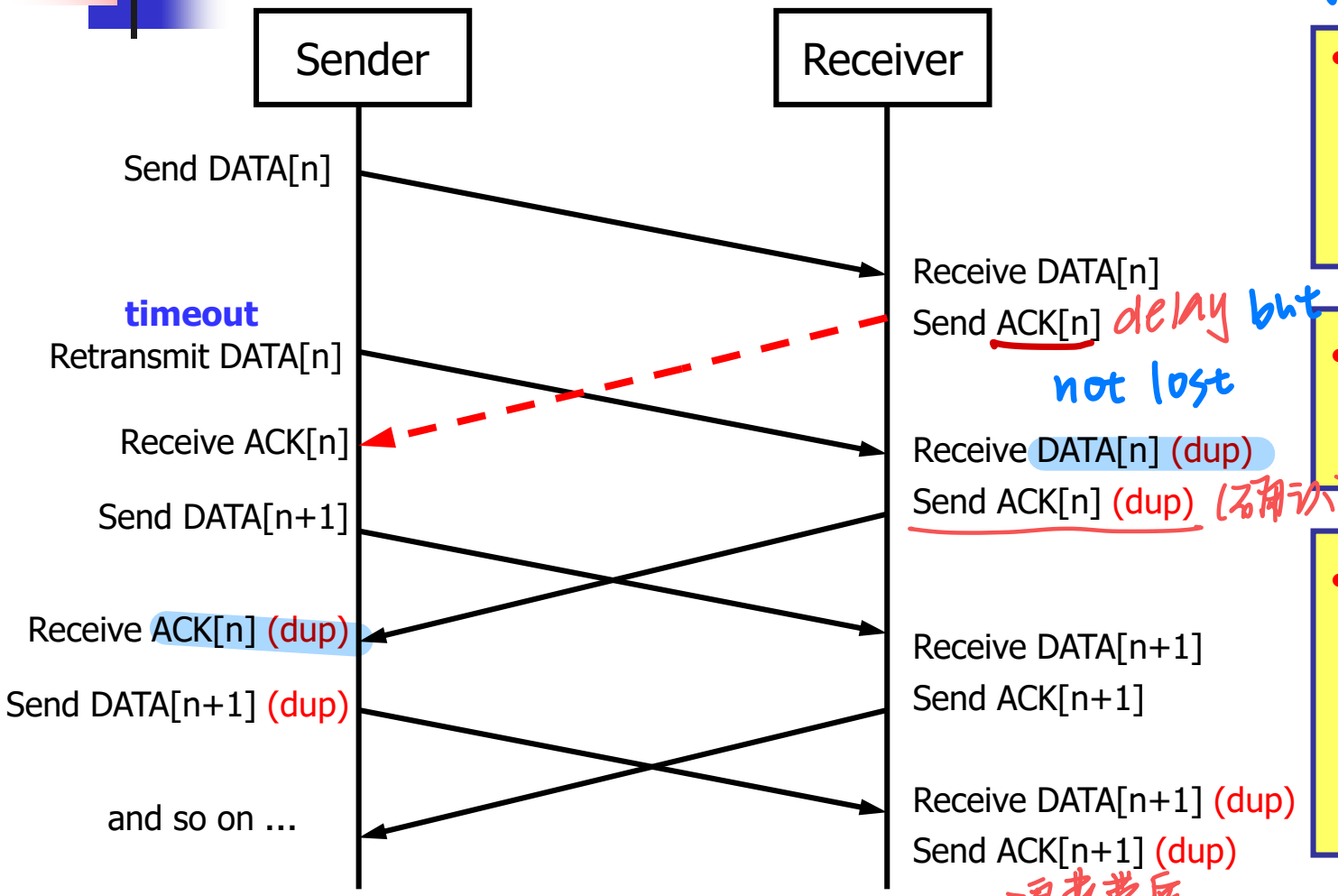


TFTP Operations – Retransmission

- Symmetric
  - Both machines involved in a transfer are considered senders and receivers.
    - One sends data and receives acknowledgments
    - The other receives data and sends acknowledgments
  - Each side implement the timeout and retransmission
    - If a data packet gets lost in the network, the data sender times out and retransmits the last data packet
    - If an acknowledgment is lost, the acknowledgment sender retransmits the last acknowledgment
- The sender has to keep just one packet on hand for retransmission, since the stop and wait mechanism guarantees that all older packets have been received
- Duplicate data packets must be recognized (ignored) and acknowledgment retransmitted
- This original protocol suffers from the *sorcerer's apprentice syndrome (SAS)*

TFTP Operations – Sorcerer's Apprentice Syndrome

过度重传，开始就无限循环



- Arising when an acknowledgment for a data packet is delayed but not lost
- Leading to excessive retransmissions
- Once started, the cycle continues indefinitely with each data packet being transmitted exactly twice

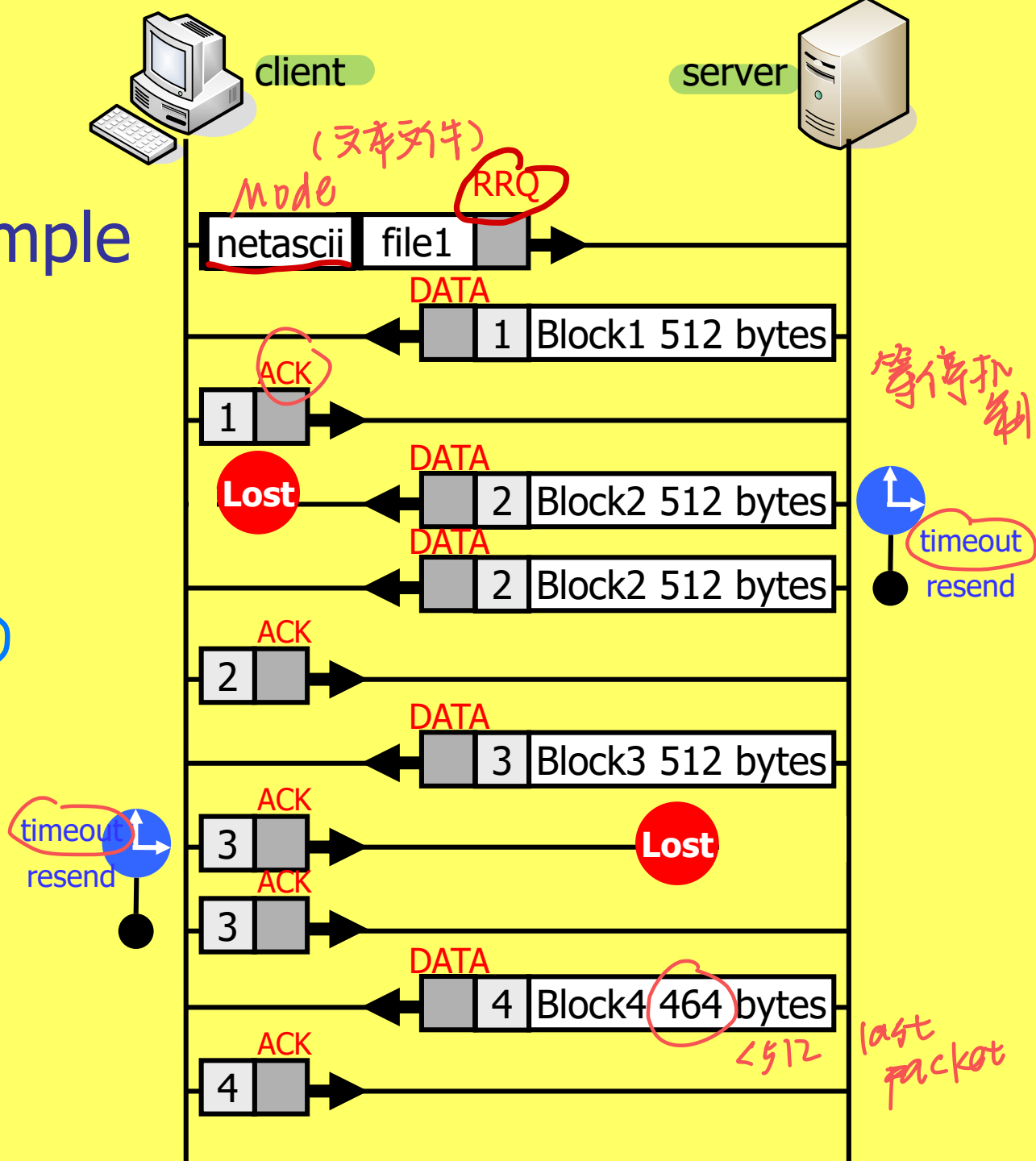
TFTP Operations – How to fix SAS

- Principle: *break the retransmission loop*
  - Sender should not resend a data packet in response to a duplicate ACK
  - If sender receives ACK[n] - don't send DATA[n+1] if the ACK was a duplicate

- See details in *RFC 1123*

TFTP Example

- Beginning with RRQ/WRQ
- Every good block is Acknowledged
- Both sides use timers
- Simple stop and wait mechanism provides reliable delivery and flow control
- A block of less than 512 bytes signals the end of the file



TFTP Summary

Server port 69

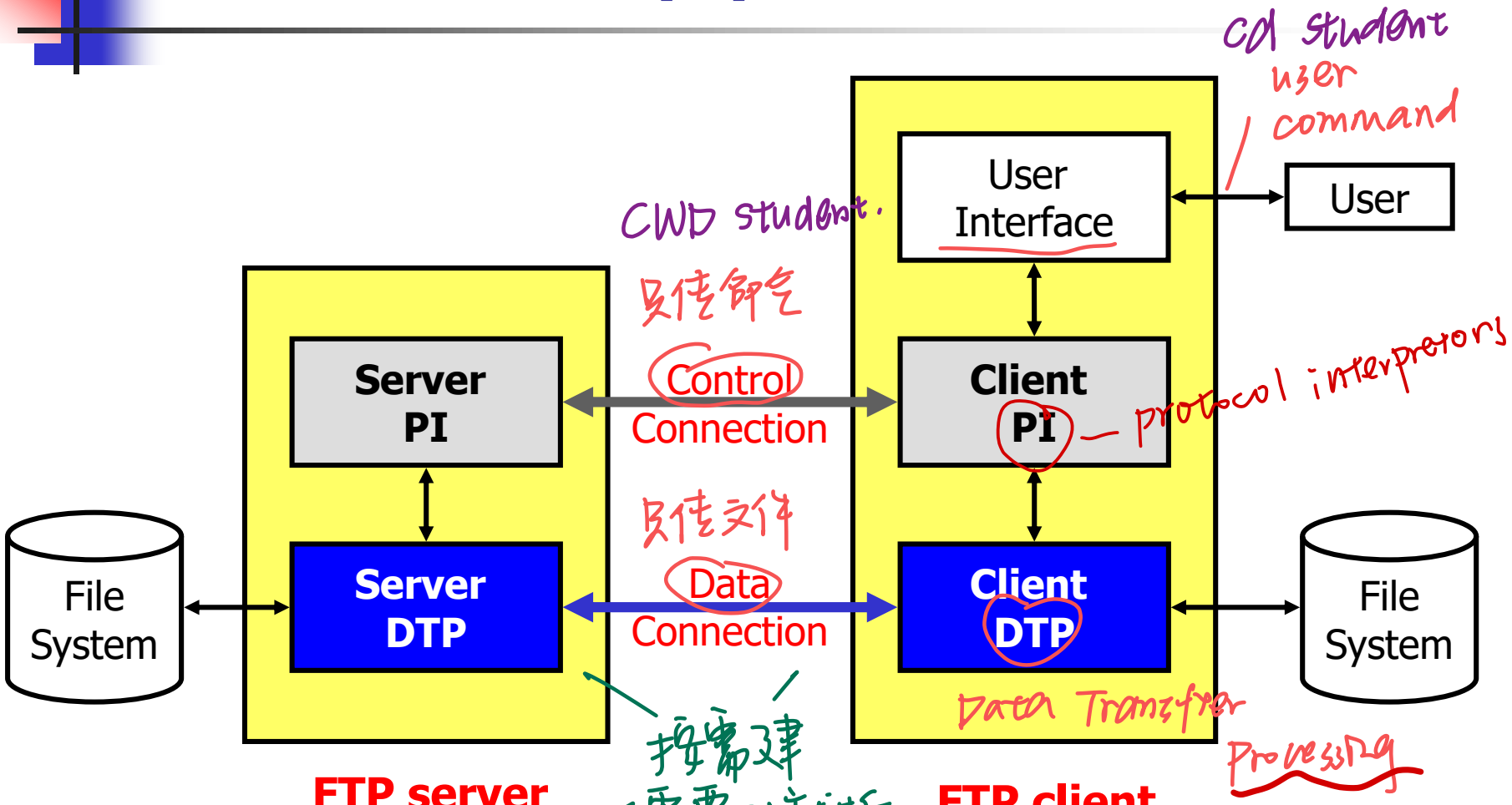
- Used to move files between machines on different networks implementing UDP, TFTP server waits for Read/Write request on port 69
- The protocol is very restrictive, in order to simplify implementation
- Used with BOOTP and DHCP Configuration applications (RFC 906, "Bootstrap loading using TFTP")
- The fixed length blocks make allocation straight forward, and the stop and wait mechanism provides reliable delivery, flow control and eliminates the need to reorder incoming data packets

对比：TFTP数据块可变长

FTP Features

- Used to transfer files between hosts
- Used to manipulate files, for example:
  1. List directories
  2. Delete files
  3. Rename files
- Uses TCP for reliable transfers
- Official Internet protocol

FTP Model (1)


User Interface
User
Server PI
Control Connection
Client PI
Server DTP
Data Connection
Client DTP
File System
File System
FTP server
FTP client

FTP Model (2)

- FTP Client
  - Users interact with Client directly
  - Active open of control connection
  - Control connection uses ASCII plain-text
  - Sends commands (over control connection)
  - Receives replies (over control connection)
  - Data connection used to transfer file data
- FTP Server
  - System process
  - "Listens" for connection on well-known port 21
  - Receives commands
  - Sends replies

FTP Model (3)

- PI (Protocol Interpreter): The user and server sides of the protocol have distinct roles implemented in a user-PI and a server-PI.
- DTP (Data Transfer Process): The data transfer process establishes and manages the data connection. The DTP can be *passive* or *active*.
- Control Connection: The communication path between the client-PI and server-PI for the exchange of commands and replies. This connection follows the Telnet Protocol.
- Data Connection: A full duplex connection over which data is transferred, in a specified mode and type. The data transferred may be a part of a file, an entire file or a number of files. The path may be between a server-DTP and a client-DTP, or between two server-DTPs.

FTP Model (5) - Protocol Interpreter (PI)

- Interprets the user's commands

用户输的　　真正传的

User Command	FTP Control Command
cd mystuff	CWD mystuff
get dns.pdf	RETR dns.pdf
put rfc1123.txt	STOR rfc1123.txt
dir	NLST

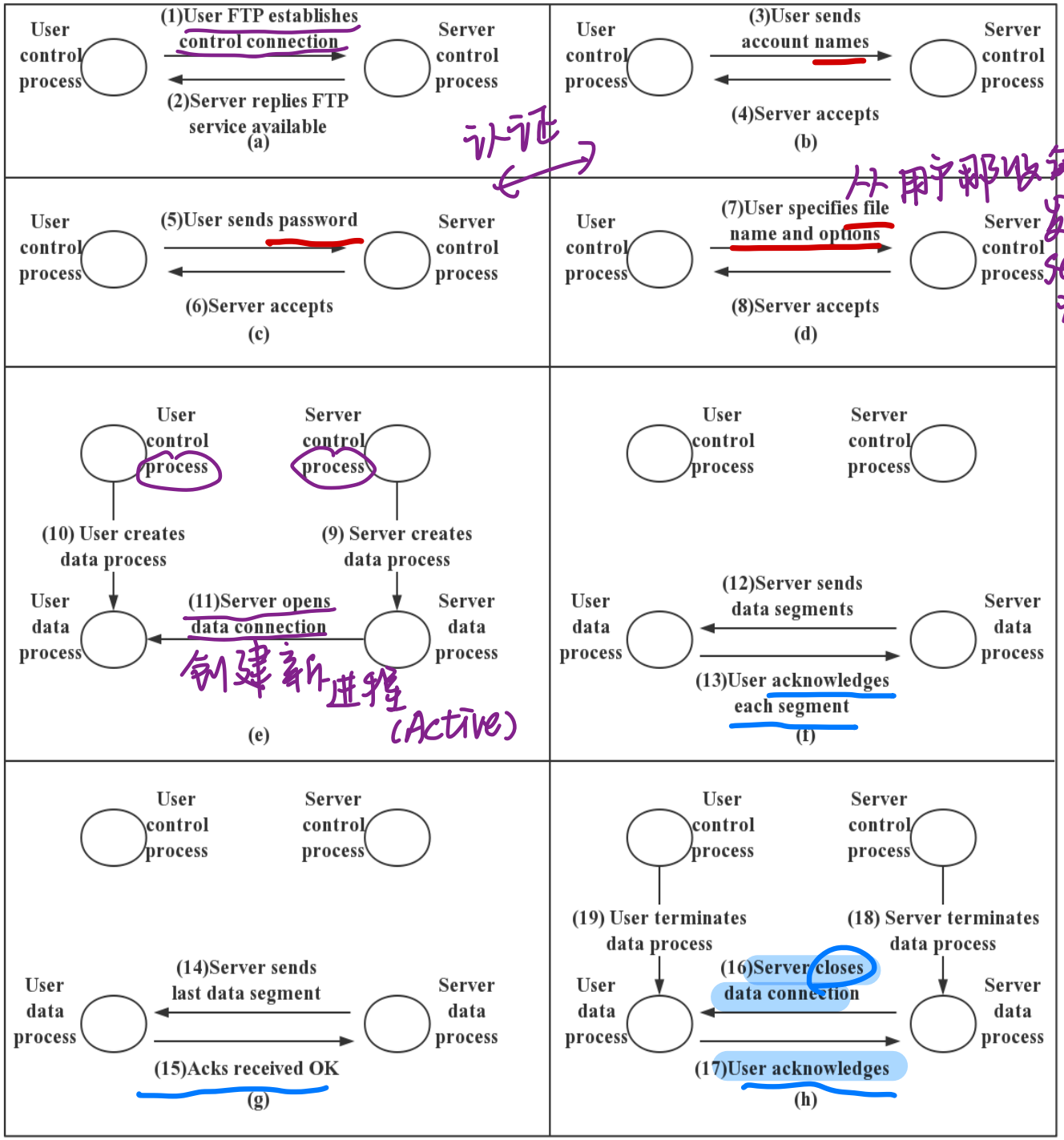
Overview of an FTP Transfer

Downloading a file in

Active Mode

↓

Server端主动建立Data连接



FTP Control Connection & Data Connection

- Control connection
  - Remain alive as long as the user keeps the FTP session active
  - Passing commands and replies
  - Used to coordinate the ports used for data connection and establish data connection
- Data connection
  - Be created dynamically when needed
  - One data connection persists for one file transfer
  - Used for data transmission

FTP Control Commands: Access control group

登录相关

- Specify user name: *USER* username
- Specify password: *PASS* password
- Specify account: *ACCT* account
- Change directory: *CWD* directory
  - *CDUP:* Change to parent directory
- Reinitialize: *REIN* 初始化
- Terminate session: *QUIT*

FTP Control Commands: Transfer parameter group(1)

- Define data connection port:
  - *PORT* h1,h2,h3,h4,p1,p2

    --used in *active mode*, telling FTP server the port number of client to accept data connection

    需知端口和IP才能建立连接

    ➢ Example of port number calculation

    PORT 210,25,137,230,23,189

    client 发送给 server

    IP: 210.25.137.230

    Port: 23*256+189=6077

    $p_1 \times 256 + p_2$

  - *PASV*

    --used in *passive mode*, informs server that client will contact to set up data connections

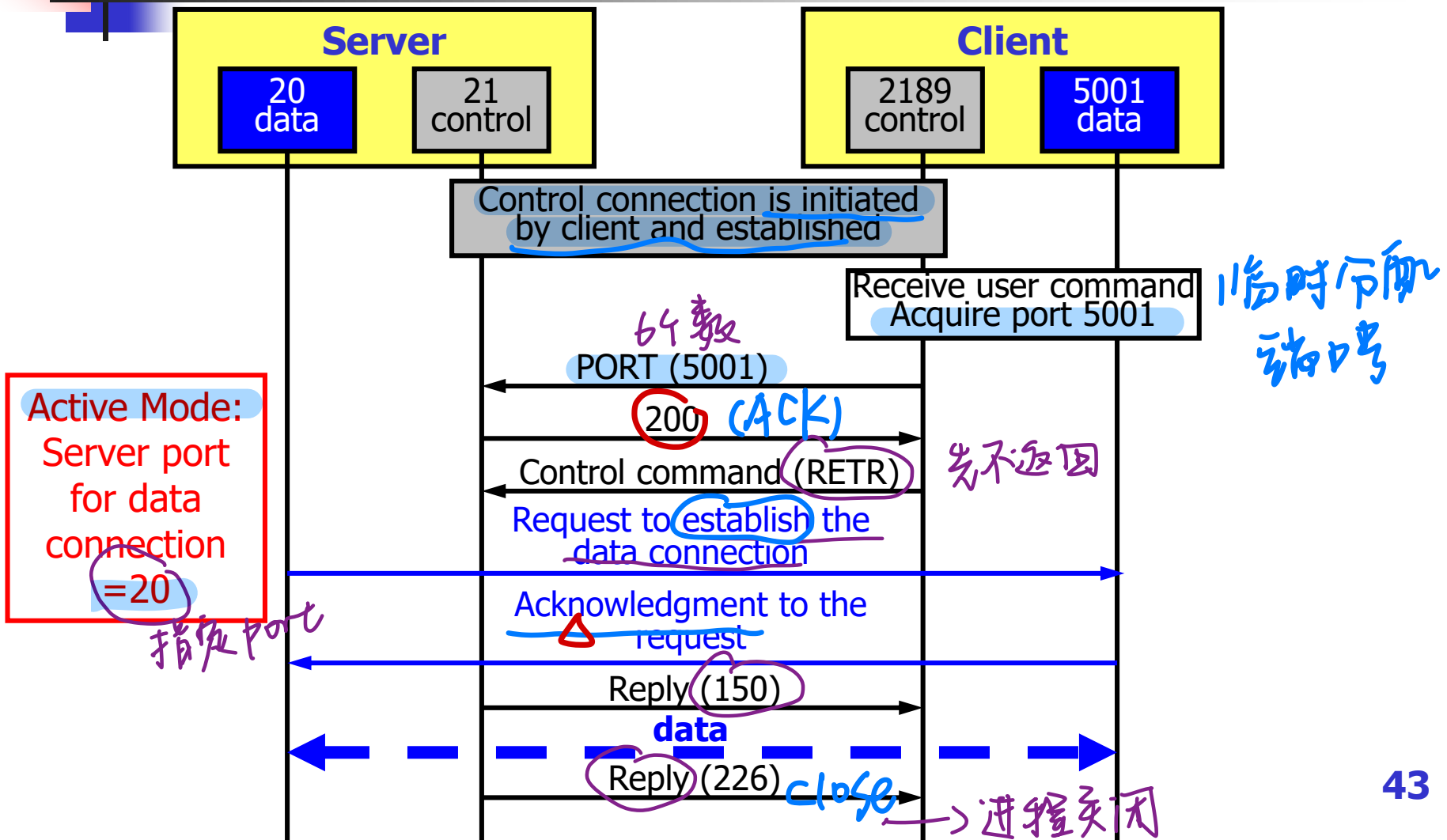
    server 回复时发自己的port和IP (用 h1 h2 h3 h4 p1 p2)

FTP Control Commands: Service group

- Retrieve (get) file: *RETR* file
- Store (put) file: *STOR* file
- Append to file: *APPE* file 拼接文件
- Delete a file: *DELE* file
- Create a directory: *MKD* directory
- Delete a directory: *RMD* directory
- Rename a file: *RNFR* file and *RNTO* file 重命名：发两个命令.
- List a directory: *LIST* spec and *NLST* spec
- Others such as *HELP*, *SITE*, *SYST*, ...

关control：client发QUIT，server发"221"回应

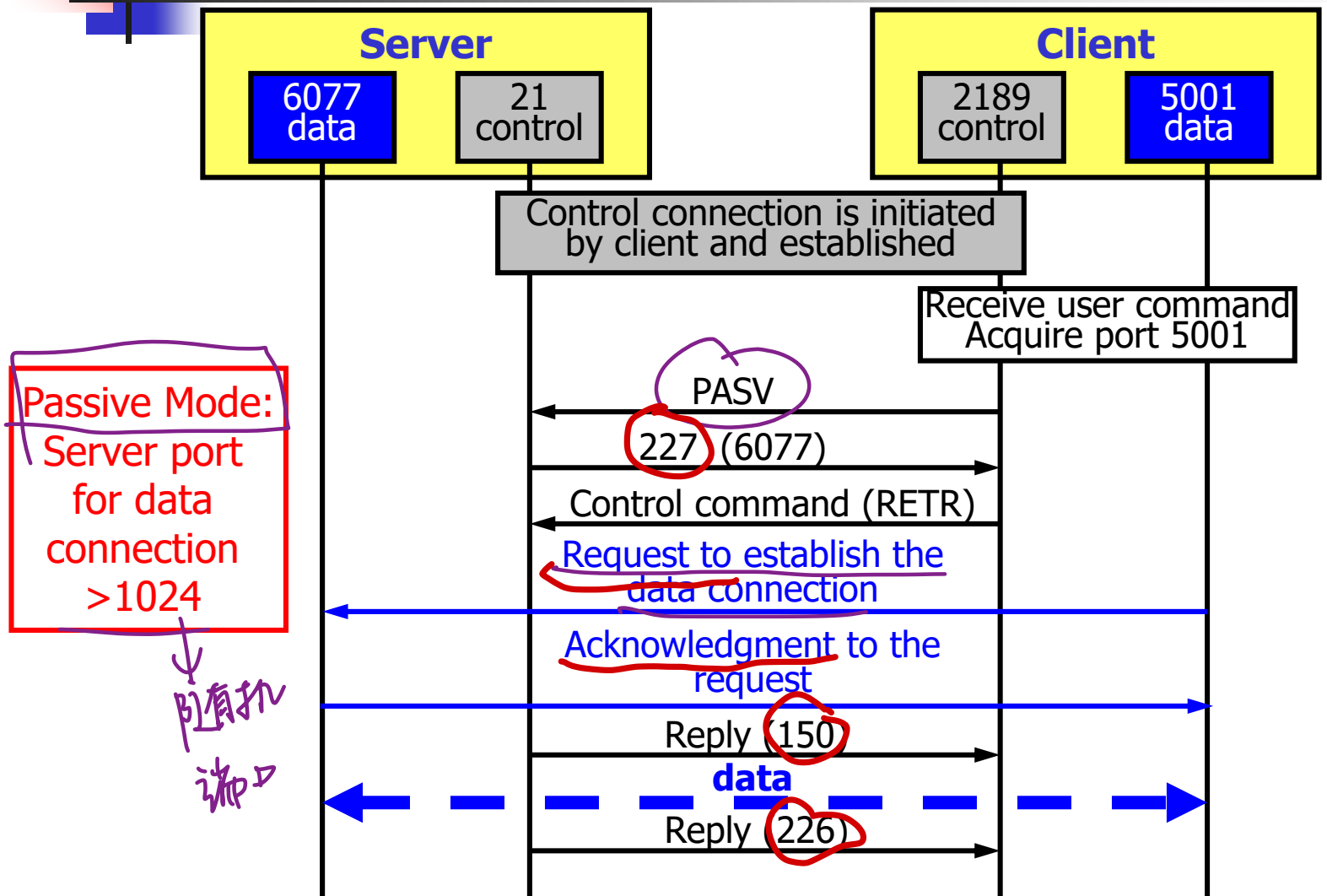
FTP Control Connection & Data Connection (1)



FTP Control Connection & Data Connection (2)

- Typical data connection handling sequence (in active mode)
  - Client sets up to "listen" on a unique port
  - Client uses local socket information to send PORT command to server
  - Server responds with "200" reply to acknowledge the port number
  - Client sends RETR, STOR, or other transfer command
  - Server sends preliminary reply
  - Server does active open ("connect")
  - File data sent over connection
  - Server sends "226" or other reply
  - Server/client closes data connection
- Another mode: passive mode
  - Client sends command *PASV*
  - server listens to a specific port and client should access that port

FTP Control Connection & Data Connection (3)

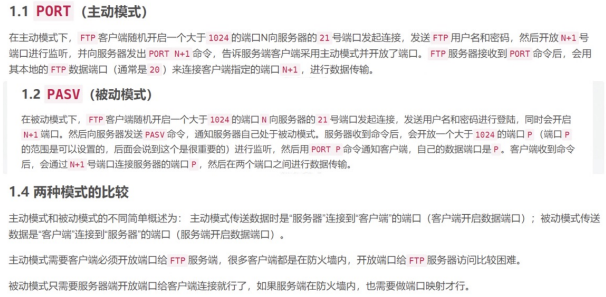


Comparing FTP/NFS

FTP	NFS
FTP just provides file transfer	NFS provides transparent file access for clients to files and file-systems on a server
With FTP a complete copy of the file is made	NFS accesses portions of a file
FTP uses TCP	NFS usually uses UDP on port 2049

Summary Of Key Points

- TFTP
  - TFTP packet format
  - TFTP transfer mode
  - Sorcerer's Apprentice Syndrome
  - Typical communication procedure
- FTP
  - FTP model
  - FTP control commands and replies
  - User commands vs. Control commands
  - Control connection vs. Data connection
  - Active FTP vs. Passive FTP
  - Traditional FTP vs. Anonymous FTP



1.1 PORT（主动模式）

在主动模式下，FTP 客户端随机开启一个大于 1024 的端口N向服务器的 21 号端口发起连接，发送 FTP 用户名和密码，然后开放 N+1 号端口进行监听，并向服务器发出 PORT N+1 命令，告诉服务端客户端采用主动模式并开放了端口。FTP 服务器接收到 PORT 命令后，会用其本地的 FTP 数据端口（通常是 20）来连接客户端指定的端口 N+1，进行数据传输。

1.2 PASV（被动模式）

在被动模式下，FTP 客户端随机开启一个大于 1024 的端口 N 向服务器的 21 号端口发起连接，发送用户名和密码进行登陆，同时会开启 N+1 端口。然后向服务器发送 PASV 命令，通知服务器自己处于被动模式。服务器收到命令后，会开放一个大于 1024 的端口 P（端口 P 的范围是可以设置的，后面会说到这个是很重要的）进行监听，然后用 PORT P 命令通知客户端，自己的数据端口是 P。客户端收到命令后，会通过 N+1 号端口连接服务器的端口 P，然后在两个端口之间进行数据传输。

1.4 两种模式的比较

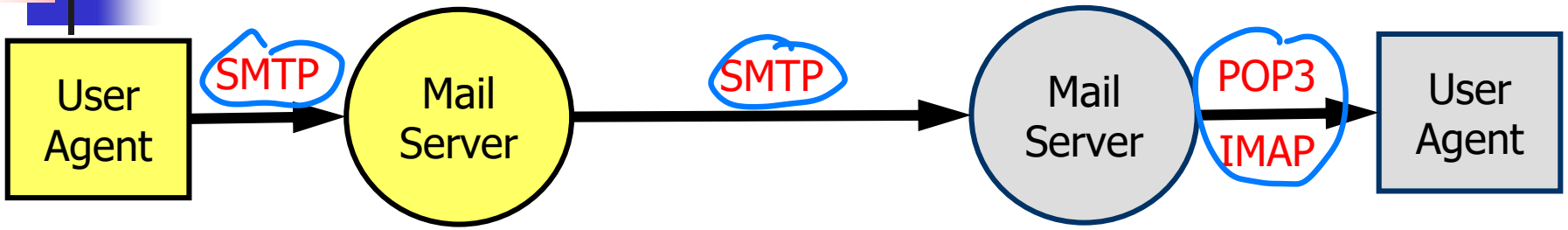
主动模式和被动模式的不同简单概述为：主动模式传送数据时是"服务器"连接到"客户端"的端口（客户端开启数据端口）；被动模式传送数据是"客户端"连接到"服务器"的端口（服务端开启数据端口）。

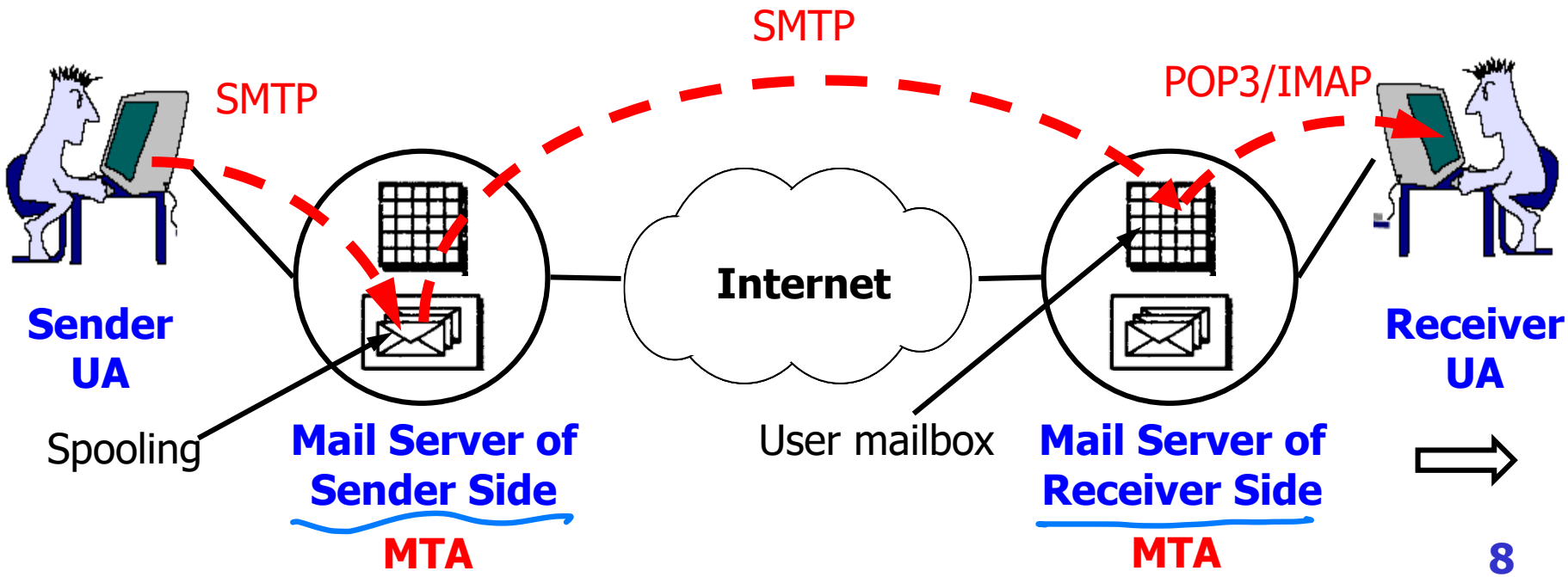
主动模式需要客户端必须开放端口给 FTP 服务端，很多客户端都是在防火墙内，开放端口给 FTP 服务器访问比较困难。

被动模式只需要服务器端开放端口给客户端连接就行了，如果服务端在防火墙内，也需要做端口映射才行。

EMAIL

Components Of Email System (1)





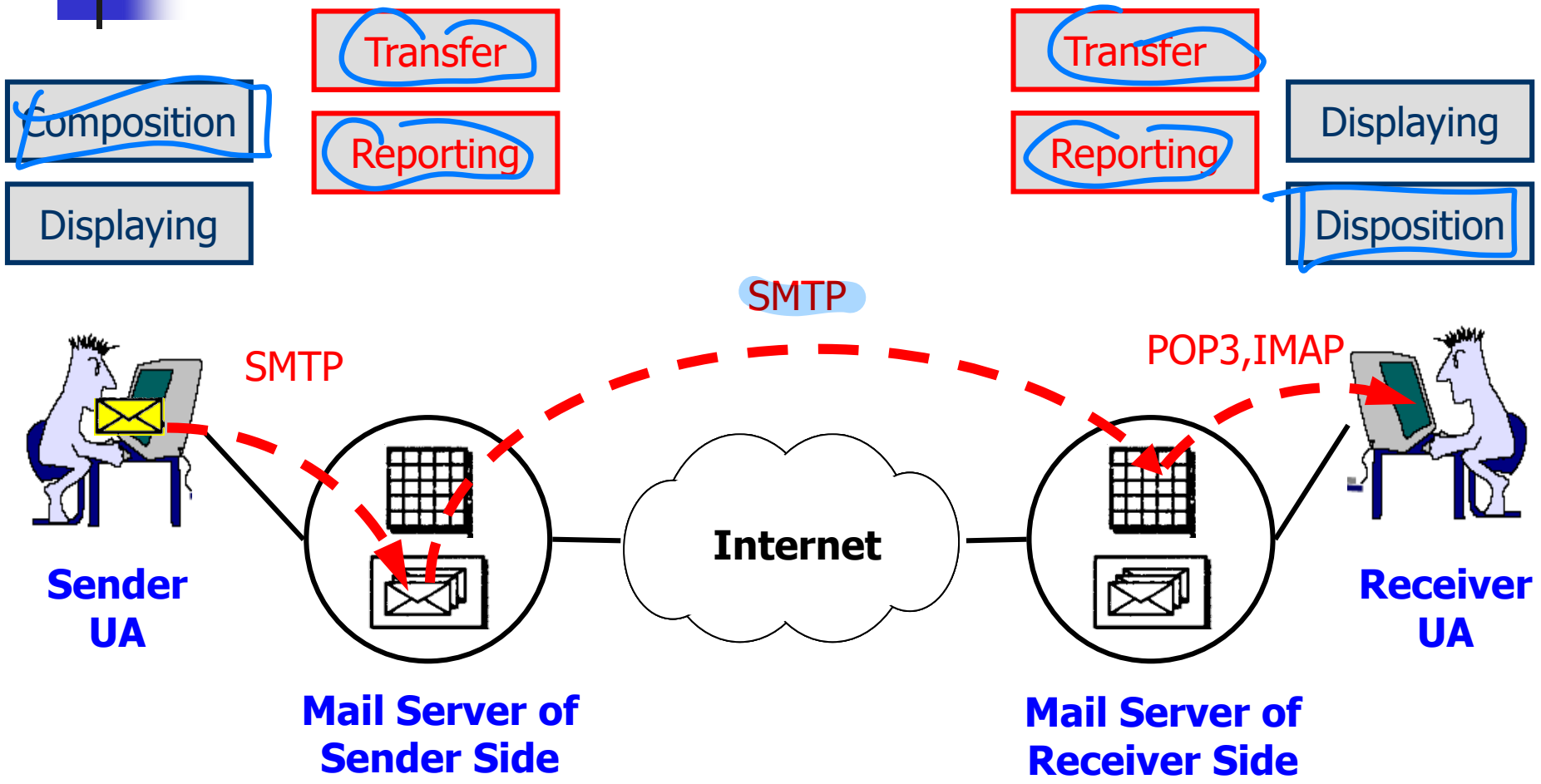
Components Of Email System (2)

- UA (User Agent)
  - end-user mail program
  - Interface between the end users and the email servers
  - E.g., outlook, foxmail, …



- Mail Server
  - Responsible for transmitting/receiving emails and reporting status information about mail transferring to the mail sender
  - **Both a client and a server**
- Email protocols
  - SMTP: used for sending an email
  - POP3/IMAP: used for receiving an email

Components Of Email System (3)



Basic Functions Of Email System

- *Composition* - refers to the process of creating messages and answers.
- *Transfer* - refers to moving messages from the originator to the recipient.
- *Reporting* - refers to the process of informing the originator what happened to the message.
- *Displaying* - means of showing the messages.
- *Disposition* - refers to what happened to the message after it has been read by the receiver.

读邮件后的处理

Other Terminologies

- *Mailboxes* – created by the user to store incoming email.
- *Mailing lists* – means of sending identical emails to a group
- *MTA (Mail Transfer Agent)* – SMTP servers and clients provide a mail transport service



Email Sending

- To send an email message, a user must provide the message, the destination address and possibly some other parameters (e.g., security or security level)

- Email Address
  - Many user agents expect DNS addresses of the form *mailboxname@domain*. (See RFC 5322)
  - Each email address is unique on the Internet because
    - *Domain name* is unique on the Internet
    - *Mailboxname* is unique in the domain

Internet Message Format (1)

- Message envelop
  - contains whatever information is needed to accomplish transmission and delivery
- Message contents: comprise the object to be delivered to the recipient
  - Headers: from, to, subject, date, postmarks
  - Blank line
  - Body: actual message, may have many parts

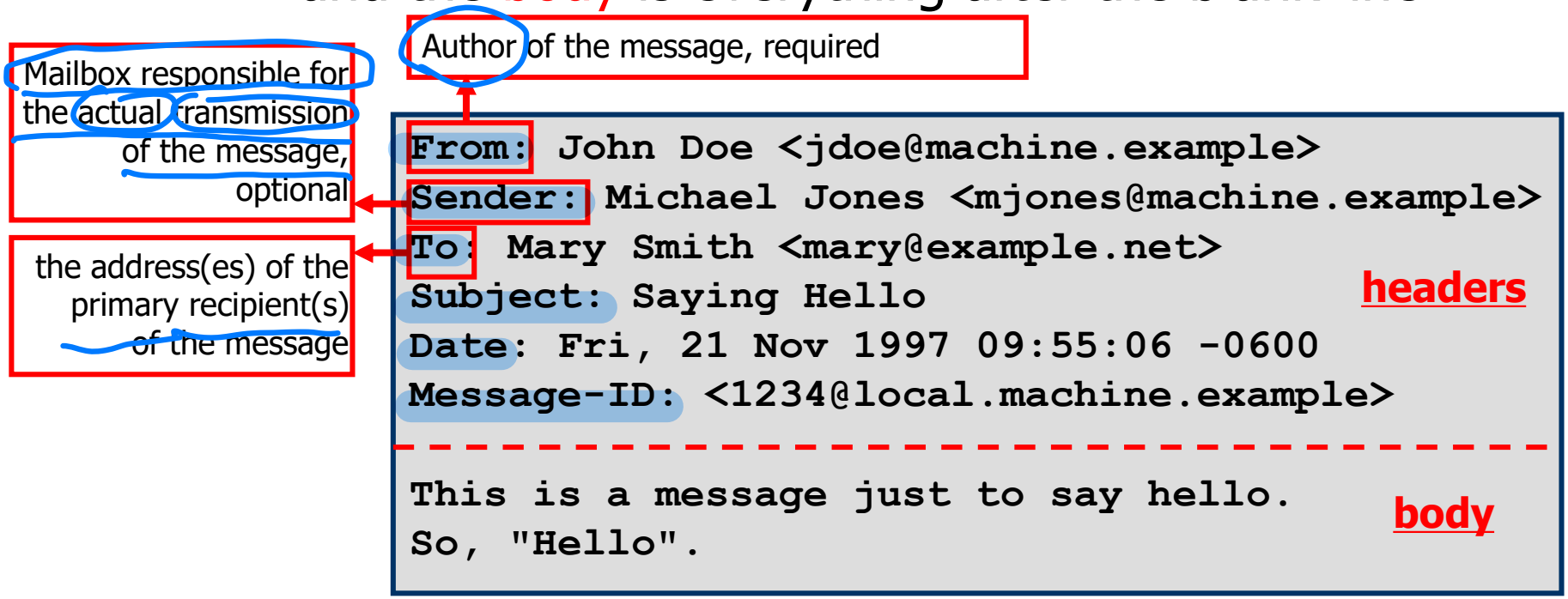
Internet Message Format (2)

- Each header field consists of a single line of ASCII text containing the field name, a colon, and, for most fields, a value
  - eg. from:abc@gmail.com
- In normal usage, the User Agent builds a message and passes it to MTA
- The MTA then uses some of the header fields to construct the actual envelope
- User provides body & key headers, while mail system provides the rest

用户提供body和header关键部分，其他由mail system填充

Internet Message Format ——Example RFC 5322 Message

- The header part is everything up to the blank line, and the body is everything after the blank line



Internet Message Format ——Field Definitions

Categories	Header	Meaning
Originator fields	From:	Person or people who created the message
	Sender:	Email address of the actual sender
	Reply-to:	Email address to which replies should be sent
Destination address fields	To:	Email address(es) of primary recipient(s)
	Cc:	Email address(es) of secondary recipient(s)
	Bcc:	Email address(es) for blind carbon copies
The origination date field	Date:	The date and time the message was sent
Identification fields	Message-Id:	Unique number for referencing this message later
	References:	Other relevant Message-Ids
Information fields	Subject:	Short summary of the message for the one-line display
	Keywords:	User chosen keyword
Trace fields	Received:	Line added by each transfer agent along the route
	Return-Path:	Can be used to identify a path back to the sender

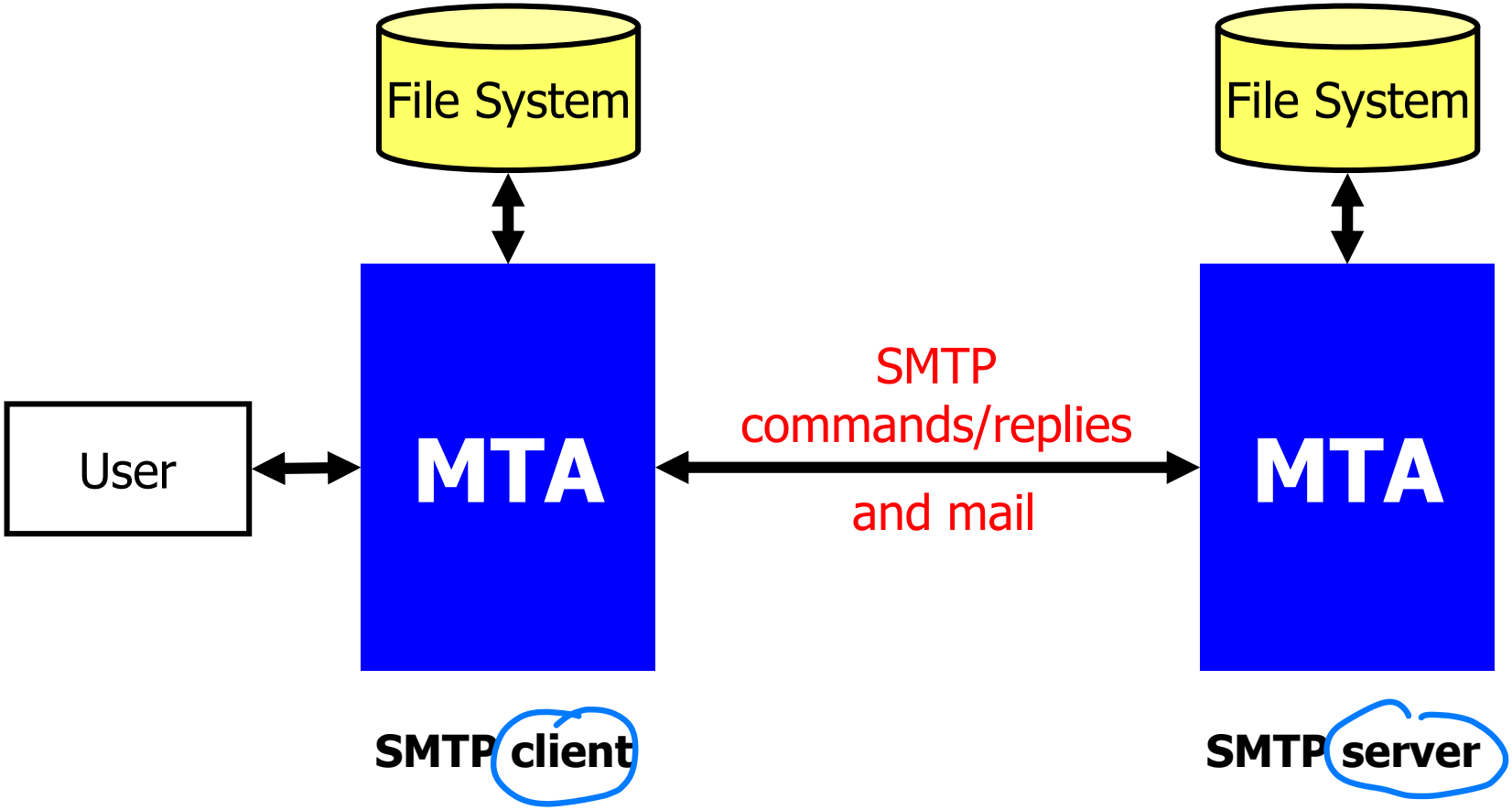
SMTP

- The source machine establishes a TCP connection to Port 25 of the destination machine
- Listening to this port is a SMTP server
  - Accepts incoming connections
  - Receives messages over the connections
  - If the message cannot be delivered, an error report containing the first part of the undeliverable message is returned to the sender
- SMTP is a simple ASCII protocol

不足以代表所有文件，如二进制文件

- After establishing the TCP connection to port 25, the sending machine, operating as a client, waits for the receiving machine operating as the server to talk first

SMTP Basic Model



SMTP Commands: Basics

commands	description
HELO	▪ identifies sender's Domain name
MAIL FROM:	▪ starts a mail transaction and identifies the mail originator
RCPT TO:	▪ identifies individual recipient. There may be multiple RCPT TO: commands
DATA	▪ Sender ready to transmit body of the message. A series of lines of text, each ends with **\r\n**. A line containing only a period '**.**' indicates the end of the data
QUIT	▪ close the connection

每次只能指定一个，但可以写多个RCPT TO

EHLO HELO+返回支持的各种扩展列表

SMTP: Status Codes

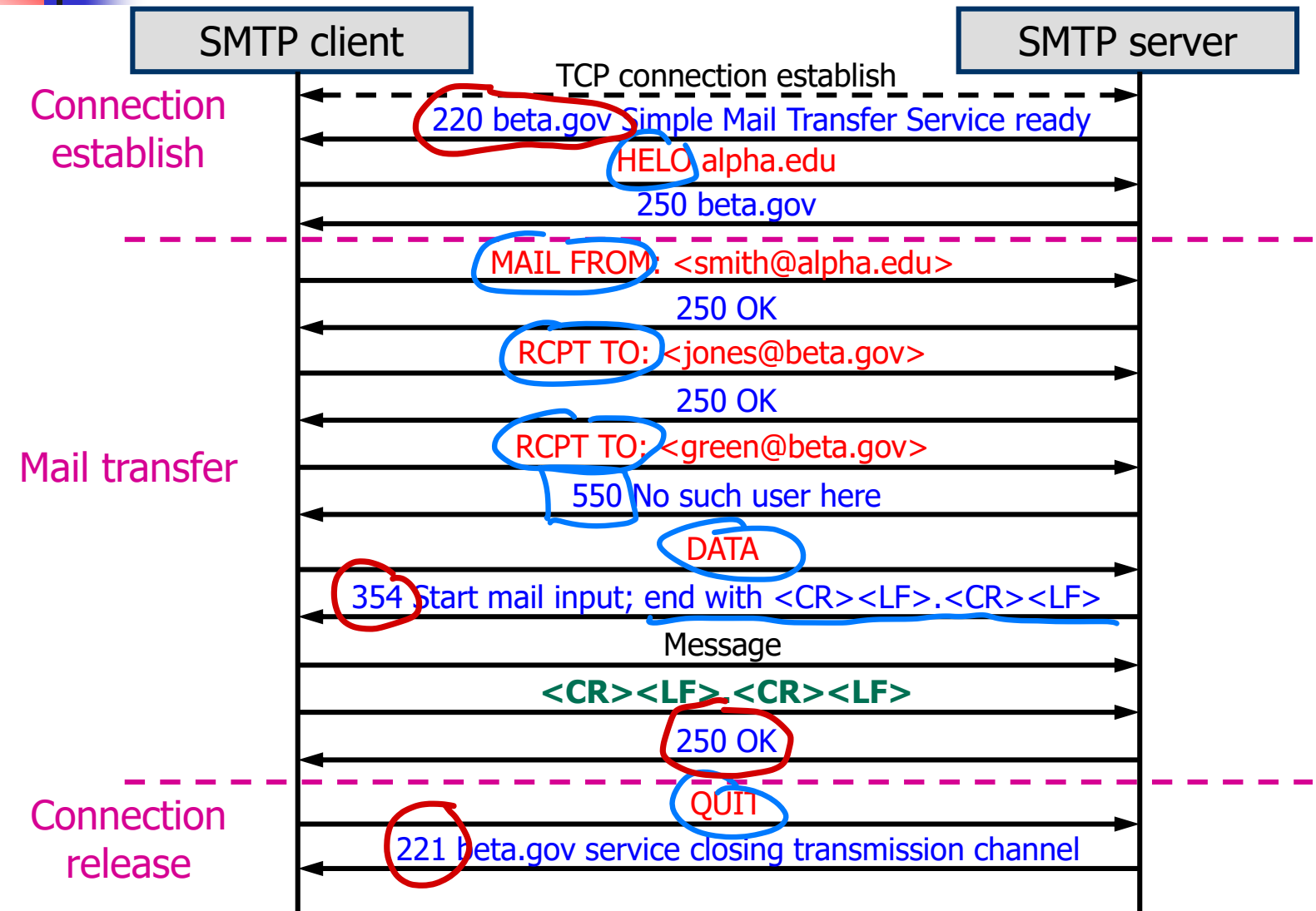
- The Server responds with a 3 digit code that may be followed by text info

Reply:

- 2## -- Success
- 3## -- Command can be accepted with more information 需要更多信息
- 4## -- Command was rejected, but error condition is temporary 暂时错误
- 5## -- Command rejected, and the error condition is permanent, eg. Bad User! 临时错误

SMTP Commands and Status codes – example

- smith@alpha.edu sends a mail to jones@beta.gov, green@beta.gov



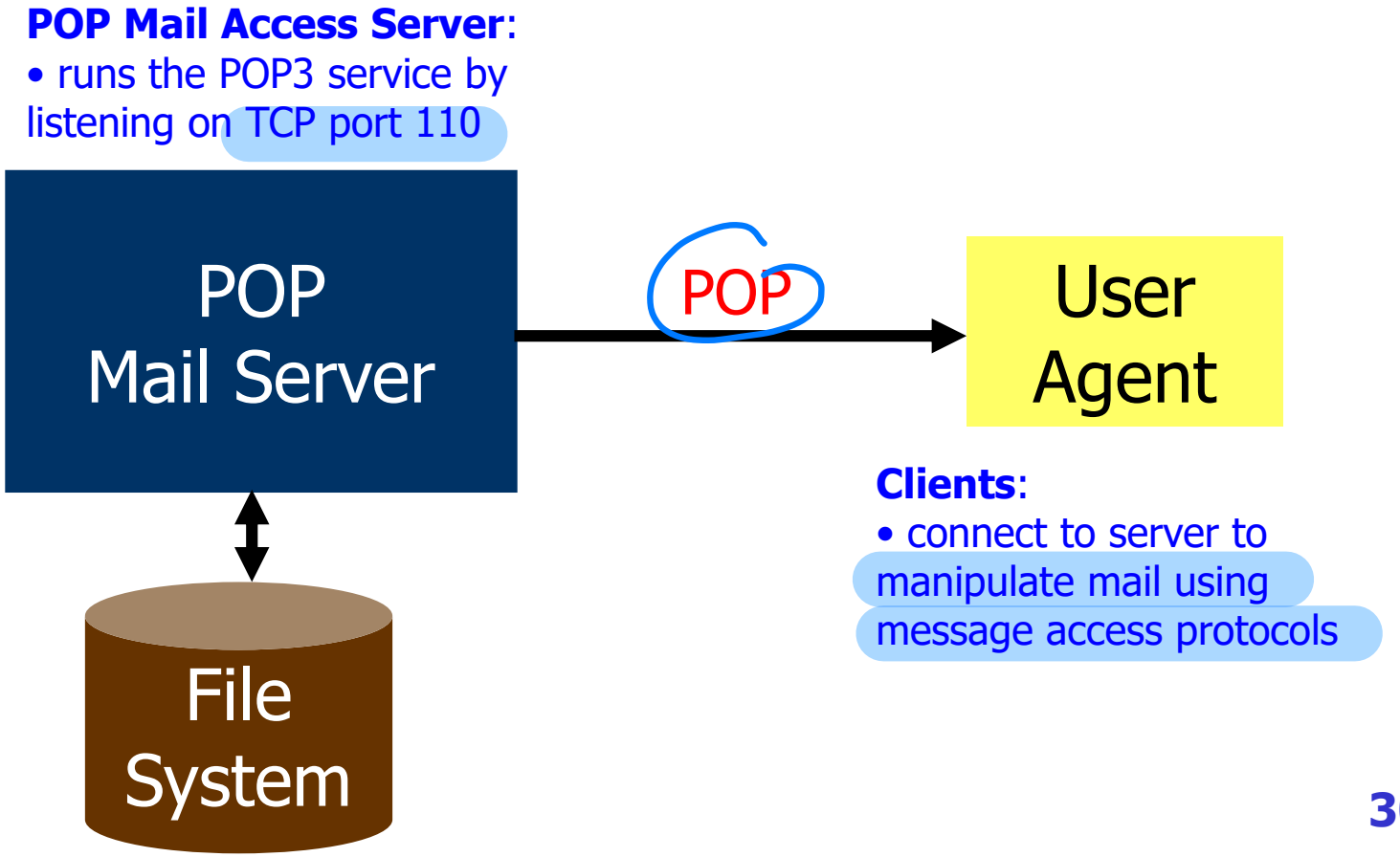
Limitations in SMTP

- Only uses ASCII format
  - How to represent other data types?
- No authentication mechanisms 无授权
- Messages are sent un-encrypted 无加密
- Susceptible to misuse (Spamming, faking sender address)

易被滥用

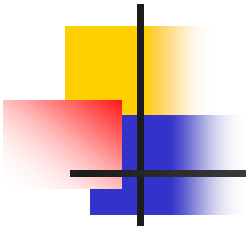
POP/IMAP – Basic Model

- Used to transfer mail from a mail server to a UA



POP – Features

- Essentially store and forward. Mail is stored on the server until the client connects and then is downloaded to the client. You MAY be able to leave a copy on the server
- Simple protocol and widely used.
- Many clients available such as Eudora, foxmail, outlook
- However, very bad for mobile users or users that use multiple machines during the day
- Common used version: POP3 (POP Version 3)



POP3

- Similar to SMTP command/reply lockstep protocol
- Used to retrieve mail for a single user
  - requires authentication
- Commands and replies are ASCII lines
  - Replies start with "+OK" or "-ERR"
  - Replies may contain multiple lines

POP3 Commands

- USER - specify username
- PASS - specify password
- STAT - get mailbox status
  - number of messages in the mailbox.
- LIST - get a list of messages and sizes.
  - One per line, termination line contains '.' only
- RETR - retrieve a message
- DELE - mark a message for deletion from the mailbox
- NOOP - send back positive reply
- RSET - reset. All deletion marks are unmarked
- QUIT - remove marked messages and close the (TCP) connection

Retrieving Emails Through Telnet (1)

```
C:\Documents and Settings\Administrator>telnet pop3.126.com 110
+OK Welcome to coremail Mail Pop3 Server
(126coms[3adb99eb4207ae5256632eecb8f8b485s])
USER catshiy
+OK core mail
PASS 123456
+OK 1 message(s) [885 byte(s)]
STAT
+OK 1 885
LIST
+OK 1 885
1 885
.
```

Retrieving Emails Through Telnet (2)

```
RETR 1
+OK 885 octets
Content-Transfer-Encoding: 8bit
MIME-Version: 1.0
Message-ID: <DQ958982777179.06131@mcard.bta.net.cn>
Date: Sun, 17 Oct 2004 22:28:20 +0800 (CST)
From: shp1234@public.bta.net.cn
To: catshiy@126.com
Cc:
Subject:

 我十一月中旬以后有空，欢迎你们过来玩。

shp
-----------------------------------------------

QUIT
+OK core mail

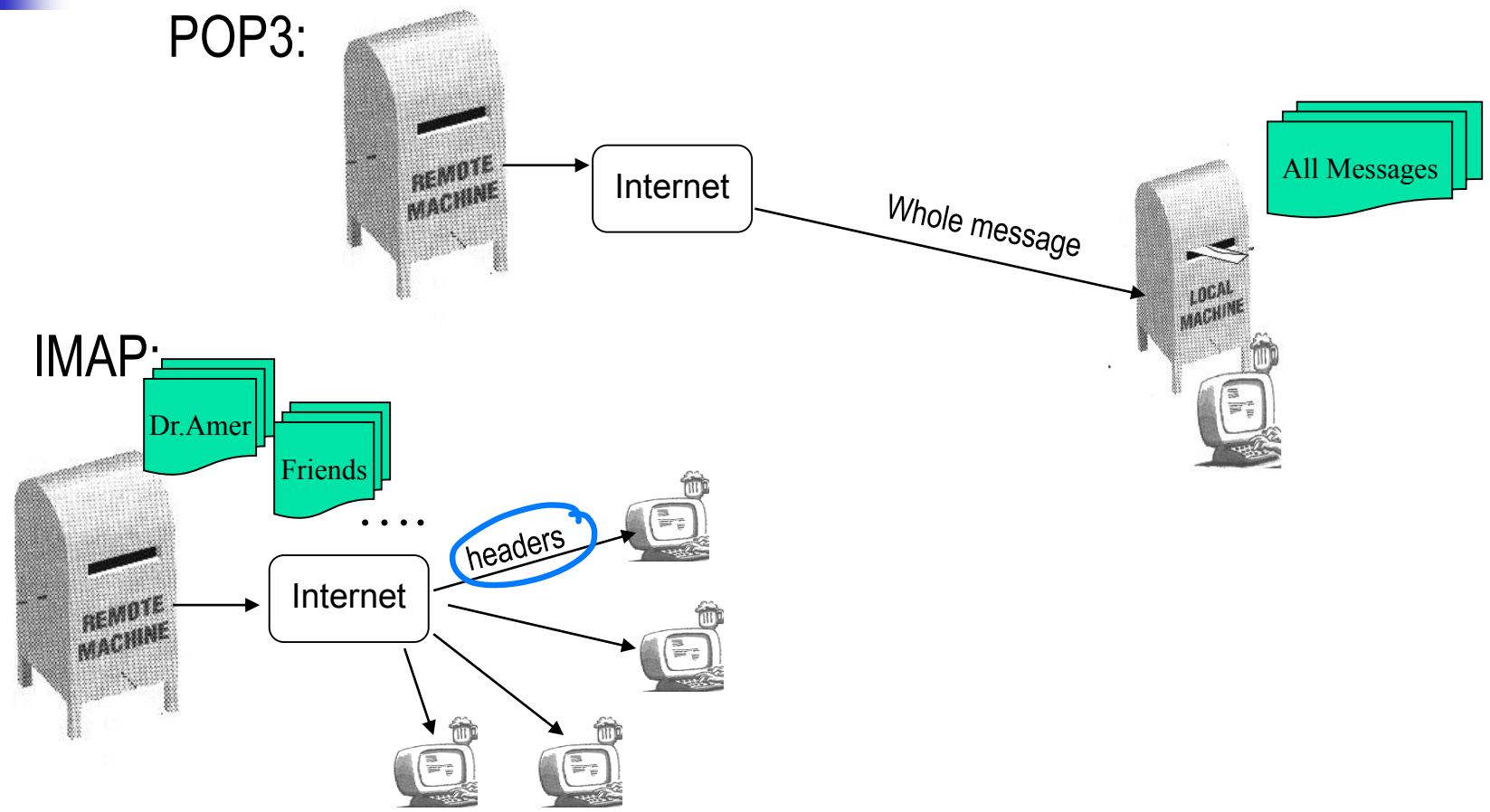
失去了跟主机的连接。

C:\Documents and Settings\Administrator>
```

IMAP (Internet Message Access Protocol)

- Features
  - Folders and messages can be stored either on the server or on the local computer
  - Since folders can remain on server, it is possible to access your same mail store even using a dumb terminal character based client like Pine.
  - Much better for mobile users than POP (since mail remains on the server)
  - Can selectively copy (part of) messages from the server to the local client based on many criteria
- QQ/163/gmail supports both IMAP and POP3
- Comparison of POP3 and IMAP
  - https://www.diffen.com/difference/IMAP_vs_POP3

POP vs. IMAP


POP3:
REMOTE MACHINE
Internet
Whole message
All Messages
LOCAL MACHINE
IMAP:
Dr.Amer
Friends
REMOTE MACHINE
Internet
headers

POP vs. IMAP(2)

Feature	POP3	IMAP
Where is protocol defined?	RFC 1939	RFC 2060
Which TCP port is used?	110	143
Where is email stored?	User's PC	Server
Where is email read?	Off-line	On-line
Mail Syncing	No	Yes
Direction	One-direction	Bi-directional
Good for mobile users?	No	Yes
Partial message downloads?	No	Yes
Speed	Fast	Low

MIME – Motivation

- Originally, email consisted exclusively of the text messages written in English and expressed in ASCII (RFC 5322)

- Nowadays, this approach is no longer appropriate, due to:
  - Messages in languages with accents (e.g. ö, ç, ğ)
  - Messages in non-Latin alphabets
  - Messages in languages without alphabets
  - Messages are not containing text at all -- audio/video

- MIME: Multipurpose Internet Mail Extension

MIME – Features

- Extension for multipart & multimedia email
- Additional mail headers define content
  - type (text, image, audio, video, application) and subtype within (eg text/html, image/gif)
  - encoding (ASCII , quoted printable, base64) to handle arbitrary binary data when email system can only handle normal ASCII chars
- Supports multipart message content type
  - each part has its own type and encoding
- The basic idea of MIME is to use the ASCII format (RFC 5322), but to add structure to the message body and define encoding rules for non-ASCII messages
- By not deviating from RFC 5322, MIME messages can be sent using the existing mail programs and protocols
- Widely used now

二进制文件先用编码后才转成ASCII文本文件

∵SMTP是基于文本的协议

MIME – New Headers (1)

Header	Meaning
MIME-Version:	Identifies the MIME version
Content-Description:	Human readable string telling what is in the message
Content-Id:	Unique identifier
Content-Type:	Nature of the message
Content-Transfer-Encoding:	How the body is wrapped for transmission

MIME – New Headers (2)

- *MIME-Version*: simply tells the user agent receiving the messages that it is dealing with MIME messages and which version of MIME it uses - any message not containing a MIME-version is assumed to an English plain-text message
- *Content-Description:* is an ASCII string telling what is in the message
- *Content-Id:* header uniquely identifies the content
- *Content-Transfer-Encoding:* tells how the body is wrapped for transmission - multiple schemes, from the the simplest - ASCII text, through to base64 encoding
- *Content-Type*: tells the type and subtype of the content

MIME – New Headers (3)

- Content types and subtypes

Type	Subtype	Description
Text	Plain	Unformatted text
	Richtext	Text including simple formatting commands
Image	Gif	Still picture in GIF format
	Jpeg	Still picture in JPEG format
Audio	Basic	Audible sound
Video	Mpeg	Movie in MPEG format
Application	octet-stream	An uninterpreted byte sequence
	Postscript	A printable document in PostScript
Message	RFC822	A MIME RFC 822(current RFC5322) message
	Partial	Message has been split for transmission
	External-body	Message itself must be fetched over the net
Multipart	Mixed	Independent parts in the specified order
	Alternative	Same message in different formats
	Parallel	Parts must be viewed simultaneously
	Digest	Each part is a complete RFC 5322 message

Summary

- Email
  - Components of email system
  - Basic functions of email system
  - Email address
- SMTP
  - Communication procedure
  - Model
  - Commands and replies
- POP
  - Model
  - Commands and replies
  - Communication procedure
- IMAP
  - Comparison of POP and IMAP
- Message formats
  - RFC 5322
  - MIME

Only uses NVT 7bit ASCII format - How to represent other data types? No authentication mechanisms; Messages are sent un-encrypted; Susceptible to misuse (Spamming, fakingsender address)....MIME: help send multipart and multimedia message.

- What are the limitations of SMTP? How are MIME and SSL used to offset the limitations of SMTP?



WWW Components

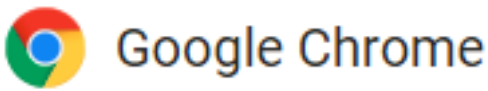
- Structural Components
  - Clients/browsers – various implementations
  - Servers – run on sophisticated hardware
  - Caches – used to improve response time
  - Internet – the global infrastructure which facilitates data transfer
- Semantic Components
  - Hyper Text Transfer Protocol (HTTP)
  - Hyper Text Markup Language (HTML)
    - eXtensible Markup Language (XML)
  - Uniform Resource Locators (URLs)

WWW Clients

- The Web is designed like all the client/server applications
  - The client is called a "browser"
  - The server is where the data is stored and it is software that runs on well known port (80) … usually
- The browser and server talk using a protocol – HTTP
- We already know from past experience that this architecture gives us client options
  - Netscape, Internet Explorer, Maxthon, Mozilla, Firefox, Lynx, …











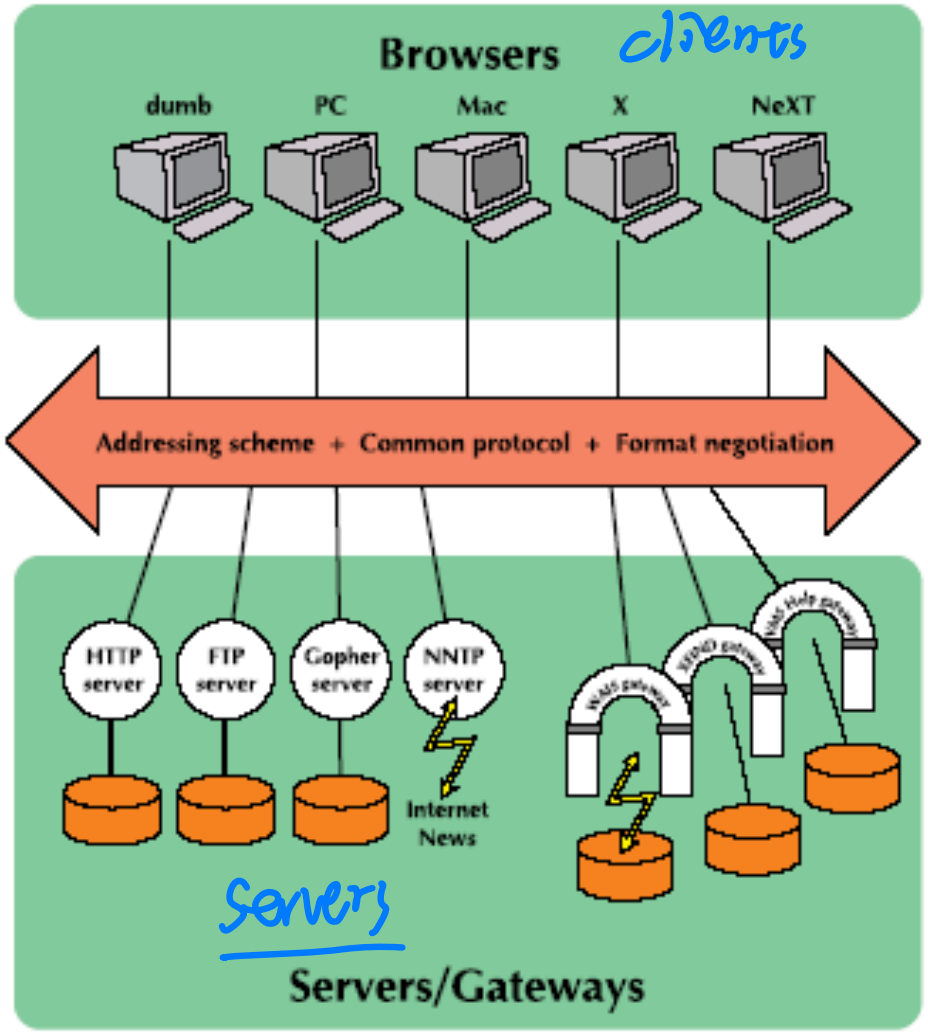


Basic Client Properties (1)

- All the different browsers show us the same information but they display it differently (depending on their capabilities)
- In front of each Web address there is an http:// to indicate to the browser that it is talking HTTP, the protocol of the Web
- A user on a client machine uses a browser to download a Web page by either entering a URL or clicking on a HyperLink

Basic Client Properties (2)

- Web browsers are often called Universal Clients because most can talk other protocols besides HTTP
  - ftp://home.domain: to use our Web browser as an FTP client
  - telnet://home.domain
  - ...
- The Web is capable of accessing data on many different Internet services:
  - Web pages, FTP, Email service, file directories, Telnet services, HTML, plain ASCII, etc.



WWW Servers (1)

- The server is software that is running on a remote location. Its job is to make "pages" available to the client - so when a client requests a page the server responds appropriately
- Web servers are typically on Unix or Windows NT boxes rather than on individual PCs
- Popular Web Servers:
  - On Unix - Apache, On Windows NT - IIS (Internet Information Server), Both - Netscape's Web Server

WWW Servers (2)

- Every Web site has a server process listening to TCP port 80 for incoming connections from clients – normally browsers
- After a connection has been established, the client sends one request and the server sends one response
- Then the connection is released
- The protocol that defines the legal request and response is HTTP
- The operation is Stateless

URLs (Uniform Resource Locators)

- The global address of a Web page is described by its URL
- URLs identify
  - the protocol you want to talk
  - the site (domain name or IP Address) you want to go to
  - possible the item you want to see
- They have the form:
  - **protocol://hostname [:port]/directory/item-you-want**

Resources can be dynamically generated on server upon query (Dynamic documents)

Structure Of URLs

- A URL consists of three parts:
  - The protocol – for example **http** or **ftp**
  - The DNS name of the host
  - The directory and file name



- Protocol: http by default
- Port: 80 by default 可更改.
- Index.html, index.htm, default.htm, default.asp etc. are assumed if no file-name given

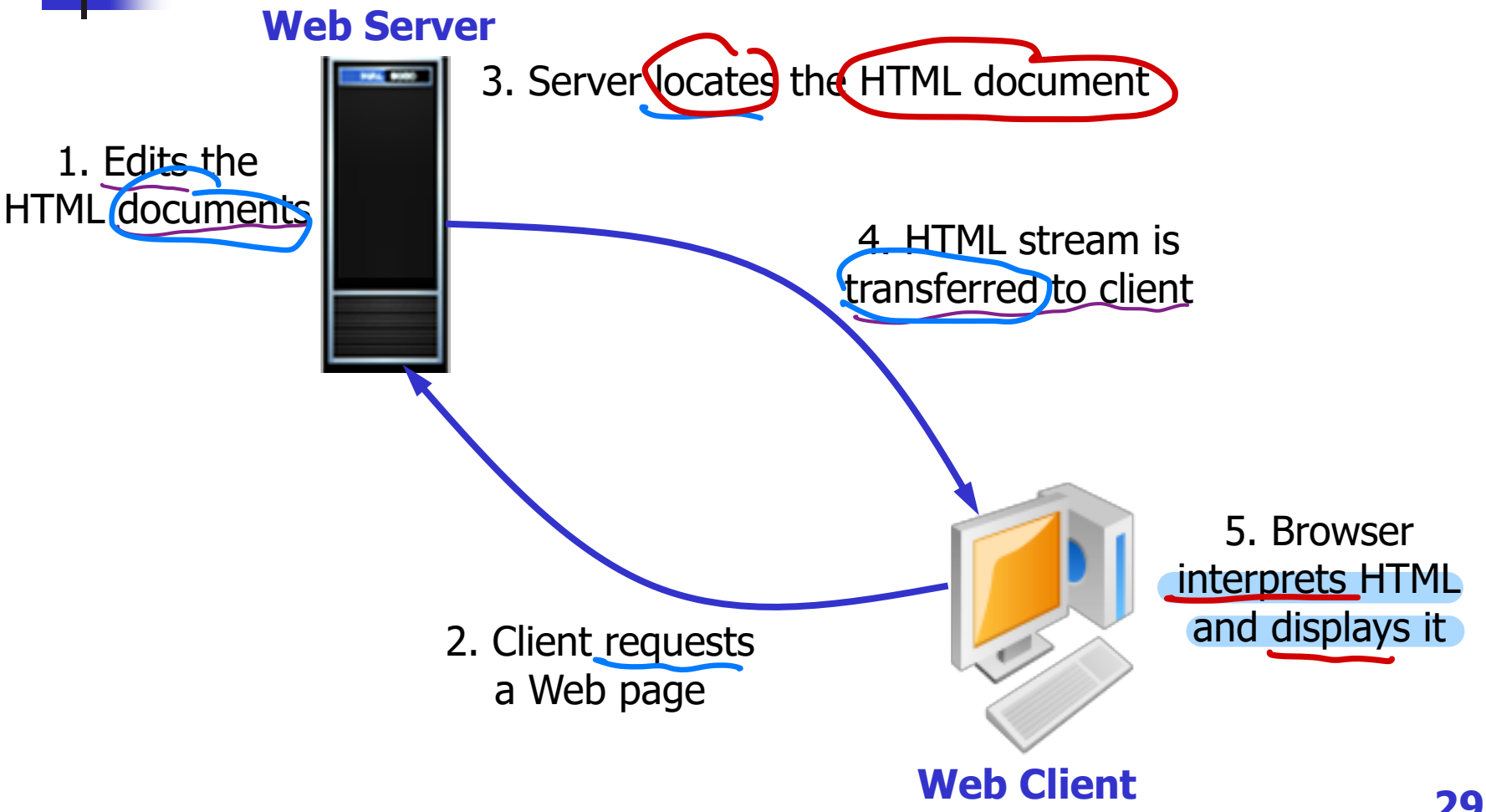
Some URLs Examples

Protocol	Use	Example
http	Web pages	http://www.elec.qmul.ac.uk
ftp	File transfer	ftp://elec.qmul.ac.uk/pub/info.doc
file	Local files	file://D:/src/multim/filter.txt
mailto	E-mail	mailto:cip@elec.qmul.ac.uk
telnet	Remote login	telnet://www.elec.qmul.ac.uk

Static vs. Dynamic

- At the beginning, WWW was made up of static documents
  - Each URL corresponded to a single file stored on some hard disk
  - Edit in HTML format
  - .html, .htm
- Today - many of WWW documents are built at request time
  - The URL doesn't correspond to a single file
  - Examples: website access counter, WWW based date-time server, BBS, …
  - Generated dynamically by ASP, JSP, VB Script, PHP, CGI or other programs
  - .asp, .shtm, .php, .cgi etc.
- Why dynamic documents?
  - automation of web site maintenance
  - customized advertising
  - database access
  - shopping carts
  - date and time service
  - jobs for ElecEng students

Procedure Of Static Pages



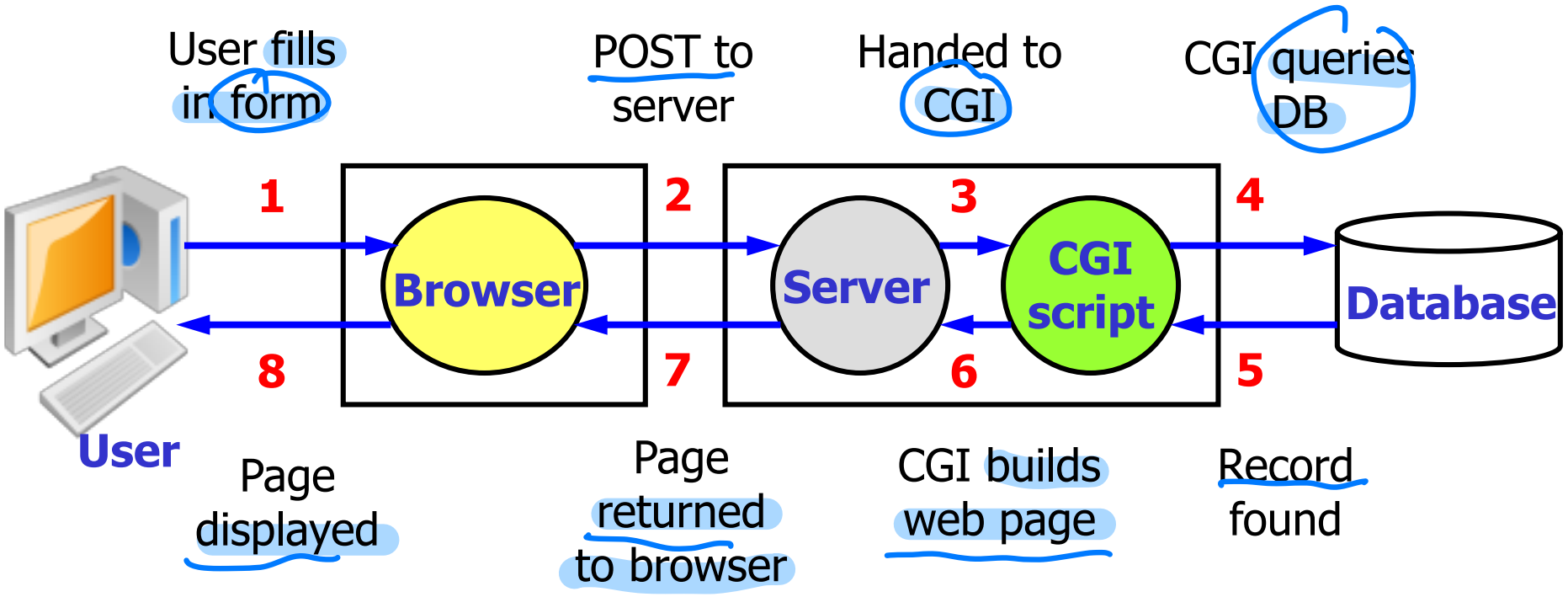
Procedure Of Server-based Dynamic Pages



CGI (Common Gateway Interface)

- The Common Gateway Interface (CGI) is a standard for interfacing external applications with information servers, such as HTTP or Web servers.
- A plain HTML document that the Web client retrieves is static, which means it exists in a constant state: a text file that doesn't change. A CGI program, on the other hand, is executed in realtime, so that it can output dynamic information for the server.
- The Web server executes a CGI program to transmit information to the database engine, receive the results and display them to the client. This is an example of a gateway. Currently version is 1.1.
- A CGI program is basically the equivalent of letting the world run a program on your system. For safety, security precautions are taken.

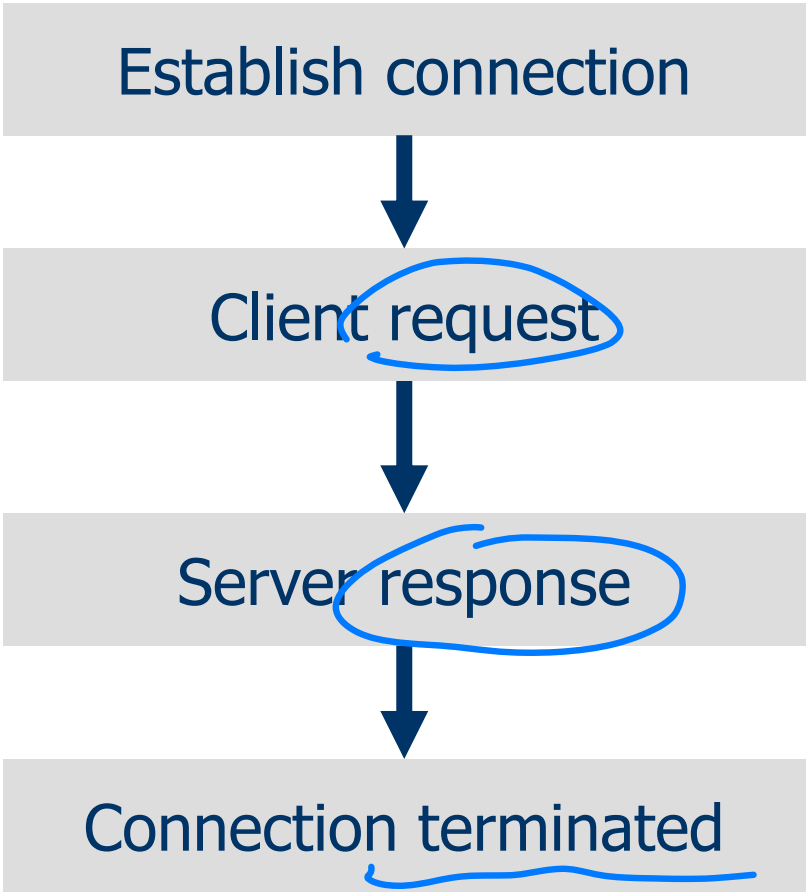
CGI – Procedure



HTTP – Basics

- The heart of the Web
- Features
  - Application layer protocol for client/server communication
  - Request/response based
  - Stateless
  - Bi-directional transfer
  - Capability negotiation
  - Support for cache
  - Support for intermediaries: HTTP proxy

HTTP – HTTP Transaction



- TCP connection set up
- uses a port number as application reference
- usually port 80

- HTTP message sent with a request line
- request-line = **method URL HTTP version**

- server sends HTTP message and optionally requested data
- resp-message = HTTP version status code reason-phrase [optional stuff]

- usually the server
- sometimes the client "stops" it
- anything else, whoever notices terminates

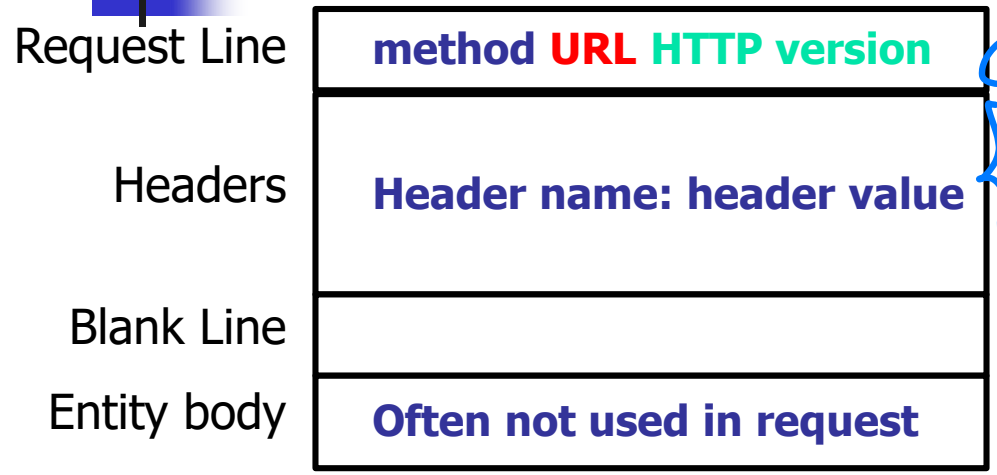
HTTP – Getting Remote Web Pages

- The browser determines the URL
- Browser asks DNS for the IP address of web-page being referred to
- DNS returns the IP address to the browser
- The browser makes a TCP connection to port 80 at the web-page IP address
- The browser sends a get request, eg.
  - GET /dir/FileName.html HTTP/1.0
- The remote server sends the file FileName.html
- The TCP connection is released
- The browser displays all the text in FileName.html
- The browser fetches and displays all the images in FileName.html

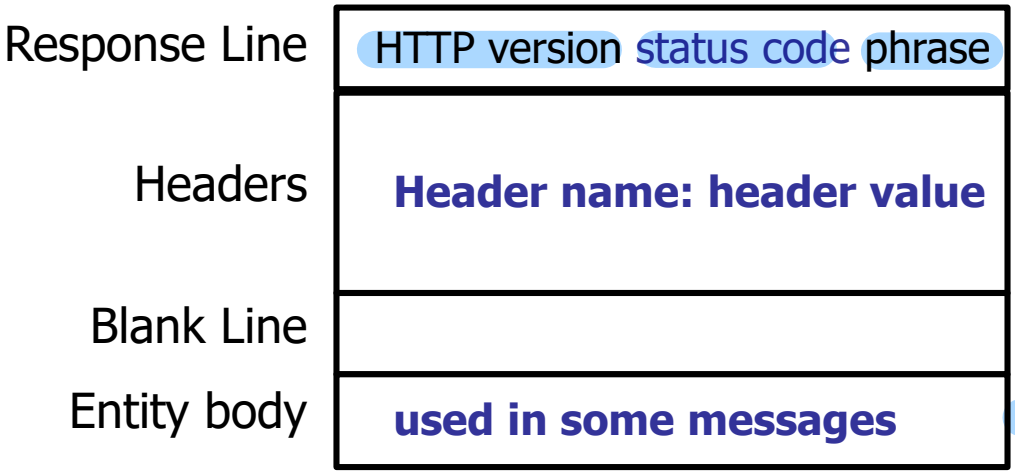
HTTP – HTTP Methods

Method	Description
GET	retrieve document specified by URL
PUT	store specified document under given URL
HEAD	identical to GET except that the server MUST NOT return a message-body in the response
OPTIONS	retrieve information about available options
POST	give information (eg. annotation) to the server
DELETE	remove document specified by URL
TRACE	loopback request message
CONNECT	reserved for use with a proxy

HTTP Request and Response Examples




```
GET /chn/yxsz/index.htm HTTP/1.1
Host: www.tsinghua.edu.cn
Connection: close
User-Agent: Mozilla/5.0
Accept-Language: cn
[blank line]
```




```
HTTP/1.1 404 Not Found
```

```
HTTP/1.1 301 Moved Permanently
Location: http://www.xyz.com/index.html
```

```
HTTP/1.1 200 Ok
[blank line]
<data>
```

HTTP/1.1 Performance Enhancements

- HTTP/1.0 is a "stop and wait" protocol
  - Separate TCP connection for each file
    - Connect setup and tear down is incurred for each file
    - Inefficient use of packets
    - Server must maintain many connections
- HTTP/1.1 specification focus on performance enhancements
  - Persistent connections
  - Pipelining
  - Enhanced caching options
  - Support for compression

HTTP/1.1 Persistent Connections and Pipelining

- Persistent connections
  - Use the same TCP connection(s) for transfer of multiple files
  - Reduces packet traffic significantly

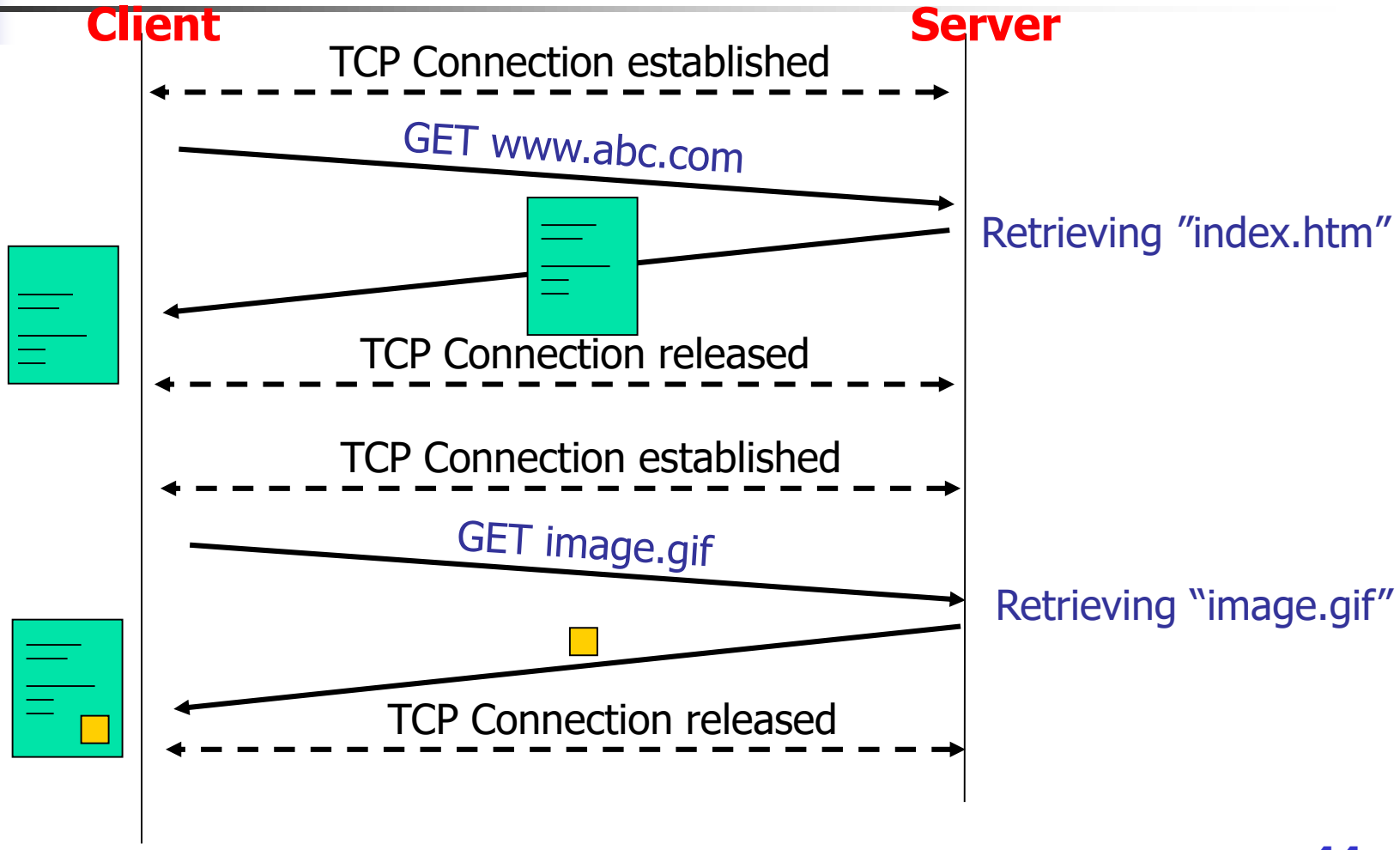
- Pipelining

一个socket发多个HTTP请求

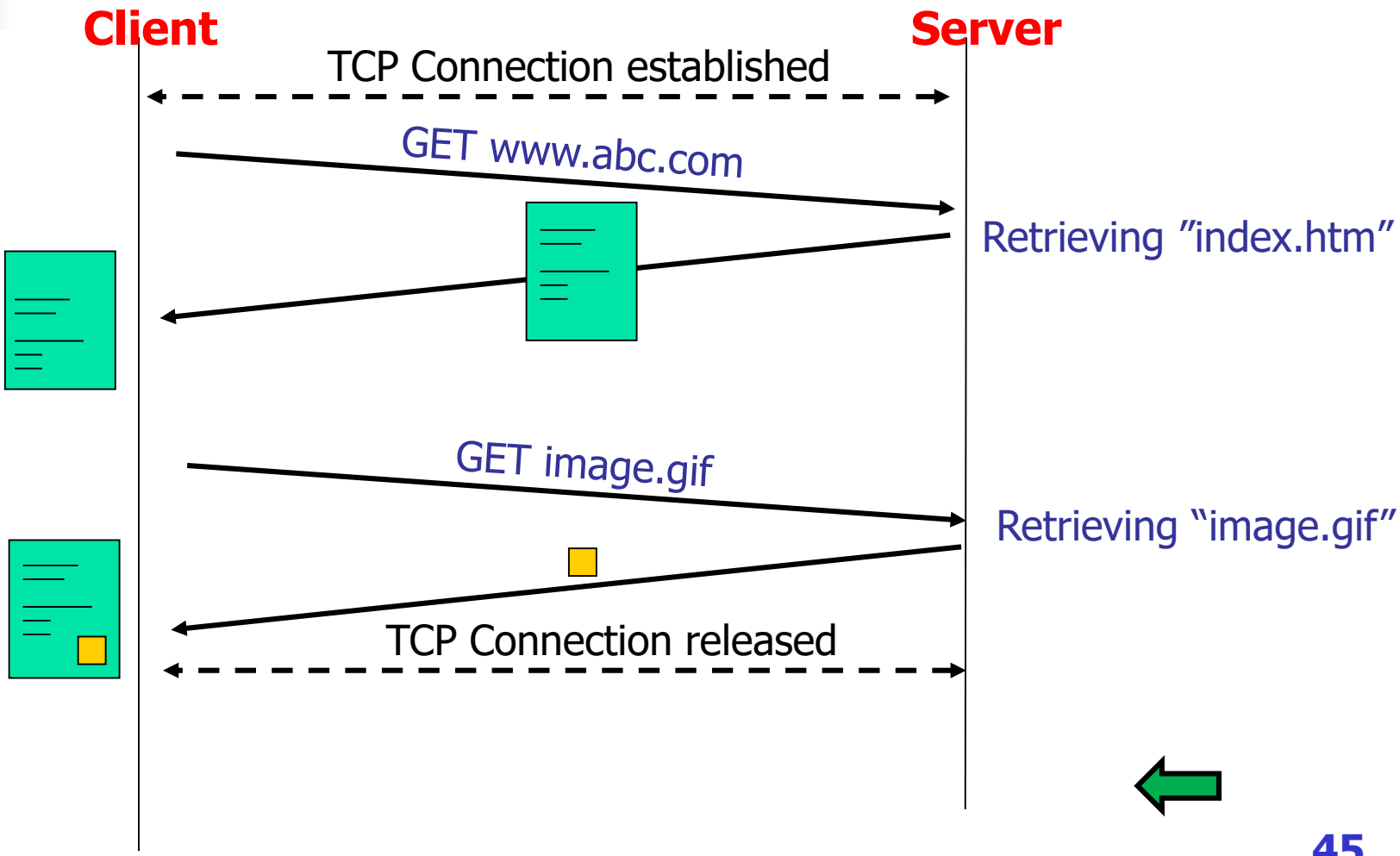
  - Multiple HTTP requests can be written out to a socket together without waiting for the corresponding responses.
  - Pack several HTTP requests into one TCP/IP packet

Example of Non-persistent connections

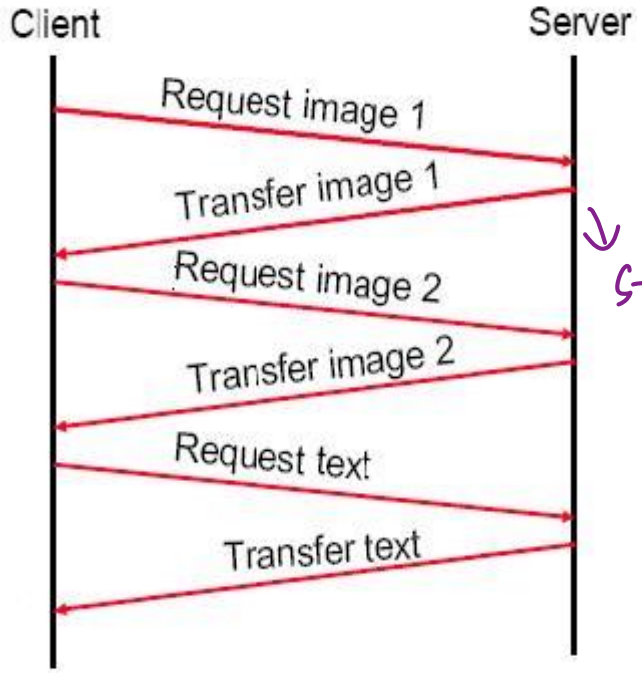
多次连接、每次都产生报文



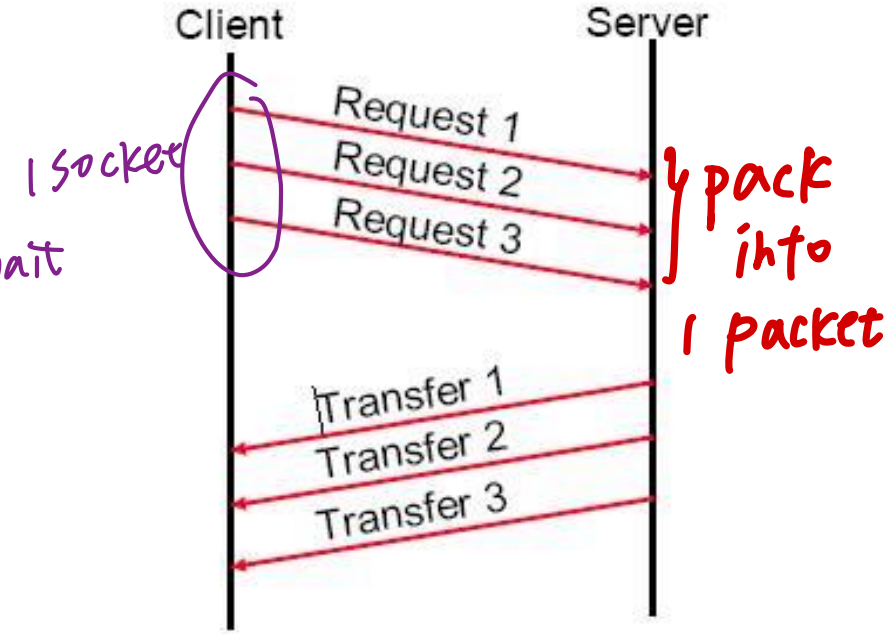
Example of Persistent Connections



Example of Pipelining



Non-pipelining



Pipelining

User-server state: cookies

keep the client state in server

- Many major Web sites use cookies

keep the state of clients in server

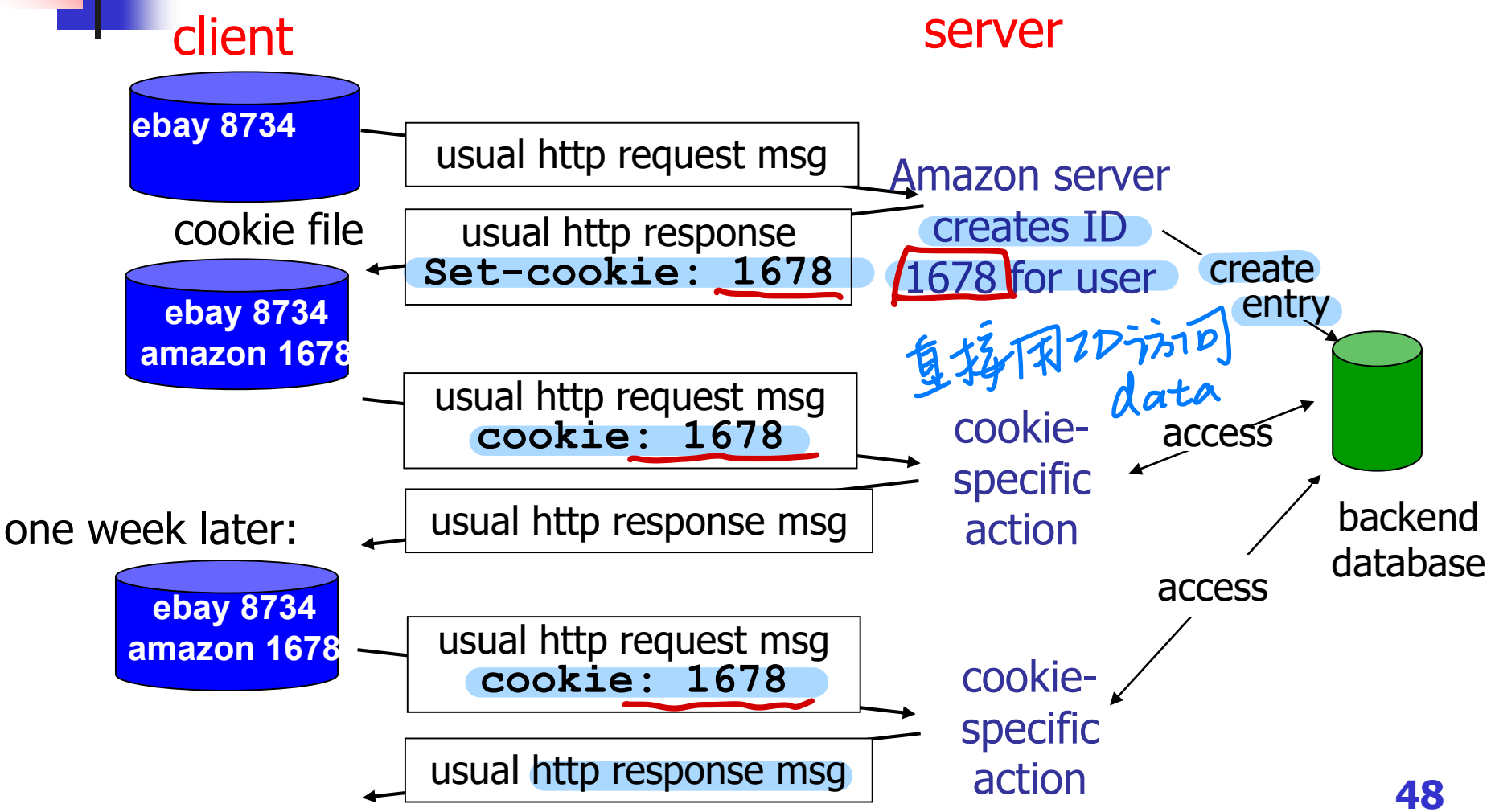
Example:

- Susan always access Internet from PC
- She visits specific e-commerce site for first time
- when initial HTTP request arrives at site, site creates:
  - unique ID
  - entry in backend database for ID

Four components:

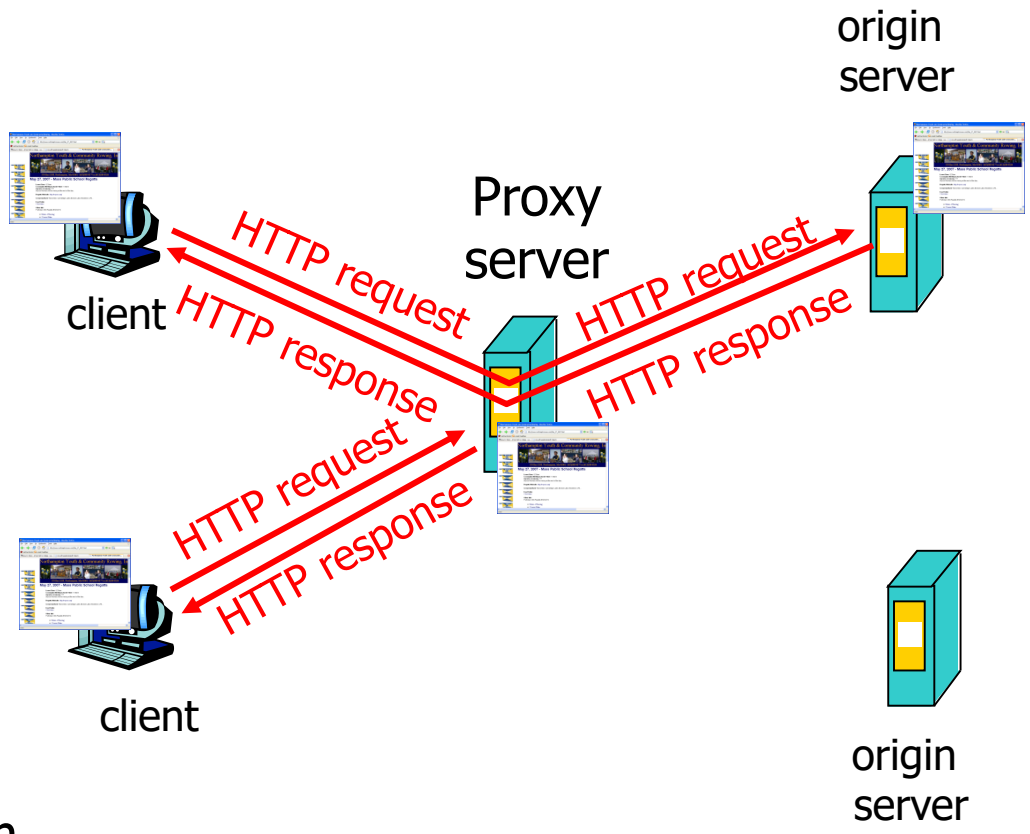
1) cookie header line of HTTP *response* message
2) cookie header line in HTTP *request* message
3) cookie file kept on user's host, managed by user's browser
4) back-end database at Web site

Cookies: keeping "state"



Web Caches (Proxy Server)

- Motivation: satisfy client request without involving origin server
- User sets browser: Web accesses via a proxy server
- Browser sends all HTTP requests to proxy server
  - If requested file in cache: proxy server returns file
  - else proxy requests file from origin server, then forwards to client

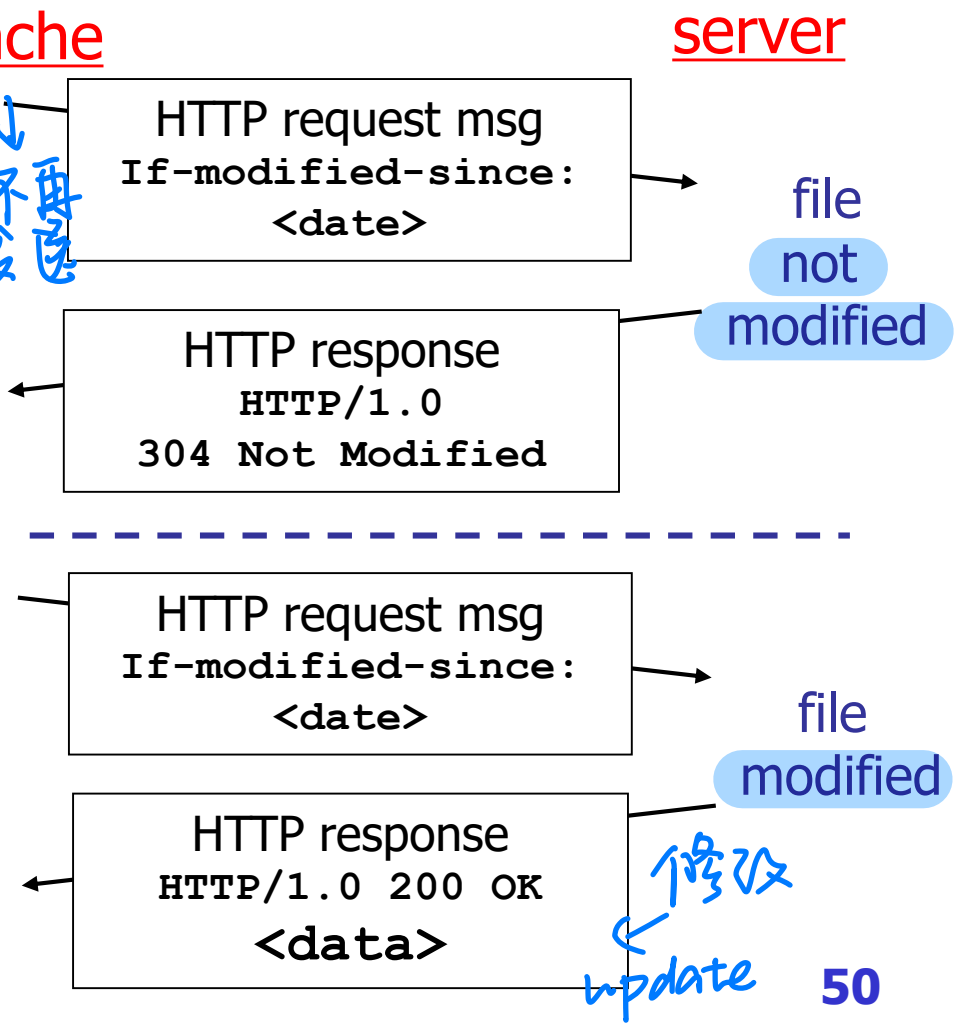


Conditional get

- Server does not send required files if cache has up-to-date cached version
- cache: specify date of cached copy in HTTP request

  `If-modified-since: <date>`
- server: response contains no object if cached copy is up-to-date:

  `HTTP/1.0 304 Not Modified`



SNMP

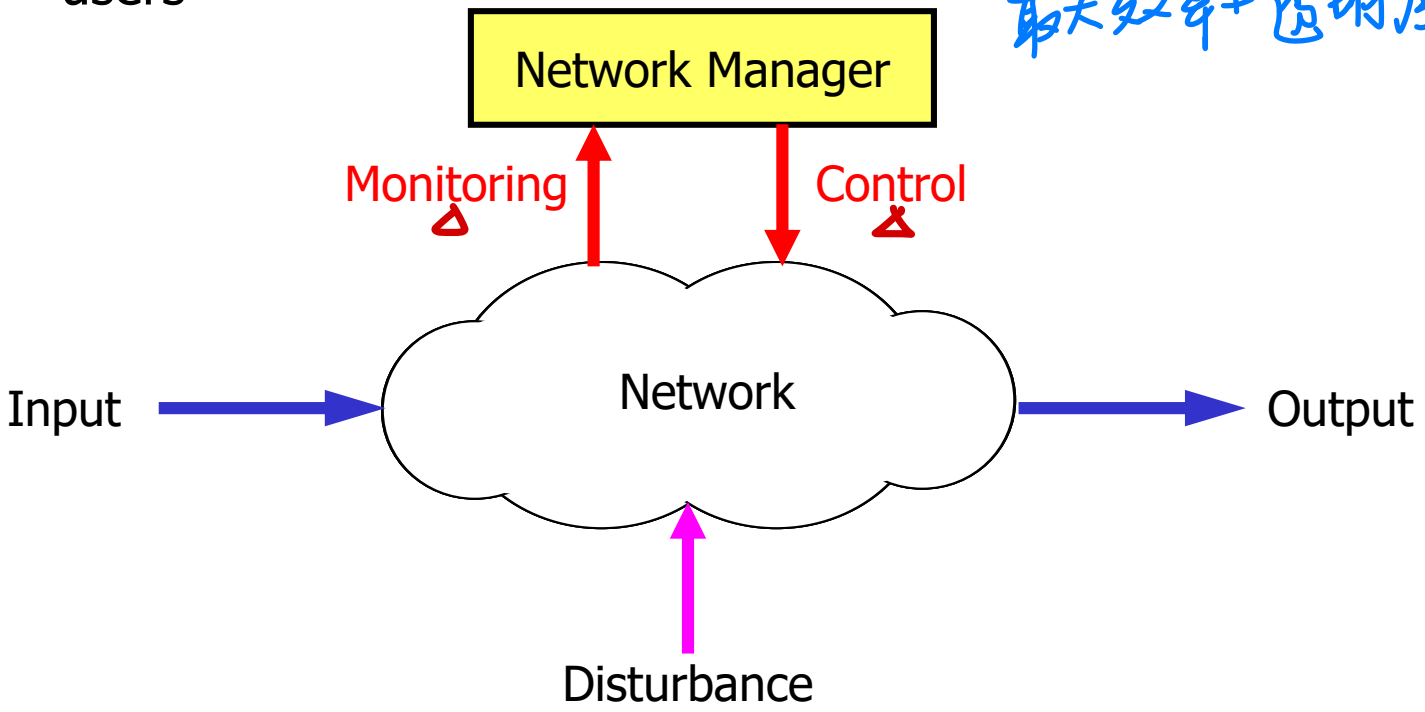
What is Network Management?

- Different things to different people, e.g.,
  - Monitoring network activity with protocol analyzer
  - Based on a distributed database, autopolling of network devices, generating real-time graphical views of network topology changes and traffic etc.
- Definition
  - Network management is a service that employs a variety of tools, applications, and devices to assist human network managers in monitoring and maintaining networks

The Goal Of Network Management

- The overall goal of network management is to help with the complexity of a data network and to ensure that data can go across it with maximum efficiency and transparency to the users

最大效率+透明度



Functional Areas Defined By ISO

- Defined by ISO Network Management Forum
- FCAPS
  - Fault Management
  - Configuration Management
  - Accounting Management
  - Performance Management
  - Security Management

FCAPS (1)

- Fault management
  - Is the process of locating problems, or faults, on the data network
  - It involves the following steps:
    - Discover the problem
    - Isolate the problem
    - Fix the problem (if possible)
- Configuration management
  - The configuration of certain network devices controls the behaviour of the data network
  - Configuration management is the process of finding and setting up (configuring) these critical devices

FCAPS (2)

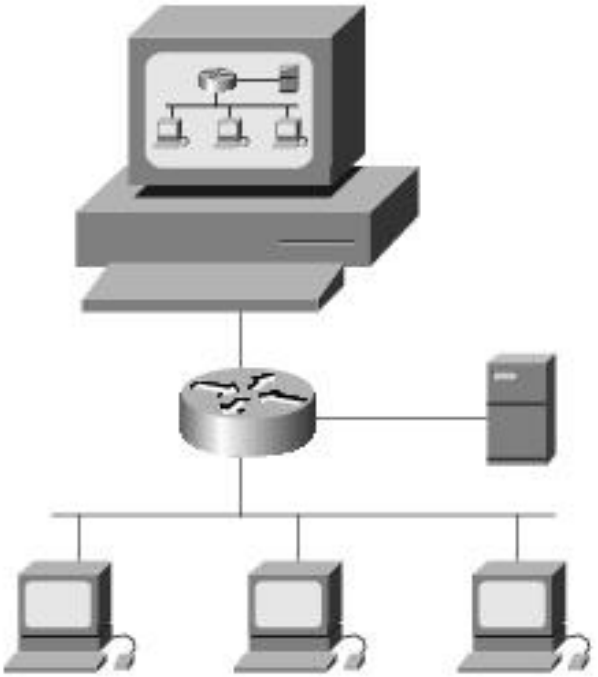
- Accounting management
  - Involves tracking individual's utilization and grouping of network resources to ensure that users have sufficient resources
  - Involves granting or removing permission for access to the network
- Performance management
  - Involves measuring the performance of the network hardware, software, and media
  - Examples of measured activities are:
    - Overall throughput
    - Percentage utilization
    - Error rates
    - Response time

FCAPS (3)

- Security management
  - Is the process of controlling access to information on the data network
  - Provides a way to monitor access points and records information on a periodic basis
  - Provides audit trails and sounds alarms for security breaches

What Is SNMP?

- Simple Network Management Protocol
- An application layer protocol that provides a way of monitoring and managing a heterogeneous computer network
- A part of TCP/IP protocol suite
- Based on client/server model
- Based on UDP
- Well-known ports
  - UDP Port 161: SNMP Get/Set Messages
  - UDP Port 162: SNMP Trap Messages



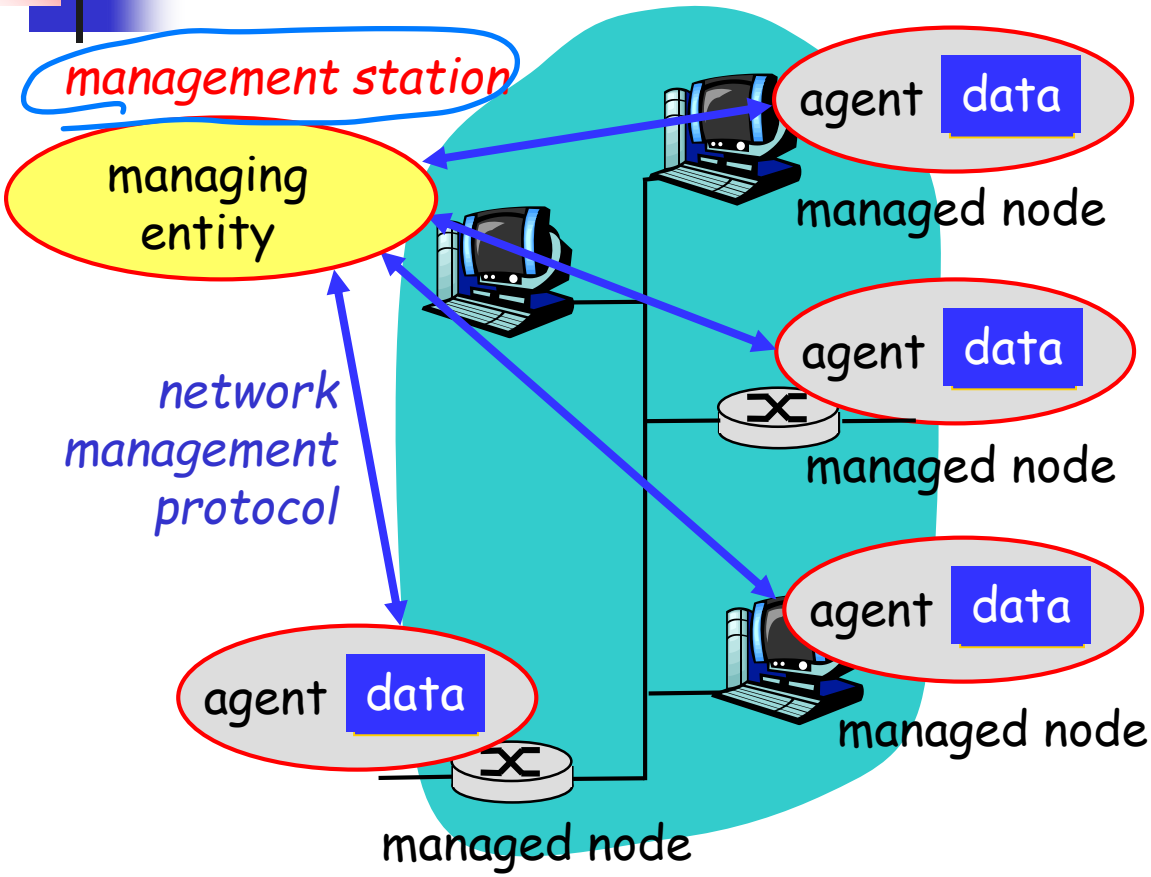
SNMP History (2)

- SNMPv1
  - *Basic function of read/write MIB*
- SNMPv2
  - *improve performance, security, confidentiality, and manager-to-manager communications*
- SNMPv3
  - *Security enhancement*
- RMON1
  - *Providing monitoring capability at data link layer in OSI model*
- RMON2
  - *Providing monitoring capability above data link layer in OSI model*

SNMP vs. Network Management

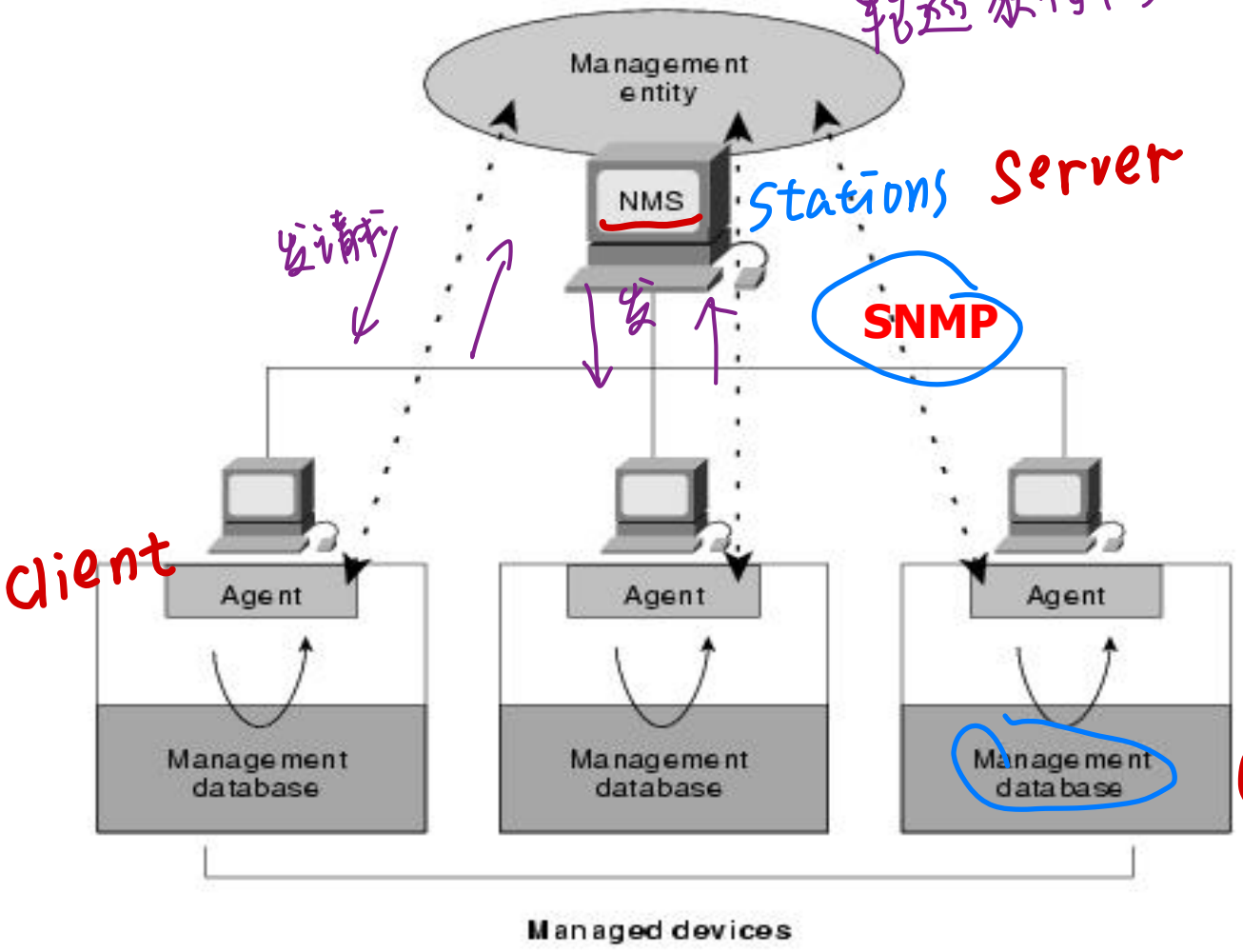
- SNMP realizes the F-C-P functions of network management
- SNMP does not cover all the function areas of network management
- Network management is a systematic work, in which SNMP is an important tool and protocol

SNMP Model (1)



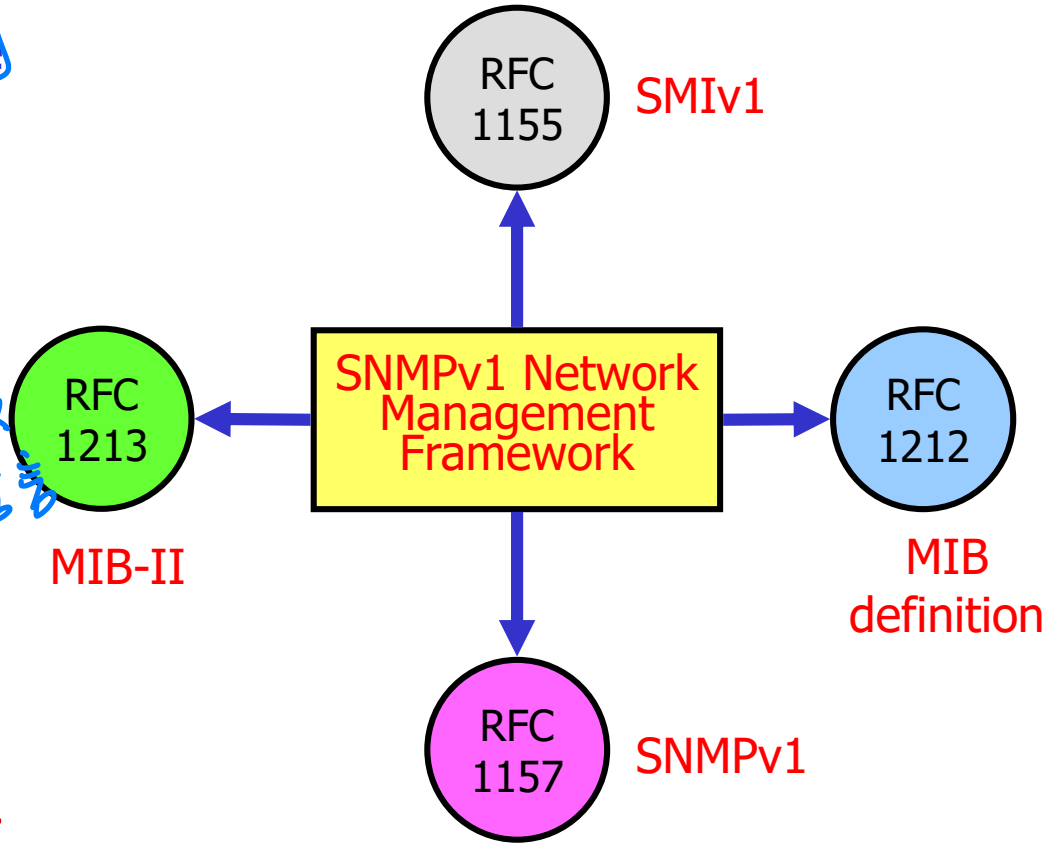
- The SNMP model of a managed network consists of four components:
  - Managed Nodes (Agent)
  - Management Stations (NMS) = manager
  - Management Information (MIB)
  - A Management Protocol (SNMP)

SNMP Model (2)
– more abstract description



SNMP Network Management Framework

- Management Information Base (MIB)
  - distributed information store of network management data
- Structure of Management Information (SMI)
  - data definition language for MIB objects
- SNMP protocol
  - convey information, commands between manager<->managed object



SMI: Structure of Management Information

- The *SMI* defines the rules for describing management information
- Syntax, semantics of management data, well-defined, unambiguous
- using ASN.1 (Abstract Syntax Notation One) for an unambiguous description without inconsistencies
- only a subset of ASN.1

MIB: Management Information Base

MIB是一个数据库，指明了被管理设备所维护的变量，是能够被Agent查询和设置的信息。MIB在数据库中定义了被管理设备的一系列属性：对象的名称、对象的状态、对象的访问权限和对象的数据类型等

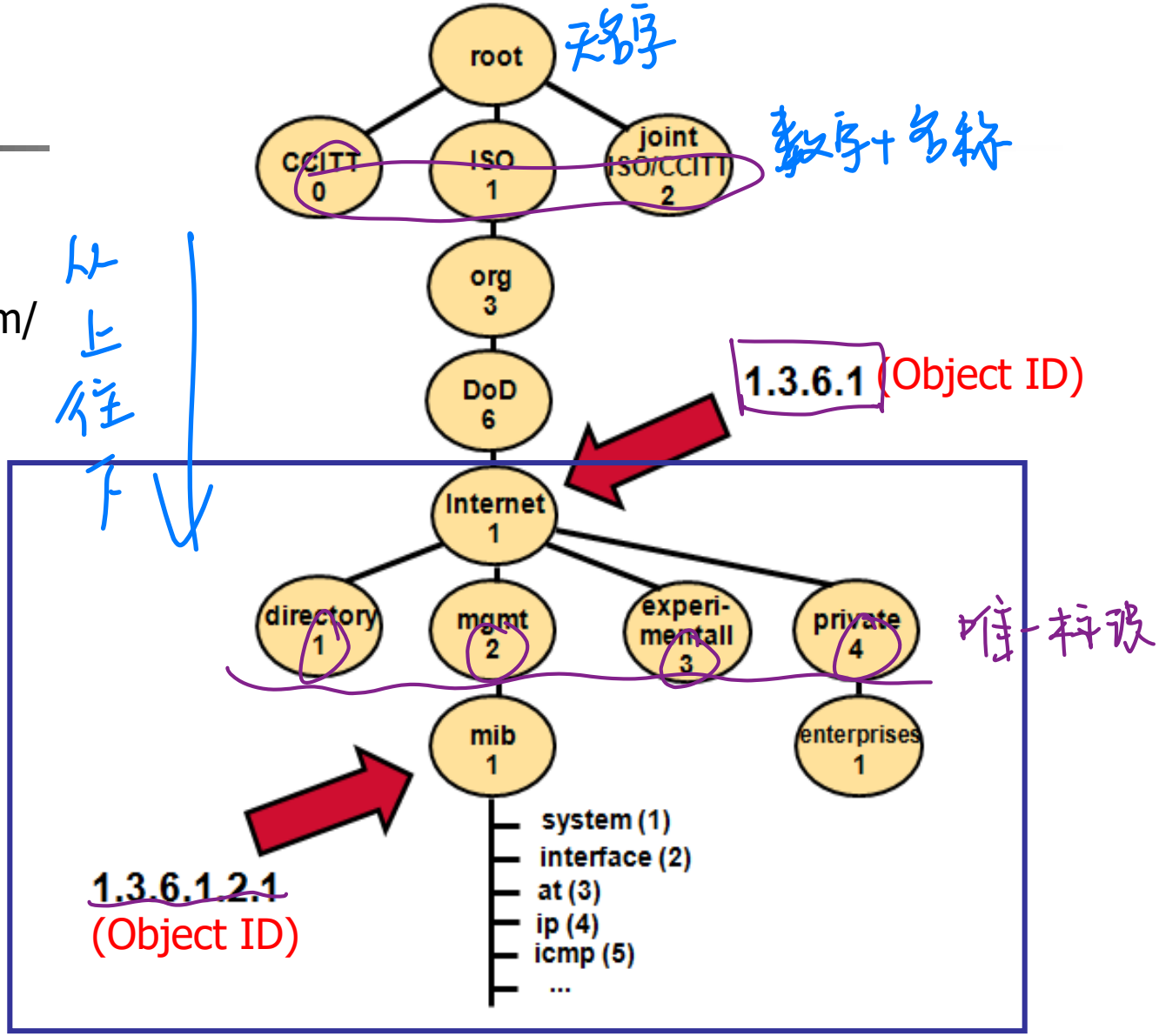
- A *MIB* is a collection of information that is organized *hierarchically*
  - MIBs are comprised of managed objects and are identified by *OIDs* (object identifiers)
- Two types of managed objects exist
  - *Scalar objects* define a single object instance
    - E.g., tcpInSegs, icmpInMsgs
  - *Tabular objects* define multiple related object instances that are grouped in MIB tables
    - E.g., udpTable, tcpConnTable, ipRouteTable
- *SMI* is the data definition language for MIB objects

MIB – ISO Object Identifier Tree

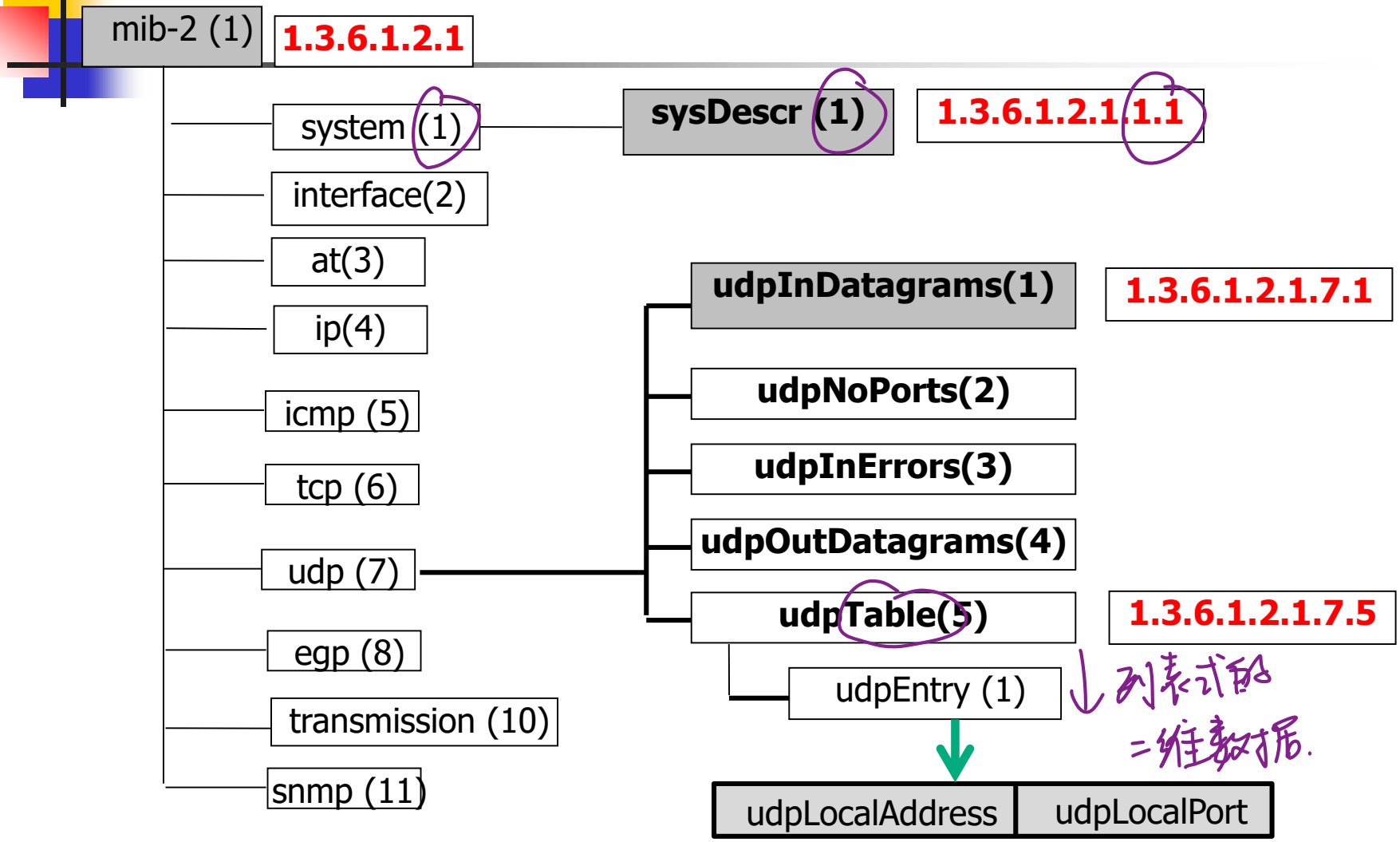
Check out:

http://www.oid-info.com/

Subtree of Internet object IDs

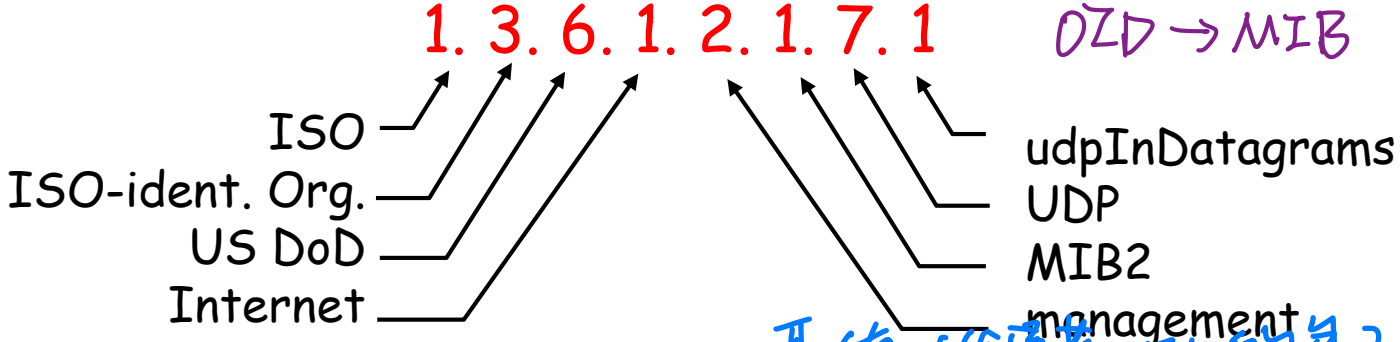


SMI MIB


mib-2 (1)
1.3.6.1.2.1
system (1)
sysDescr (1)
1.3.6.1.2.1.1.1
interface(2)
at(3)
ip(4)
icmp (5)
tcp (6)
udp (7)
egp (8)
transmission (10)
snmp (11)
udpInDatagrams(1)
1.3.6.1.2.1.7.1
udpNoPorts(2)
udpInErrors(3)
udpOutDatagrams(4)
udpTable(5)
1.3.6.1.2.1.7.5
udpEntry (1)
列表式的
二维数据.
udpLocalAddress
udpLocalPort

MIB – Naming

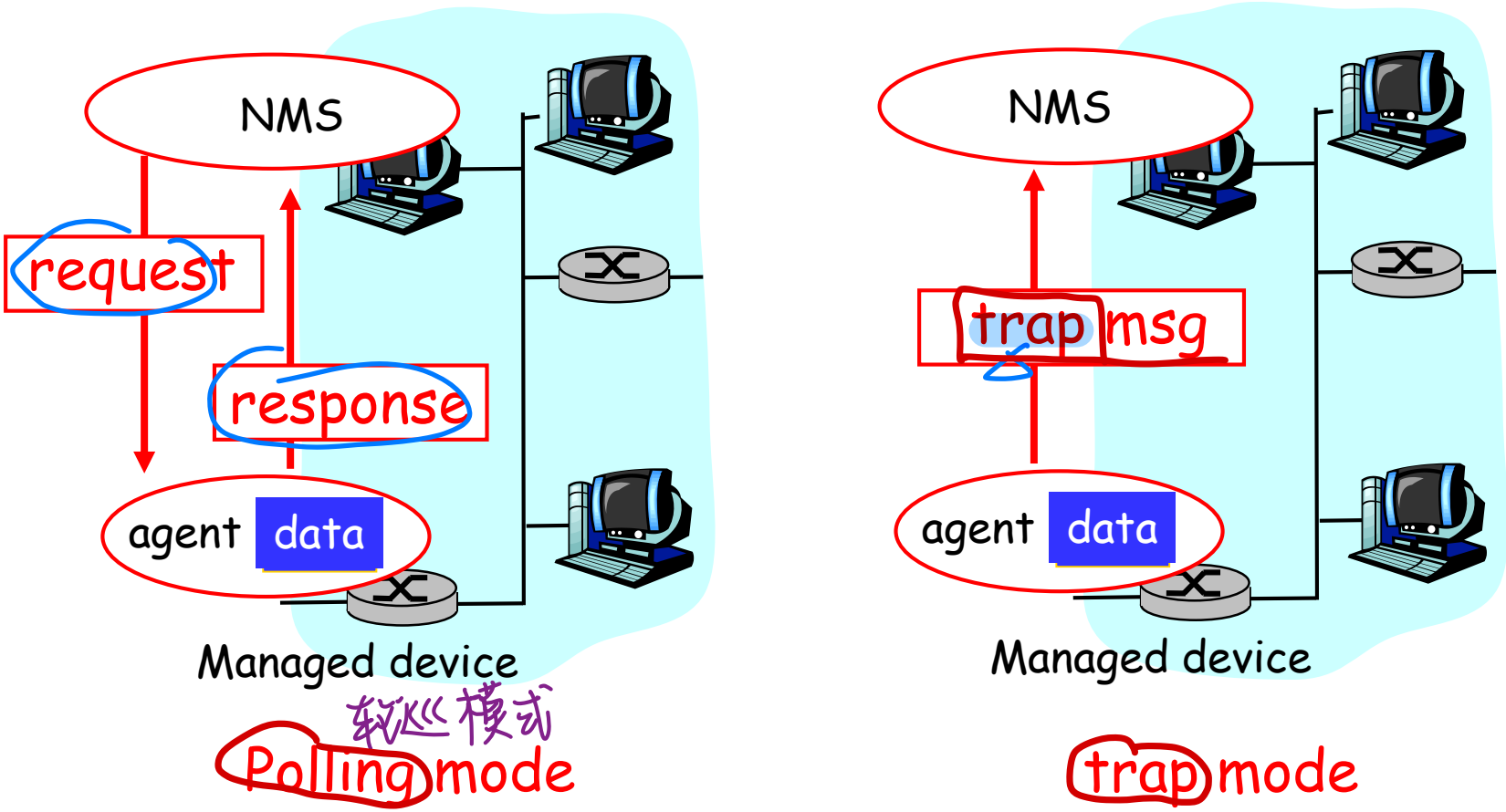
- Each object has a unique OID consisting of numbers separated by decimal points, and a more readable name. E.g.,
  - 1.3.6.1.2.1.7.1
  - iso.org.dod.internet.mgmt.mib.udp.udpInDatagrams



- When an SNMP manager wants to know the value of an object, it will assemble a GetRequest packet that includes the OID for that object.
- The agent receives the request and looks up the OID in its MIB. If the OID is found, a response packet is assembled and sent with the current value of the object. If the OID is not found, a special error response is sent

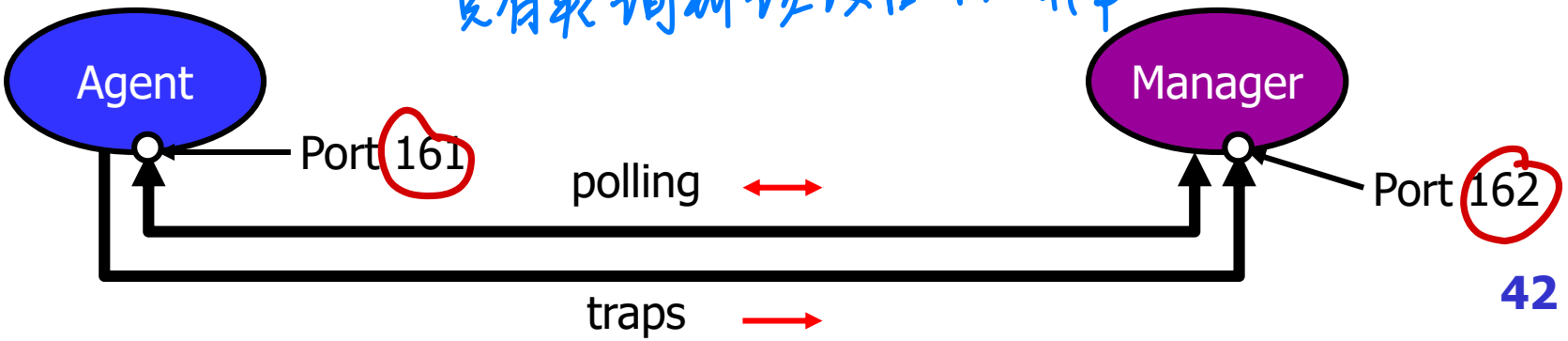
SNMP Traps / Polling (1)

- Two ways to deliver MIB information, commands



SNMP Traps / Polling (2)

- **Traps**
  - When abnormal event occurs, an agent sends a trap message to nominated NMS(s)
    - Trap indicates broad class of error [type], network device name and which object(s) should be queried for more information and time of event.
    - Hence keeps the message short and simple
  - NMS may then query the agent for more information on the named objects
  - NMS must be listening for TRAP messages
- **Polling**
  - The NMS periodically queries the network devices for information
  - The advantage is NMS is in control and knows the "big picture"
  - The disadvantage is the amount of delay from when an event occurs to when it's noticed

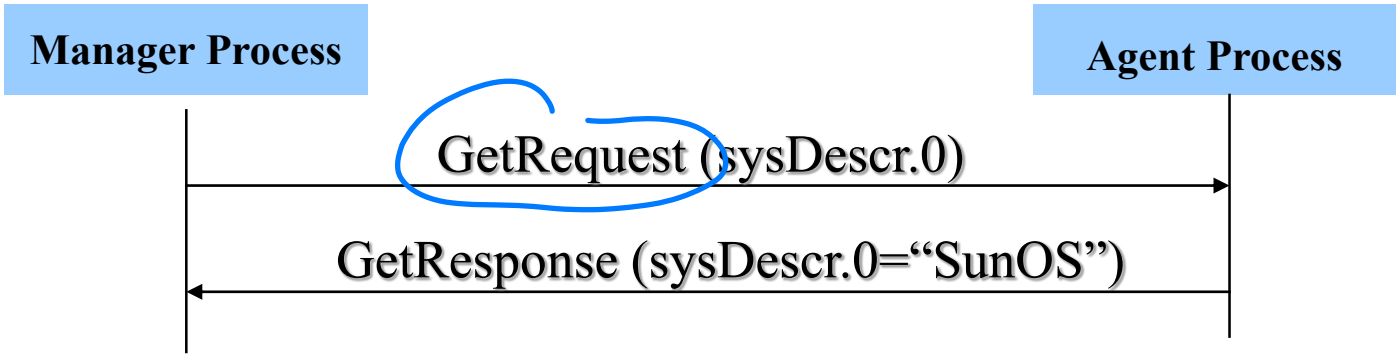


SNMP Commands

Command	Description	Version
GetRequest	NMS-to-Agent: get data (instance)	SNMPv1
GetNextRequest	NMS-to-Agent: get data (next in list)	SNMPv1
GetBulkRequest	NMS-to-Agent: get data (block)	SNMPv2
InformRequest	NMS-to-NMS: MIB information exchange	SNMPv2
SetRequest	NMS-to-Agent: set MIB value	SNMPv1
GetResponse	Agent-to-NMS: value, response to request	SNMPv1
Trap	Agent-to-NMS: report exceptional event to NMS	SNMPv1

GetRequest [Get]

- Most common PDU(Packet Data Unit).
- Used to ask SNMP agent for value of a particular MIB agent.
- NMS sends out 1 Get PDU for each instance, which is a unique OID string.



GetNextRequest

- Retrieves the NEXT variable instance existing on the agent in the tree of objects
- It either returns the next existing object, or error if none
- Can be used to traverse any part or all of the objects present on an agent
- Starting from the known mandatory **sysDescr** object, a NMS can find all others
- Simple, powerful mechanism
  - easy to implement on an agent, but
  - makes NMS do more work to discover necessary information